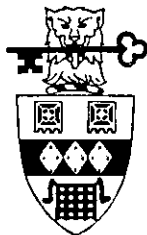


VECTOR

100+ pages of the Best in APL

- 16-page Educational Supplement 17
- New APL Statistics Section 33
- APL*PLUS/PC Release 10 reviewed 68
- PostScript Graphics in APL 76
- Jizba on OOPS and APL 109
- Cannon on APL Rodent Power 130



*The Journal of the
British APL Association*

ISSN 0955-1433

Vol.7 No.3 January 1991

A Specialist Group of the British Computer Society

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (ideally with a photograph of the author and a brief biographical note). Unless otherwise specified, each item will be considered for publication as a personal statement by the author.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette (any format). APL code can be accepted as camera-ready copy, or in workspaces from APL*PLUS/PC, IBM APL2/PC or Dyalog APL.

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the editor of your intention to re-use material from VECTOR.

Membership Rates 1990-91

Category	Fee	Vectors	Passes
UK Private	£10	1	1
Overseas Private	£18	1	1
(Supplement for Airmail)	£8		
UK Corporate Membership	£85	10+	5
Overseas Corporate	£140	10+	
Sustaining	£360	50+	5
Non-voting Student	£5	1	1

The membership year runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa or Mastercard at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready A5 portrait format with a 20mm blank border. Illustrations should be black-and-white photographs or line drawings. Rates are £250 per full page, £125 for half-page or less (There is a £50 surcharge per advertisement if spot colour is required).

Deadlines for bookings and copy are given under the Quick-reference Diary. Advertisements should be booked with, and sent to, Alison Chatterton, whose address is given on the inside back cover.

VECTOR 7.3

STOP PRESS

CONTENTS (Corrected):

Ray Cannon's article on Mandelbrot Sets has been held for Vector 7.4

Dyadic Systems Sustaining Members' News:

Arrived by Fax on 21st Dec at 2.00pm ... unfortunately we were all at the pub!

Would YOU like to help manage the British APL Association?

An Appeal to the members from the Hon. Sec.

CONTENTS

EDITORIAL: Solving Problems with APL	Jonathan Barman	Page 3
APL NEWS		
Quick Reference Diary		5
APL Training Courses 1991		7
General Correspondence		8
British APL Association News: Projects Report	John Searle	9
News from Sustaining Members	Alison Chatterton	13
The Education Vector	Alan Sykes	17
The Random Vector	David Eastwood	33
REVIEWS		
APL Product Guide	Alison Chatterton	50
Windows 3: the Implications for APL	Adrian Smith	61
First Catch your Function: the STSC Compiler	Ray Cannon	65
APL*PLUS/PC Release 10	Jonathan Barman	68
RECENT MEETINGS		
PostScript Graphics in APL	Adrian Smith	76
Field Trip to Rowntree Mackintosh	Sylvia Camacho	93
GENERAL ARTICLES		
APL91 Update	Charles Schulz	99
Soviet APL: a Historical Outline	Alexei I. Miroshnikov	102
Object Oriented Programming and APL	Zdenek V Jizba	109
TECHNICAL SECTION		
Hackers Corner: Check that Disk!	Adrian Smith	125
Technical Correspondence		126
Mice do it on the Mat	Ray Cannon	130
Problems for APL Buffs (III)	Zdenek V Jizba	136
Index to Advertisers		143

Sustaining Members' News

Dyadic Systems Ltd.

On 17th December, Dyadic began shipping the second major release of Dyalog APL/X. Since its launch at APL90, this version of Dyalog APL for the X Window system has proved extremely popular. Demand has been particularly strong from the major financial institutions in New York - an indication of the growing move to UNIX workstations in this sector.

Dyalog APL/X can now be run as a background job even if the X Server is not running. The Session window is now displayed only if required for session input/output, so that in "run-time" applications only the $\square SM$ window and graphics window(s) need appear on the user's screen. The size and position of all types of window (session, $\square SM$, edit, trace, via and prefect) can now be specified in the X11 defaults file, as can a user-defined title for the $\square SM$ window. Automatic control of the mouse pointer has been improved so that, in general, it is not necessary for the user to move the mouse when input is required in a different window. The cursor colour is now user-definable, and the cursor shape has been changed to improve readability.

Thanks to some assistance from Graham Parkhouse at Surrey University and a font from APL-385 (Adrian Smith), Dyalog APL/X now includes PostScript printer support. This workspace will also be made available in other implementations of Dyalog APL as new versions are released.

Dyalog APL/X workspaces, component files, and external variables are 100% binary compatible between Sun SPARC, IBM RS/6000 and HP9000-300/400 machines. This means that users in a heterogeneous network can share data and workspaces as if they all had the same type of system. Keyboard tables for all three types of workstation are now distributed with every copy of Dyalog APL/X to make remote login between different types of system easier to establish.

This type of binary compatibility and inter-operability is fairly unique in the UNIX environment, where hardware architectures vary widely and applications have to be recompiled (introducing new bugs) when they are moved from one machine to another. This unique ability may give APL a new lease of life in organisations where there is a trend towards C, other languages, 4GLs and application generators which rely on compilation.

Although the Sun SPARC version of Dyalog APL remains very popular, Dyadic is pleased to report strong and growing international demand for Dyalog APL (both the X and the standard version) on the IBM RISC System/6000. This implementation of Dyalog APL has now been shipping for over three months and, mainframes excluded, provides an unparalleled level of performance. Rated at 56MIPS and 23MFlops, with up to 512b of RAM, the recently announced RS/6000 Model 550 may even outperform quite respectably sized mainframes - and do so at a fraction of the cost.

Would YOU like to help manage the British APL Association?

You do not have to be an ace APLer, indeed a beginner may be better at helping us to satisfy our members' needs as many of them are beginners.

Do you have any ideas about the things we are trying to do? Here is a list (probably incomplete), but feel free to suggest new ventures.

- Promote the use of APL ...

as a programming language
as an educational tool in physics, maths, statistics
as a notation

- Help APLers to do it better ...

guidance on help at any level
which good books are good for what purposes
what's important about the differences between ISO and other APLs
find out what teachers need and help to give it to them
provide a platform for product announcements & a product guide
get the software library back into operation

- Recruit APL users to join the BAA

- Publicise APL and the BAA

- Encourage APLers to undertake projects to further any of these aims then help with, control and pay for the work

- Help the APL Association to become a centre of excellence and technical advice on APL

- Write it up for Vector

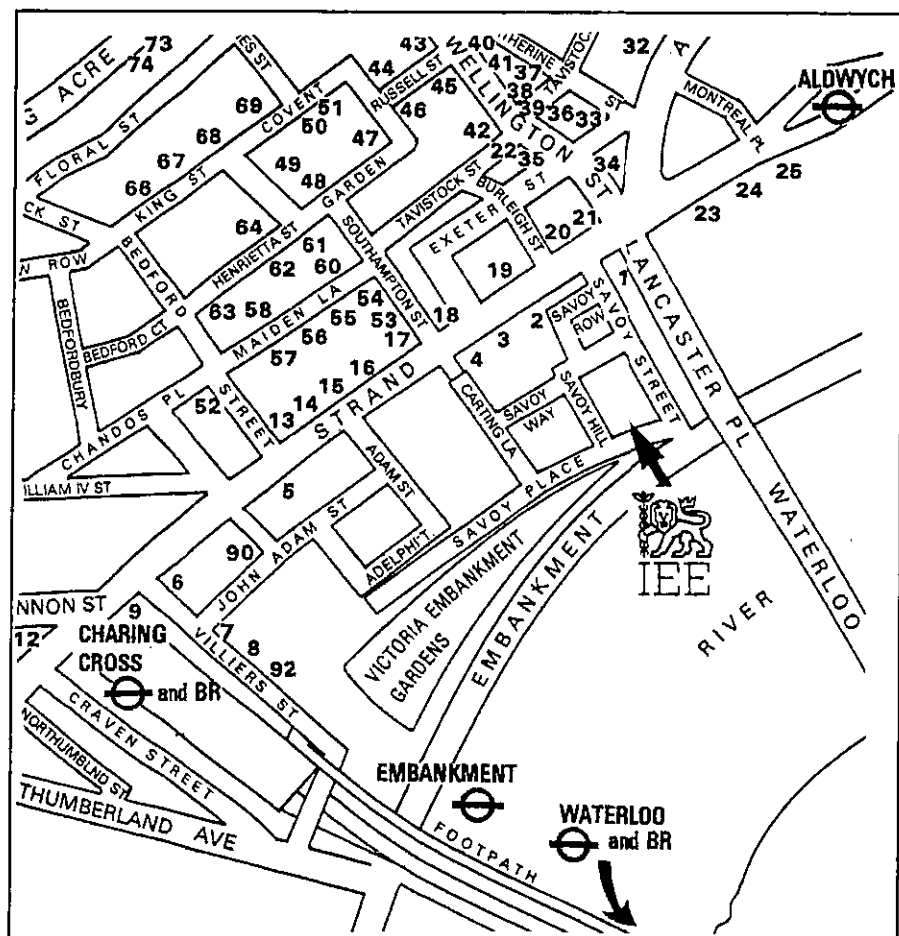
Your contribution will be very welcome. You don't have to join the committee unless you would like that: most committee members would welcome help and there are several working groups. Of course candidates for committee are welcome and temporary co-option is possible instead of the annual election.

If you are interested, please telephone the Hon Sec on 0727-860130 to discuss what you would like to do, or write to me - Anthony Camacho at 2 Blenheim Road, St Albans, AL1 4NR. Of course you may approach any member of the committee.

CONTENTS

EDITORIAL: Solving Problems with APL	Jonathan Barman	Page 3
APL NEWS		
Quick Reference Diary		5
APL Training Courses 1990-91		7
General Correspondence		8
British APL Association News:		
Projects Report	John Searle	9
News from Sustaining Members	Alison Chatterton	13
The Education Vector	Alan Sykes	17
The Random Vector	David Eastwood	33
REVIEWS		
APL Product Guide	Alison Chatterton	50
Windows 3: the Implications for APL	Adrian Smith	60
First Catch your Function: the STSC Compiler	Ray Cannon	64
APL*PLUS/PC Release 10	Jonathan Barman	67
RECENT MEETINGS		
PostScript Graphics in APL	Adrian Smith	74
Field Trip to Rowntree Mackintosh	Sylvia Camacho	91
GENERAL ARTICLES		
APL91 Update	Charles Schulz	97
Soviet APL: a Historical Outline	Alexei I. Miroshnikov	100
Object Oriented Programming and APL	Zdenek V Jizba	108
Mandelbrot Sets	Ray Cannon	123
TECHNICAL SECTION		
Hackers Corner: Check that Disk!	Adrian Smith	129
Technical Correspondence		130
Mice do it on the Mat	Ray Cannon	134
Problems for APL Buffs (3)	Zdenek V Jizba	140
Index to Advertisers		147

How to Find the IEE



The numbers on the map refer to various pubs and other eating houses in the vicinity of the new venue for London meetings. John Searle has conducted extensive research (at great personal sacrifice) in the neighbourhood ... his results may be found in the BAPLA News section on page 9.

Editorial: Solving Problems with APL

by Jonathan Barman

I was most impressed with the talks given at the September meeting as they showed APL being used as a simple and elegant tool to solve ordinary problems.

Adrian Smith showed how he had developed some functions which mirrored the PostScript language, and which could be used for prototyping and testing PostScript programs. Graham Parkhouse showed how APL enabled him to display engineering ideas in three dimensions with proper perspective and shading. Both had created a 'tool box' of APL functions to tackle their respective problems, and I felt that APL had influenced the way they had gone about constructing solutions.

The APL notation seems to exert a strong influence on the way one thinks about problems. In my own case I often find the ordinary mathematical notation in a textbook difficult, and understanding only comes when I have converted it into APL. The power of the APL operators, particularly in the new extended APLs with nested arrays, makes problems easy to state, and having stated them concisely the road to the solution is often easier.

The environment in which APL works is just as important as the notation. The concept of a workspace where data, functions and operators all co-exist, make it easy to test out ideas which may lead to a solution. Having found an algorithm which is a stepping stone towards the final goal, one promptly turns it into a function which can be applied as required.

The APL syntax enabled Adrian and Graham to string functions together in a very natural and obvious way, which made their solutions seem so easy and simple. All languages allow the definition of functions or procedures, but the APL syntax combined with the workspace environment seems to make things much easier than other languages. Recently I have used REXX under VM. REXX is a deservedly popular language and is an interpreter like APL. Being a beginner I naturally looked at the procedures written by experts in order to learn the language. I was quite surprised to find that most of the procedures were self contained and that they did not seem to be built in the normal APL way of using functions as building blocks. REXX has all the facilities needed for creating procedures using the building block approach. I could only conclude that the facilities tended not to be used because passing data between independent REXX procedures was less convenient than passing data between APL functions.

Creating reusable code is vital if programming problems are to be solved quickly. Neither Adrian or Graham created all their functions from scratch - many utilities were copied into their workspaces. The current Object Oriented Programming bandwagon rightly emphasises the importance of writing reusable code, and OOPS languages have excellent facilities for encapsulating functions so that they are truly modular. APL is showing its age in this respect, and Packaged Workspaces have to be used to ensure proper encapsulation. However JCOPY has served us well over the last 25 years and will continue to do so.

In my view APL is the ideal problem solving language. The notation, syntax and environment all work together as an integrated whole to give us all a potent and effective tool, both at the computer keyboard and as a way of thinking about problems and solutions.

Change of Venue for Monthly Meetings

From February 1991, we have decided to change the venue of our monthly meetings in London. Meetings will now be held at the **Institute of Electrical Engineers**, Savoy Place, London.

There are two main reasons for the change. Firstly, the cost of hiring a room at the IEE is considerably less than at Over-Seas House. Secondly (and possibly more importantly), the IEE room comes equipped with a VGA-quality Barco projector, enabling talks with demonstrations at every meeting without the trouble and considerable expense of booking an outside company to provide one.

The committee believe that this move will greatly enhance the quality of our monthly meetings - come along in February and see if we've made the right decision.

Quick Reference Diary 1991

Date	Venue	Event
8 February 1991	IEE London	BAA Meeting: Resolve Award + British Airways - see full programme on page 6.
22 March	IEE London	BAA Meeting: The Launch of the APL Statistics Library
26 April	IEE London	BAA Meeting: Vendor Forum
7 June	IEE London	AGM and BAA Meeting: Modern Productivity Aids for APL
4th - 8th August 1991	Stanford	APL91: The Next 25 Years
20 September	IEE London	BAA Meeting
25 October	IEE London	BAA Meeting
22 November	IEE London	BAA Meeting

Starting from February 1991, all British APL Association meetings will be held in the IEE, Savoy Place. Nearest tube outlets: Temple or Embankment. A map (with an indication of good places to eat and drink) is included on page 2.

Meetings are normally held on the 3rd Friday of the month throughout the autumn and spring.

Dates for Future Issues of VECTOR

	Vol.7 No.4	Vol.8 No.1	Vol.8 No.2
Copy date	1st March 91	4th June 91	5th Sept 91
Ad booking	12th March 91	15th June 91	12th Sept 91
Ad Copy	21st March 91	23rd June 91	21st Sept 91
Distribution	April 91	July 91	October 91

Programme for February Meeting

Date: 8th February at 2.00pm
Venue: The Gilbert Room,
Institute of Electrical Engineers
Savoy Place, London.

Outstanding Achievement Award Presentation

This year's Outstanding Achievement Award was awarded to Metapraxix of Kingston, Surrey for their world-renowned Management Information system 'Resolve'. At this month's meeting, David Preedy (of Metapraxix) will receive the award, and he will follow with a full demonstration of Resolve.

The remainder of the meeting will be taken up by a talk from Paul Gallagher (British Airways) on a PC-based APL application from British Airways that looks into "long term strategic planning in the modern world".

VENDORS' FORUM

Date: 26th April at 2.00pm
Venue: as above

For the last two years, we have reserved a meeting for vendors of APL interpreters to come along and tell us about the new "goodies" they have in store for us. This has proved very successful, so this year we continue the tradition.

Speakers this year will represent:

IBM	MicroAPL	REUTER:FILE (formerly I.P. Sharp)
I-APL	Dyadic Systems	Cocking & Drury (S.T.S.C.)

If you want to be right up to date ... come along and partake!

APL Training Courses for 1991

compiled by Alison Chatterton

Dates shown cover the period from January 1991. For confirmation of dates and/or further details please contact the vendor directly.

Level	Company	Days	Dates
Beginners	Cocking & Drury	3	15 Jan, 26 Feb
	MicroAPL	1	7 Feb, 4 Apr, 30 May, 25 July, 19 Sept, 14 Nov
	Mercia Software	3	
APL*PLUS/PC	MicroAPL	1	7 Mar, 2 May, 27 June, 22 Aug, 17 Oct, 12 Dec
	Mercia Software	2	17 Jan
Intro to APL2	MicroAPL	1	24 Jan, 21 Mar, 16 May, 11 Jul, 5 Sep, 31 Oct
Intermediate	MicroAPL	1	14 Feb, 11 Apr, 6 June, 1 Aug, 26 Sep, 24 Nov
APL*PLUS II conversion	MicroAPL	1	17 Jan, 14 Mar, 9 May, 4 July, 29 Aug, 24 Oct, 19 Dec
Advanced	MicroAPL	1	21 Feb, 18 Apr, 13 June, 8 Aug, 3 Oct, 28 Nov
(Nested Arrays)	Cocking & Drury	2	
System Design	Cocking & Drury	2	5 Mar
Statgraphics	Cocking & Drury	2	

GENERAL CORRESPONDENCE

From: Pan Pantziarka

21st November 1990

The Compulink Information eXchange (CIX) is the premier UK electronic bulletin board enabling a wide range of computer users, from IBM mainframers to the users of the tiniest home micros, to dial-in and exchange data, programs, leave mail and to have chats and discussions in real time. One of the primary attractions of CIX is the large number of conferences available, where people who share a particular interest can "meet" like minded folk, ask and answer questions and swap files.

A new addition to the conference list (which includes such special interests as home-brewing and anarchist politics!) is an APL conference. The conference APL.LANG has the following topics: 'general', 'BAA', 'Vector' and 'WS'.

This conference could serve as an ideal forum to attract new people to the language, for novices to ask questions and for the British APL Association and Vector to issue announcements, meeting dates and APL news etc.

The 'WS' topic is also a good place for public domain and shareware APL interpreters and workspaces to be made available to the public.

To join CIX dial-in on 081-399 5252 (or if your modem supports V32, use 081-390 9787). Once you've dialled in, just follow the onscreen instructions to become a subscriber. Alternatively call the CIX voice-line on 081-390 8446 for details on costs etc.

See you all online

Pan Pantziarka
Compass R&D Ltd
15 Frederick Sanger Rd
Surrey Research Park
Guildford GU2 5YD

British APL Association News

Projects Report

from John Searle (November 29, 1990)

I see my role as Projects Officer as a coordinator and facilitator of members' ideas. If you have an idea for worthwhile expenditure of the Association's resources (money, expertise, APL skills and/or personpower) please submit it to me. A one-page proposal is quite sufficient, with convincing arguments in sufficient detail to promote further discussion. I undertake to put all proposals to the committee, and if agreed, to try to find someone to implement them. I do not undertake to do them all myself!

Here are some current members' projects, and their recent status:

- ASL - APL Statistics Library, which now has its own Management Team, as documented in Vector 7.1, and to be launched at the March 1991 meeting.
- Membership Records System - would you believe that our membership records are kept in DBASE? An APL solution has been proposed, prompting much debate within the committee as to the suitability of APL for this purpose.
- Software Library - how can we rescue it from its current state of chaos? (Apologies to those who have suffered through non-supply.) The workload had become too much, but a solution is in sight. Please watch this space for details of a relaunch in the next issue of Vector.
- ISO Standards work - due to the withdrawal of DTI funding, the BAPLA now subsidises the BSI Principal Expert's travel and accommodation at ISO/APL technical meetings.
- Booklist / Book Library proposal - how can we organize and distribute APL literature and reviews to our members as a lending and reference library? (including our international members?). Ideas please.
- Meeting Venue Change to IEE - an important component of a successful BAPLA meeting is a drink before or afterwards, and if needed, a good meal. I have undertaken to make an extensive review of all of the pubs in the vicinity, and preliminary results reveal that the nearest pub is in fact quite suitable. This is the New Savoy Tavern in Savoy St, and the upstairs bar has been tested and ratified under simulated BAPLA conditions (i.e. a meal at 1pm on a Friday and a drink at 5pm). See you there on February 8th.

Publicity Officer's Report from Jonathan Martin

APL has not been renowned over the years for attracting a great deal of publicity - and the small amount it does get is seldom good.

Publicity of the BAPLA comes in two forms - publicity of what the committee is doing to its members and publicity to the outside world.

The first is what you as a member (assuming you are one??) see as the fruits of your membership fee. Vector is the most obvious and goes out to all members. The notice of meetings, however, is just as important (for those people who live in Britain) as they have a great bearing on the numbers who turn up on a Friday afternoon in London. If we don't give people enough notice, they may already have something booked; if we don't 'sell' the meeting well enough, they may decide they can't afford the time; if we do get these and other things right, the turnout is good and everybody is happy.

The second is much harder though. Publicity to the outside world is mainly through the computer press and here we are not starting from a good base. They have to sell papers and are generally only interested if we have some up-to-the-minute news. Are we CUA-compliant for instance? Do we run under OS/2 or Windows-3? Can we do a bit of co-operative processing here and there? How do we do Object Orientation in APL (APL++?). Generally the answer to all these questions is either we don't or we can't.

Consequently we have to rely on more major events like the Outstanding Achievement Award and the sponsoring of the APL Statistics Library, events that aren't run of the mill and may attract the attention of even the most hardened press hack.

So I don't think you are going to see APL emblazoned on the front pages of the weekly magazines in the immediate future, but with some effort and a bit of luck you may come across the odd snatch here and there.

Thoughts on APL 1994 from David Eastwood (Nov 1990)

The British APL Association is teetering on the verge of putting in a bid to host APL94. I would like to put forward some suggestions regarding such as event and the way in which we plan for it.

There are a number of areas where I feel that previous APL conferences could be improved upon, speaking as someone who is both an APL vendor, APL programmer and (still) an APL enthusiast.

1. **Committee.** The decision has already, and quite rightly, been made that the conference committee should exist independently of the BAA Committee as it will have to be in existence for some years. I am however disturbed by the possibility that the Committee will be made up of the usual 'hard core' of APL activists, with the choice of members being, at best, made casually in a BAA Committee meeting. I propose that the BAA membership be canvassed for volunteers. We could even elect the committee!
2. **Conference audience.** As a precursor to discussing themes and organisation for the conference, I wonder whether we should address the topic of who we would want to attend. I would hazard a guess that the numbers of attendees at APL conferences is gradually declining and that those who do attend are drawn from a pool of long term APLers who are regular attenders at such events.

If we wish to increase the numbers of attendees and to attract new faces we should bear two points in mind. Firstly the cost of the event and secondly the inducement to bring partners and families. Delegates to an APL conference will either be seeking sponsorship by their employer or will be using a portion of the annual leave to attend. In either case the conference should be seen as offering 'value for money' - it should offer an attractive mix of informative and educational material as well as a varied support program. Note that I do not count singing appalling songs about APL as a support program.

3. **Conference theme.** It goes without saying that a strong theme for the conference should be selected. (I would suggest the controversial one of 'APL as A Programming Language' for those of us who couldn't care less about yet more language extensions.) What I would ask, however, is that the conference actually remembers that it does have a theme and arranges that papers (see below) and the conference itself stick to the theme. The conference theme is there to attract contributions and also to set the overall tone of the event.

4. **Papers.** These are the backbone of any conference, and I would hope that some effort be made to improve matters here. There seems to be little evidence that papers are chosen to match a theme, and in line with the comment above, why do we need so many? I think that the desire to have three parallel sessions is excessive. I would also wish to see two classes of paper strictly forbidden:

- Overt attempts to read from sales literature
- Papers with no effective content

Both these categories have been in evidence at previous APL conferences. In the second category, I place papers whose sole apparent content was simple minded discussions of topics covered in APL reference manuals.

I have a feeling that program committees set out an empty timetable containing a number of parallel sessions and then attempt to fill them with the submitted papers rather than arranging as many sessions as there are papers worth presenting.

5. **Education Day.** At APL86 and APL89 (I missed 88 and 90) the inevitable education day was an embarrassing waste of time. No one other than the usual coterie of enthusiasts turned up. Let's abandon this part of the conference.
6. **Home users.** At the same time I would like to speak on behalf of those neglected home users who buy I-APL, APL68000 or Pocket APL. Some might spend several times the price of their APL (and indeed of their computer!) to attend, but I suggest that many wouldn't. Could we make some effort to target some event or events to these users?
7. **Vendor forums.** A vendor forum seems to be an excuse to extract yet more money from APL vendors. Invariably the conference organisers make large charges but make no effort to attract an audience. The more astute vendors make their presentations as papers anyway. If this sort of nonsense is to be repeated, make it a free event.
8. **Sponsors.** Whilst I believe that Rowntrees have shown interest in sponsoring part of APL94, which is extremely good news, I would sincerely hope that some effort is to be made to canvass other commercial sponsors. I actually believe that a good level of commercial sponsorship will offset the bad impression that is always given by the APL 'nutters' who tend to dominate these events.

News from Sustaining Members

Compiled by Alison Chatterton

APL People Limited

Due to demand from the manufacturing industry, the estimating package, PEFAC (which is written in APL) is now being extended to cover financial estimating as well as time estimating. The company that originally developed the product, H Walton Technical Systems Limited (founded by Herbert Walton - first Chairman of the British APL Association), is being renamed APL Manufacturing Limited.

We have no plans to change the name of APL People however, and it will continue to provide the same high level of service as before, helping clients find suitably qualified people for permanent vacancies and for contract assignments.

Despite the recession, we have placed a record number of people in jobs in 1990, and plan to do even better in '91! We currently have a good selection of APL consultants registered with us for contract work and are always pleased to visit clients to discuss their APL requirements in detail with them. Companies contemplating recruiting 'APLers' or with development work in APL planned for the New Year should contact me, Jill Moss on 0225 462602 to book a free consultation with myself or one of my colleagues.

Cocking and Drury Ltd

Having just completed the introduction of APL*PLUS II Version 3 we are now shipping upgrades to APL*PLUS PC Version 10 - reviewed on page 67 in this issue.

This release marks ten years of APL micro technology and the enhancements are significant. Support for Windows 3 with the ability - on an appropriate machine - to have an APL session running while you work with other software or to have two APL sessions running simultaneously. The inclusion of `□NA` to call non-APL subroutines written in C or FORTRAN. Support for mice.

January will also see the launch of a new release of the APL*PLUS UNIX product. This release uses X Windows on SUN 4 stations, the DEC RISC platforms and the new IBM RS/6000. Release 4 should also include enhancements to the external processor interface that allow passing of nested

arrays between APL and an external processor. There will also be a new APL User Interface specifically written for X Windows that will allow:

- keyboard handling compatible with APL*PLUS PC and II
- Full Screen Handling, with colour, compatible with APL*PLUS PC and II
- Faster APL output and `WPUT`.
- The inclusion of `WIN`.

To complete the round of product updates Version 7 of the APL*PLUS System for the Mainframe has now been announced for release now for VM and first quarter 1991 for MVS. And STSC have announced a new product - Network SHAREFILE. This offers users true networking capabilities among remote sites as well as across multiple servers.

You will probably see announcements soon that Cocking & Drury have become UK dealers for Digitalk Inc of Los Angeles - developers of the Smalltalk/V line of object-oriented programming language and development environment.

This does not mean we are turning our backs on the STSC APL*PLUS products - the list of new releases coming out of Rockville means there is plenty of opportunity for further development there. It is quite some time since we have noticed such a strong level of interest in these products, both for the APL*PLUS II System and particularly for the UNIX software. We are hoping to recruit another support consultant in the New Year to assist Dave Phillips with the growing level of APL support.

MicroAPL Ltd

As we look back over the last quarter of 1990, we have been pleasantly surprised by the reception given to APL68000 Level II. Copies of the software have been shipped to most countries in Europe as well as the USA, Japan and Canada. At the time of writing, sales of the Macintosh and Atari versions are running neck and neck with the Amiga lagging considerably. Whilst we expect sales of the Amiga version to catch up as the sales in the USA (our biggest Amiga market) build up, it is interesting to see the continuing strong sales of the Macintosh version.

MicroAPL has begun to plan our upgrade program for next year and we hope to announce details in a forthcoming Vector, but users of our Atari software may care to note that work is in hand on an overhaul of the Atari versions of our software. Both our interpreters (Level I and Level II) make use of the same Supervisor (Operating System interface) on any given port and the work on the Atari ST will, for example, apply equally to the Level I and Level II versions.

Our free runtime policy continues to attract a steady number of licence applications ranging from Real Estate analysis to Music via Restaurant Menu evaluation. Sales of APL.68000 (both versions) into the educational sector continue to be strong, with as much as 30% of our sales going to Universities and Colleges. Readers who use APL in educational establishments should note that we have a highly successful educational site licensing scheme for APL.68000 which offers APL.68000 at very considerable discounts for bona-fide educational establishments.

At a local level, MicroAPL have just released our 1991 Training Program. We continue to specialise in intensive, one day courses either at our London offices or on a client site. We have added a new timetabled course to our schedule - 'An Introduction to APL2', which is aimed at users of any of the variants of APL2 (APL2/370, APL2/PC or APL.68000 Level II). Although not a part of the regular training schedule, our APL Programming Techniques course has been run several times in 1990 and seems to meet a gap in the market.

Our consultancy operations continue to cover a wide variety of applications, with a reasonably even spread between PC, Mac and multi-user environments. As we end 1990, for the first time our non-APL consultancy activities are generating as much revenue as our traditional APL work, a fact which reflects the growing quality of packaged software solutions and applications generators.

Dutch APL User Group

The Dutch APL User Group (DAUG for short) is happy to present itself here as a sustaining member of Vector. The DAUG exists now in Holland for about fifteen years. In those years we endeavoured to

- * stimulate the use of APL
- * exchange knowledge and experience about APL between members
- * inform outsiders about APL
- * stimulate standardisation in matters of APL
- * intermediate between users and APL vendors.

The vigour in which we developed activities for each of those topics varied over the years of course. Nevertheless we organised almost every year a one-day mini-congress with several speakers. Here APL users listened to and discussed with the speakers, and the users exchanged ideas among themselves in between the talks.

Currently the DAUG is engaged in the following 2 activities: the organisation of the next mini-congress on 6 April 1991 and the project ASWI which stands for "APL ShareWare Initiative".

MINI-CONGRESS. The next APL mini-congress is titled "Informal software/experience Exchange" and it will be held on 6 April 1991 in Nieuwegein. It will be organised with various booths in which users as well as official vendors will present some of their products. A booth can be a poster, or a demonstration, or a pile of books, or whatever a person feels comfortable to express his ideas. At the moment we have a minimum of at least the following booths: ASWI, I-APL, RNA-applications in APL68000, Menu-system in APL-2, APL68000, Dyalog APL, STSC, APL68000, APL consultancy, Public domain software. Anybody who is interested in showing his/her application is invited to contact Theo Zwart, Lekstraat 4, 3433ZB Nieuwegein, tel.03402-66336.

ASWI. ASWI stands for APL ShareWare Initiative. This initiative started at the end of 1990 with the aim of developing high-quality software for APL programmers. ASWI aims to attract the attention of professional software developers to use APL and to minimise costs by offering this software on a shareware base.

Two of the most important issues which are almost resolved are: choice of basic hardware/software platform and standardisation. We have chosen IBM's APL2 for PC's as the primary platform and we will possibly choose APL68000 for Mac, Atari and other 68000 micros as the second platform. Our main reasons are the free run-time version, nested arrays, and easy interchange between both platforms by `IN` and `OUT`. A standard is developed for documentation as well as for programming style. This standard will be submitted for publication in Vector for the next issue.

Various other topics which are still under discussion are: quality assurance, a document library, public relations/communication and many others. Most important of all will be a wish list with priorities of required software. ASWI intends to start with developing functional specifications of those wishes first and subsequently delegate them for implementation to the various interested parties.

And finally, if you are not a member of the DAUG yet, we would like to welcome you. Please write to Postbus 1341, 3430BH Nieuwegein tel.03474-2337. Currently, the board of the DAUG consists of Theo Zwart (president), Bernard Smoor (secretary), Thom Bonnema (treasurer), and Louis van Geldrop, Herman van Loon, Gerrit Teule and Eke van Batenburg.

THE EDUCATION VECTOR

January 1991

Editor Alan Sykes

This Education Vector has been reprinted from VECTOR Vol.7 No.3. VECTOR is the Quarterly Journal of the British APL Association; for more information about the British APL Association, contact the Secretary: Anthony Camacho, 2 Blenheim Rd, St Albans, Herts AL1 4NR Tel: 0727 860130.

Contents

Editorial	Alan Sykes	18
Letters		19
An APL SCRIPT Function	Alan Sykes	24
A To B Or Not A To B: That Is The Question	Alan Sykes	27
APL Booklist	Ed Shaw	32

Dr Alan Sykes,
c/o European Business Management School
Swansea University
Singleton Park, Swansea SA2 8PP
Wales, UK

Editorial

Welcome to this edition of Education Vector. As you know from previous discussion in these columns, committee members of the British APL Association have been looking carefully at the current expenditure involved in producing and posting Education Vector. The results of the survey, and letters sent to me indicate that most people appreciate Education Vector in its present form. The committee wish to continue the present distribution lists as far as possible, but are aware of the not insignificant cost of postage, both to readers in this country and particularly abroad.

After much careful thought the decision taken was that in the first place I should take this opportunity to encourage receivers of Education Vector to consider becoming members of the British APL Association for an annual cost of only £10 (£5 student rate). That way you would receive full information concerning the activities of the APL community both at home and abroad in the quarterly issues of VECTOR, which of course includes Education Vector as well as a new section on the progress of the APL Statistics Library project.

Secondly, for those readers abroad who are not able to take up this option, the committee agreed to continue posting Education Vector to anybody that wrote to us asking us so to do. So please send me a postcard or letter as outlined below.

Once again I hope readers will find something of interest in the following pages. I am indebted to those who have sent in comments and contributions; please continue to share your experiences in using APL by writing to the editor.

To: Dr Alan M Sykes (Education Vector)
European Business Management School
University College of Swansea
Singleton Park, Swansea SA2 8PP
United Kingdom

Dear Alan,

Please continue to send copies of Education Vector to me. My current address is:-

Name:
Address:

Letters on String Manipulations in APL

My recent series of articles on string manipulation has resulted in a number of letters from readers of Education Vector, and I am grateful for all who took the trouble to write to me. Here are some thoughts from home and abroad.

1. A Jotter-Notepad

Bill McLean from Dunoon sent me a set of APL functions that allow you to create and edit a set of notes, being a matrix called PAD. Here is his description of its use and the function listings.

MENU

Use the following command words to run the JOTTER program

SHOW.... to see the up-to-date JOTTER pad
 CREATE.. to create a NEW jotter pad
 DELETE.. to remove a line from the pad
 ADDB.... ADD a line to the Beginning of the pad
 ADDE.... ADD a line to the End of the pad
 INSERT.. insert a line after a given line
 REWRITE.. rewrite a line
 MENU.... to get this menu

PAD

SEND WS TO DR SYKES
 WRITE TO ALISON, ROBERT, ALASTAIR
 SCOTVEC MODULES
 REPORT LINES FOR COMPUTER
 S3 TESTS

SHOW

1SEND WS TO DR SYKES
 2WRITE TO ALISON, ROBERT, ALASTAIR
 3SCOTVEC MODULES
 4REPORT LINES FOR COMPUTER
 5S3 TESTS

DELETE

Delete which line?
 0:

2

1SEND WS TO DR SYKES
 2SCOTVEC MODULES
 3REPORT LINES FOR COMPUTER
 4S3 TESTS

ADDE

Type the jotting

WRITE TO FAMILY

1SEND WS TO DR SYKES

2SCOTVEC MODULES

3REPORT LINES FOR COMPUTER

4S3 TESTS

5WRITE TO FAMILY

INSERT

AFTER WHICH LINE WILL WE INSERT

□:

1

Type in the jotting

PHONE BROTHER ROBERT, JOHN

1SEND QA TO DR SYKES

2PHONE BROTHER ROBERT, JOHN

3SCOTVEC MODULES

4REPORT LINES FOR COMPUTER

5S3 TESTS

6WRITE TO FAMILY

REWRITE

Which line is to be rewritten?

□:

4

Type the new jotting

ATTITUDE AND HOMEWORK :TWO LINES FOR S3 REPORTS

1SENS WS TO DR SYKES

2PHONE BROTHER ROBERT, JOHN

3SCOTVEC MODULES

4S3 TESTS

5WRITE TO FAMILY

VADDB[□]V

VSCREEN←ADDB;LINE;X;ORDER

[1] 'Type the jotting'

[2] LINE←1 40pX←40+□

[3] ORDER←((1+1+pPAD),1)p:1+1+pPAD

[4] PAD←LINE,[1]PAD

[5] SCREEN←(VORDER),PAD

V

VADDE[□]V

VSCREEN←ADDE;LINE;X;ORDER

[1] 'Type the jotting'

[2] LINE←1 40pX←40+□

[3] ORDER←((1+1+pPAD),1)p:1+1+pPAD

[4] PAD←PAD,[1] LINE

[5] SCREEN←(VORDER),PAD

V

VCREATE[□]V

VCREATE

[1] 'Input each line. up to 40 characters'

[2] 'End with a <RET> for each line'

[3] 'End with TWO <RET> at the end of the notes'

[4] PAD←MAT 40

V

```

VDELETE[ ] V
VSCREEN←DELETE;ORDER;X;Y;Z;LINE
[1] 'Delete which line?'
[2] X←
[3] Z←X+1+PAD
[4] PAD←Z/[1]PAD
[5] ORDER←((1+PAD),1)P+1+PAD
[6] SCREEN←(VORDER),[2]PAD
V

```

```

VINSERT[ ] V
VSCREEN←INSERT;LINE;Z;X;ORDER
[1] 'AFTER WHICH LINE WILL WE INSERT?'
[2] X←
[3] 'Type in the jotting'
[4] LINE←1 40P+40+
[5] Z←(X+1+PAD)+(1+PAD)P+1
[6] PAD←Z/[1]PAD
[7] PAD[X+1;]←LINE
[8] ORDER←((1+PAD),1)P+1+PAD
[9] SCREEN←(VORDER),[2]PAD
V

```

```

MAT:
MAT: (w+V),[1] MAT w:0=P+V+ (0,w)P'

```

```

VMENU[ ] V
VMENU
[1] 'Use the following command words to run the JOTTER program'
[2] '
[3] 'SHOW.... to see the up-to-date JOTTER pad'
[4] 'CREATE.. to create a NEW jotter pad'
[5] 'DELETE.. to remove a line from the pad'
[6] 'MOVE.... moves a line from one row to another'
[7] 'ADDB.... ADD a line to the Beginning of the pad'
[8] 'ADDE.... ADD a line to the End of the pad'
[9] 'INSERT.. insert a line after a given line'
[10] 'REWRITE. rewrite a line'
[11] 'MENU.... to get this menu'

```

```

VMOVE[ ] V
VSCREEN←MOVE;LINE;X;Y;Z;ORDER
[1] 'Move which line?'
[2] X←
[3] LINE←1 40P+PAD[X;]
[4] Z←X+1+PAD
[5] PAD←Z/[1]PAD
[6] ORDER←((1+PAD),1)P+1+PAD
[7] PAD←SCREEN←(VORDER),[2]PAD
[8] 'To put AFTER which line?'
[9] Y←
[10] Z←(Y+1+PAD)+(1+PAD)P+1
[11] PAD←Z/[1]PAD
[12] PAD[Y+1;]←LINE
[13] ORDER←((1+PAD),1)P+1+PAD
[14] SCREEN←(VORDER),[2]PAD
V

```

```

VREWRITE[ ] V
VSCREEN+REWRITE;LINE;X;X;ORDER
[1]'Which line is to be rewritten'
[2]X+
[3]'Type the new jotting'
[4]LINE+1 40pY+40+
[5]PAD[X;]+LINE
[6]ORDER+((1+pPAD),1)p:1+pPAD
[7]SCREEN+(vORDER),PAD
V

```

```

VSHOW[ ] V
VSCREEN+SHOW;ORDER
[1]ORDER+((1+pPAD),1)p:1+pPAD
[2]SCREEN+(vORDER),[2]PAD
V

```

2. A Correction From Bill Smith

Bill Smith wrote to tell me that I missed a sentence out of his note on 'dBase Files and Lotus Files into I-APL' published in the October edition of Education Vector. (My Apologies.) The corrected paragraph reads as follows.

'dBaseIII files, output with a COPY TO <filename> TYPE SDF command, yield a series of fixed length records in ASCII format. Lotus spreadsheets, 'databases' at least, printed to file result in .PRN files with one record per row in ASCII format. These are only fixed length if the right-most field of each row is of the same size but this is easily managed even if you have to create a dummy column to achieve it. A further problem with Lotus files is that they are preceded by a number of carriage returns/line feed characters which need to be disposed of in the first stages of transfer; actually in the DREAD function.'

3. Day-of-Week Algorithm

I am grateful to L Llewellyn-Jones of London for sending me a copy of an article by Bhairav Joshi published in COMPUTERWORLD (January 14 1980) who explains the mathematics of calculating the day of the week from the date. The article is easy to follow and also includes an APL program. Unfortunately I am unable to include the article in this column but will supply copies to any interested reader who cares to provide a stamped-addressed envelope. However, here is the supplied function:

```

V R+DOW D;M;R;T
[1] M+12p 7 31 30
[2] M[2]+28+/-0=(T+ 4 100 400 4000)|[R+2+D
[3] R+R+ -/|(-1+|R)÷T, 1000 2000
[4] R+1+7|(-1+R+D[2]++/-1+1+D)+M
[5] R+(7 9 p 'SATURDAY SUNDAY MONDAY TUESDAY WEDNESDAYTHURSDAY
FRIDAY ')[R;]
V

```

4. Replacing Strings

Gerard Langlet sent me lots of comments regarding string searching including *SS4* below which is 'short and fast enough to help those people whose APL interpreter does not have a system function `SS` for string searching'

```

VR←S SS4 T;C;L;N
[1]  R←T=1+S ◊ R←R/1ρR ◊ T←T,1+⊖AV ◊ L←ρS+1+S ◊
    C←L×ρN←'C'←T[R←R+1]=1+S ◊ R←C/R◊S+1+S◊' ◊ ⌊CρN ◊ R←R-L
    V

```

He also sent me a string replacement programme which no doubt is extremely efficient, but unfortunately is not very easy to understand. Nevertheless I present it to you together with a more pedestrian version of my own which is intended for those readers who prefer to work through a program knowing something of the strategy behind its construction.

```

V ΔE do ΔN
[1]  ΔN←ΔN×ρΔE←ΔE,'◊' ◊ ⌊ΔNρΔE
    V

V T←S REPLACEIN T;A;B;C;D;E;I;J;K;L;M;N;U;a;b;c;d
[1]  a←'N+0<E+ρA+B◊M+1+L+÷/∧\S=C◊E+E-ρB+L+S◊S+M+S'
[2]  b←'K+U[L+L+1]=1+A◊L+K/L◊A+1+A'
[3]  c←'L+T=1+A◊L+L/1ρL◊U+T,1+⊖AV◊b do D+ρA+1+A◊J+ρL+L-
    D◊K+ρT◊K+Kρ1◊I+0=K[L]+0[1-E'
[4]  c←c,'◊N+0[E◊K[,L◊.+~1++\Nρ1]+0◊T+K/T◊L+L-
    E×~1++\Jρ1◊K+J×I+ρB◊T[,L◊.+~1++\Iρ1]+KρB'
[5]  d←'a do 2◊c do N'
[6]  d do [0.5×S+.=B+C+'/'
    V

```

```

'CAT/HORSE/MAT/GRASS/THE/MY' REPLACEIN 'THE CAT SAT ON THE
MAT AND I JUMPED ON THIS CAT'

```

```

THE HORSE SAT ON THE MAT AND I JUMPED ON THIS HORSE

```

```

      ∇ TEXT←SUB REPLACE TEXT;OLD;NEW;POS;SD;I
[1]  ⍺ find the old string and new string; last char separates
[2]  OLD←(¯1+N+SUB;¯1+SUB)+SUB
[3]  NEW←¯1+N+SUB
[4]  ⍺ find the positions of OLD in TEXT and abort if none there
[5]  POS←,OLD SS2 TEXT ⋄ →(0=ρPOS)/0
[6]  ⍺ calculate the difference in sizes (new-old)
[7]  SD←(ρNEW)-ρOLD
[8]  →(SD≤0)/ct
[9]  I←(ρTEXT)ρ1
[10] ⍺ expand the text using replicate
[11] I[POS+POS+SD×0,¯1+ρPOS]+SD+1
[12] TEXT←I/TEXT
[13] ⍺ insert new string at the original positions, exit if SD≥0
[14] ct:TEXT[¯1+,POS⋄.+,ρNEW]+((ρPOS)×ρNEW)ρNEW
[15] →(SD≥0)/0
[16] ⍺ now remove unwanted characters in case SD<0
[17] I←(ρTEXT)ρ1
[18] I[1+,POS⋄.+,¯1-SD]+0
[19] TEXT←I/TEXT
      ∇

```

An APL SCRIPT Function

Have you felt the need for being able to record the results of a session of APL for future use? The ability to be able to record a sequence of APL statements and any results returned, so that, perhaps, text can be placed round the APL session script to form an illustrated article on APL or the use of APL? In a teaching environment, this facility is quite important, as it is in writing these articles for Education Vector.

Here then is something like what is required. I have called it *APLSCRIPT*, and it produces a vector that when printed looks approximately like the screen would have appeared had the same set of instructions been executed. The function may be terminated by typing an input of four consecutive asterisks.

Here is a typical example of its use.

```

      Z←APLSCRIPT
X←1 2 3 4
ρX
4
+/X
4
(+/X)÷ρX
2.5
3 4⍲4
1 2 3 4
1 2 3 4
1 2 3 4
****

```

```

      Z
      X←1 2 3 4
      ρX
4
      +/X
10
      (+/X)÷ρX
2.5
      3 4⍲4
1 2 3 4
1 2 3 4
1 2 3 4

```

The basic idea is to capture input from the keyboard using `⎵`, and then to execute the resulting string, attaching the result of the execution (if there is one) to the string in a suitable way.

The function used is relatively simple; no attempt has been made to be clever and scan the input line for embedded allocations to the screen (`⎵←`), though one of these occurring at the beginning of the session is acceptable.

The function (written in APL*PLUS) is recorded below. Note that it has a line (line 2) containing an error check, which allows the user to make a silly mistake and recover from it. (If you are not using APL*PLUS, then you may have to adapt the program sensibly, by replacing the existing error trap with its equivalent one for your interpreter, or perhaps even dropping the statement altogether.)

```

VAPLSCRIPT[[]]V
[0]  ΔSCRIPT←APLSCRIPT;ΔS;ΔC;ΔF;ΔD;Δ;[]ELX;ΔG
[1]  Δ
[2]  []ELX←''' ERROR IN COMMAND LINE-REDO '''.'o+lp'
[3]  ΔSCRIPT←'' o ΔF+1 Δ F RECORDS IF COMMAND ENDS IN A RESULT
[4]  lp:ΔS+[] o ΔSCRIPT←ΔSCRIPT.(0=ρΔS)/[]AV[14] ΔAPPEND LINE FEED
[5]  ΔIF SO, RETURN WITH EMPTY ST
[6]  Δ(0=ρΔS)/lp
[7]  ΔDROP LEADING BLANKS
[8]  ΔS←(Δ\ΔS=' ')/ΔS o Δ(Δ/'*****'=4+ΔS)/0
[9]  ΔFIND WHERE ← IS
[10] ΔG←ΔS1'+' o ΔG+ΔG*ΔG≤ρΔS
[11] ct:ΔS←((('[]'=1+ΔS)*ΔG)+ΔS o Δ'Δ←',ΔS
[12] ΔCHECK TO SEE IF STRING TO LEFT OF ← IS A VARIABLE
[13] ΔC←(Δ1+ΔS1'+' )+ΔS
[14] ΔD←ρ[]NL 2 o ΔF←(ΔG≠0)ΔV/((ΔD[1],ΔD[2][ρΔC)+[]NL 2)Δ.=((ΔD[2][ρΔC)+ΔC
[15] do:ΔS←(5ρ' '),ΔS
[16] Δ APPEND LINE-FEEDS TO COMMAND-LINE
[17] ΔSCRIPT←ΔSCRIPT,[]AV[14],ΔS,[]AV[14] o Δ(-ΔF)/lp
[18] Δ(ΔF/'[]+',Δ'Δ←Δ' o Δ←Δ,[]AV[14]
[19] ΔSCRIPT←ΔSCRIPT,,Δ o +lp
Δ

```

If error trapping is not included, then continuity can be provided with a little care. Here is an example:

```

ZΔ←APLSCRIPT
X←1 2 3 4
X÷0 1 2 3

DOMAIN ERROR
APLSCRIPT[9]ΔX÷0 1 2 3 4
Δ

```

At this point the local variable *SCRIPT* is available to you, except that it will not contain the statement which caused the crash. One can thus issue a statement such as

```

Z←SCRIPT (suitable for a first-time crash)

or  Z←Z,SCRIPT
    )RESET ...and then
    Z←Z,APLSCRIPT

```

... to continue.

Since I wrote this function quite recently, I have used it considerably and wonder why I didn't write it a long time ago! If you have any suggestions for alternative *APLSCRIPT* functions please share your thoughts with readers.

A To B Or Not A To B: That Is The Question

Such a title leaves the reader in no doubt as to the intended pun! But what about the subject matter? Well I decided that an article on program design might be appropriate, not that I am an expert - far from it. However in teaching students using APL, it does seem to me that much of their confusion is caused by a lack of thought about design: just what the program should do and how it should do it. These thoughts should be expressed, if possible in plain English. Then and only then follows the task of translating these thoughts into machine-readable form and then checking whether the results conform to the design specified.

To illustrate this process, and to show how involved it can become even for a simple program, look at the problem of designing a useful *INDEX* function; a generalisation of the APL monadic function *⍳*. In other words I would like to be able to produce sequences such as 3 4 5 6 7 8, or 9 10 11 12.

The syntax of the function is a major consideration. Do I wish to type *NAME* 3 8 for the first example, and *NAME* 9 12 for the second? If so, perhaps a suitable *NAME* might be *BETWEEN*. Alternatively we might prefer to use the word *TO* and type statements such as 3 *TO* 8.

Such considerations determine whether the function should have a right argument only or both a left and right argument. A further consideration is whether the function is required only to print out results or whether the result of the function should be 'returned' so that further processing can be done using it. The latter should almost invariably be the case, although functions which do not return any result are sometimes appropriate (e.g. for written reports).

When these questions have been answered, we are in a position to specify the function in the following form:

```

      Name: TO
      Left Argument: A: an integer
      Right Argument: B: an integer
      Result: R: a sequence of integers from A to B

```

Suppose $A \leftarrow 3$ and $B \leftarrow 8$. Then we need to add the vector $\iota 6$ (where $5 \leftarrow 1 + B - A$) to the starting point 2 (equal to $A - 1$). Assuming index origin 1, this suggests the expression:

```

      -1+A+⍳1+B-A
3 4 5 6 7 8

```

Hence we have our first function *TO*.

```

      ∇ R←A TO B;∇IO
[1]   ∇IO+1
[2]   R←-1+A+∇1+B-A
      ∇

```

So now to check the function. This is where you need somebody else to play a devil's advocate role. Certainly *3 TO 8* works as does *2 TO 10*. But nothing in the specification mentioned that *A* should be less than *B*.

```

      8 TO 3

DOMAIN ERROR
TO[2] R←-1+A+∇1+B-A
          ^

```

The error message of the APL interpreter points clearly to the problem area: it refuses to use the iota function *∇* on a negative integer. If we wish the function to take sequences from high to low as well as low to high, modification is needed. The signum function (monadic *×*) plays an important role here, as does the modulus function *|*. Here are some examples:

```

      ×3-8
-1
      ×8-3
1
      ×8-8
0
      |3-8
5

```

Piecing these tools together gives the updated function and some test results.

```

      ∇ R←A TO B;S
[1]   ∇IO+1 ∘ S←-×B-A
[2]   R←S+A~S×∇1+|B-A
      ∇

      3 TO 8
3 4 5 6 7 8
      5 TO -3
5 4 3 2 1 0 -1 -2 -3

```

So all seems to work well. Incidentally, the checking of the first *TO* did not use a negative *A* or *B* at all, which was rather naughty. Also we ought to have checked what happens when *A* equals *B*.

```

      5 TO 5
5
      ρ5 TO 5
1

```

This last result shows that *TO* always returns a vector, a design feature we didn't mention, though possibly of crucial importance in some applications. (Also note that the intermediate variable used *S* is declared to be a local variable when the function has been tested.)

So much for *TO*? No, there's more to come. Suppose we didn't want to index using consecutive integers, but rather jumping in steps? (Rather like the BASIC statement FOR I=2 TO 10 STEP 2.) Perhaps the simplest design for such a function is to allow the right argument to be two-element vector, with the second element (if present) specifying the step size. Here is the third possibility.

```

      V R←A TO B_ST;B;STEP
[1]      □IO+1 ◊ B+1+B_ST←,B_ST
[2]      STEP+(1,1+B_ST)[1+1<ρB_ST]
[3]      R←(-STEP)+A+STEP×ι1+[(B-A)÷STEP]
      V
      2 TO 10 2
2 4 6 8 10
      3 TO 10 3
3 6 9

```

The second example illustrates a hole in the design specification - I didn't specify what should happen if the end-point is missed - do we under-shoot or over-shoot? Again either choice is easy to program, but not to decide is dangerous when the function is to be used by other people.

And finally, another possible design for an index function, which poses an interesting problem for APL programmers (using first generation APL), is to allow more than one sequence of index numbers. For example:

```

      1 5 15 TO 3 8 19      a results in ....
1 2 3 5 6 7 8 15 16 17 18 19

```

Without the luxury of nested arrays that second-generation APL interpreters allow, this is tricky, but not impossible. Here is the approach: the largest run is of length 5, and three runs are required. Hence generate the matrices:

1 2 3 4 5	0 0 0 0 0		1 1 1 0 0
1 2 3 4 5	4 4 4 4 4	and	1 1 1 1 0
1 2 3 4 5	14 14 14 14 14		1 1 1 1 1

Denote $A \leftarrow 1 \ 5 \ 15$, $B \leftarrow 3 \ 8 \ 19$, $N \leftarrow 1 + \lceil B/A \rceil$ (5 in this case), and $D \leftarrow (\rho A), N$. The three matrices are then given by $D \rho 1 N$, $D \rho N/A - 1$ and $(1+B-A) \circ . \geq 1 N$

Adding the first two matrices gives more than the numbers required, and the rows of the third matrix tell you which of the elements to keep and which to throw away. With the use of $'$ to concatenate matrices into vectors, we get

```
(, (1+B-A) \circ . \geq 1 N) / , (D \rho 1 N) + D \rho N/A - 1
1 2 3 5 6 7 8 15 16 17 18 19
```

Here is the final version of *TO*:

```

V R←A TO B;N
[1]  IO←1 \ N←1+⌈B/A⌉ \ D←(ρA),N
[2]  R←(, (1+B-A) \circ . \geq 1 N) / , (D \rho 1 N) + D \rho N/A - 1
V
```

Or is it? Perhaps we should allow decreasing sequences here too, e.g.

```

1 10 TO 5 7
1 2 3 4 5 10 9 8 7
```

in which case we need to modify *TO* above to be

```

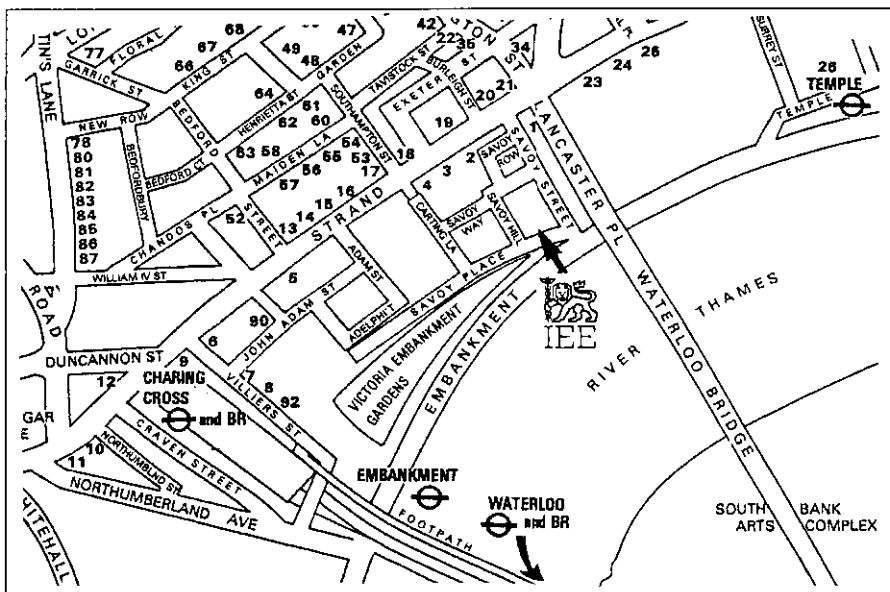
V R←A TO B
[1]  IO←1 \ N←1+⌈|B-A|⌉ \ D←(ρA),N
[2]  R←(, (1+|B-A|) \circ . \geq 1 N) / , (D \rho 1 N) \times \Phi(\Phi D) \rho \times B-A + D \rho N/A - \times B-A
V
```

And finally, perhaps you too have your own favourite $(A) \ TO \ B$?

British APL Association Monthly Meetings

From February 1991, we have decided to change the venue of our monthly meetings in London. Meetings will now be held at the **Institute of Electrical Engineers**, Savoy Place, London (nearest tube outlets: Temple or Embankment). The committee believe that this move will greatly enhance the quality of our monthly meetings - but they are still free, and of course non-members are still most welcome. Come along and see if we've made the right decision.

Date	Venue	Event
8 February 1991	IEE London	BAA Meeting: Resolve Award + British Airways - see full programme on page 6.
22 March	IEE London	BAA Meeting: The Launch of the APL Statistics Library
26 April	IEE London	BAA Meeting: Vendor Forum
7 June	IEE London	AGM and BAA Meeting: Modern Productivity Aids for APL





Renaissance Data Systems

Enlightenment Thru Information Processing

ALL APL BOOKS IN PRINT

Catalog excerpts as of September 1, 1990

ACCOUNTING STRUCTURED IN APL , Ijiri. 1984, 491p.	\$10.00
The basic principles of accounting with numerous APL expressions to model them.	
COMPUTING IN STATISTICAL SCIENCE THRU APL , Anscombe. 1981, 426p.	\$42.00
Lots of statistics and APL. His workspaces are available from Yale University. A classic.	
MATHEMATICAL EXPERIMENTS ON THE COMPUTER , Grenander. 1982, 525p.	\$64.00
Case studies using APL in a number of fields, including statistics, linear algebra, geometry, asymptotics, neural networks, invariant curves. Detailed description and analysis of APL functions in all topics discussed.	
PROBABILITY IN APL , Alvord. 1984, 142p.	\$17.50
A delightful, yet serious development of 31 functions for fun and learning with roll, deal, binomial distributions, combinations, permutations, geometric distributions, and the World Series. Friendly examples. No previous APL required.	
DISKETTE FOR ABOVE containing functions covered in book.	\$22.00
APL WITH A MATHEMATICAL ACCENT , Jones, Reiter. 1990, 200p.	\$41.95
Specially written for use in advanced high school or college math courses. By a student of Alvord's made good!	
APL - THE LANGUAGE AND ITS ACTUARIAL APPLICATIONS , de Kerf, Goovaerts, Stiers. 1987, 223p.	\$89.00
Introduction to APL. Loss reserves, credibility, probability, numerical analysis, forecasting, with APL functions.	
COMPUTATION FOR THE ANALYSIS OF DESIGNED EXPERIMENTS , Heiberger. 1989, 683p.	\$59.95
Analysis of the construction of ANOVA programs using least squares techniques, including the parsing algorithms by which a language specification is interpreted. Emphasizes the geometry as well as the algebra of the methodology. All programs in the book are included on 5 1/4" diskettes in APL, BASIC, C, and FORTRAN.	
APL2 AT A GLANCE , Brown, Pakin, Polivka. 1988, 444 p.	\$35.00
Solid, unimpeachable, introduction to APL2. Clear illustrations, modern exercises in each chapter. Gets you started.	
LEARNING APL: AN ARRAY PROCESSING LANGUAGE , Mason. 1985, 259p.	\$21.50
Emphasis on arrays, naturally. A readable, detailed introduction with thorough examples.	
APL - ADVANCED TECHNIQUES AND UTILITIES , Berquist. 1986, 450p.	\$44.95
Good discussion of alternative approaches to a wide range of programming tasks - efficiency, searching, dates, workspace documentation, file design, boolean techniques. Assumes knowledge of the basics. Many idioms.	
DISKETTE FOR ABOVE	\$15.00
Contains all functions described in text. Formatted for APL*PLUS/PC, uploadable to Sharp APL, VS/APL, APL2.	
APL AS A TOOL OF THOUGHT, PROFESSIONAL DEVELOPMENT SEMINARS, NY/SIGAPL : The first 5 years.	\$55.00
A wide range of topics in education and business. Logic, insurance, statistics, A. I., accounting, fractals, teachers toolbox, computer science, biology, graphics, engineering, data bases, and much more. 1983 - 1987, approximately 600p.	
APL89 - STATISTICS TUTORIAL , Alvord, Traberman, et al. 72p.	\$10.00
A unique collection of papers on statistics and its exposition in APL.	

Name: _____	Date: _____	Payment in check or money order payable in \$U. S. must accompany order. Send to: Renaissance Data Systems P. O. Box 20023 Park West Finance Station New York, New York 10025-1510 (212)884-3078
Street: _____		
City: _____ State: _____ Postal Code: _____		
Country: _____ Telephone Day: _____ Evening: _____		

We regret that we cannot accept purchase orders or credit cards.

Title _____	Price _____	Quantity _____	Total _____
Title _____	Price _____	Quantity _____	Total _____
Title _____	Price _____	Quantity _____	Total _____
Subtotal: _____			
Postage and handling - U.S. and Canada (\$2.50 minimum; \$2.50 each item over \$25.00): _____			
- International (surface mail add 15%, airmail add 40%): _____			
Total: _____			

Thank You For Your Order!

Please send complete CATALOG of APL books and software ☐

THE RANDOM VECTOR

January 1991

The Newsletter of the APL Statistics Library

Editor David Eastwood

This *Random Vector* has been reprinted from VECTOR Vol.7 No.3. VECTOR is the Quarterly Journal of the British APL Association; for more information about the British APL Association, contact the Secretary: Anthony Camacho, 2 Blenheim Rd, St Albans, Herts AL1 4NR Tel: 0727 860130.

Contents

Editorial	David Eastwood	34
The ASL team	David Eastwood	34
ASL News	David Eastwood	35
The Genesis of ASL	Tony O'Hagan	36
The Basic Statistics Volume		38
WAGS: A Graphics Specification Language for ASL	Tony O'Hagan	41

Editorial

The APL Statistics Library project has now been formally running for over a year, with invitational conferences at Gregynog in 1989 (Vector 6.3) and Swansea in 1990 (Vector 7.3). The ASL project, for those readers who have not seen any of the previously published material, is an initiative launched by a number of users of APL for statistical work - largely in higher education. As you will see from Tony O'Hagan's article later in this newsletter, the purpose behind the project is to underline the usefulness of APL for statistical research and teaching purposes by providing a tested and approved library of statistical routines written in APL.

The British APL Association has been sponsoring the ASL project over this period and has kindly agreed to dedicate a section of Vector to news of the ASL project. It is hoped that we can find a steady series of items which cover the progress of the ASL project and which are also of interest to the general reader. In this issue we begin to look at the history of the ASL project (see also Vector 7.1), review the current status of the project as it runs through the beta test phase for its first volumes and look in a little bit of detail at the concepts that underpin one of the initial 'Bottom Shelf' volumes - Basic Graphics.

The ASL Management Team

Perhaps the most important thing for many of you is to know who is running ASL. The management of the project is conducted by the Management Team comprising a Chairman, Deputy Chairman, Manager and two representatives of the British APL Association (BAA). BAA representation is important because it is they who are funding the project, and we are consequently extremely grateful for their support.

Strategic Planning and all major issues are dealt with at full Team meetings, but day to day running is handled by the Manager. Names and addresses are as follows.

Chairman:	Tony O'Hagan, Department of Mathematics, University of Nottingham, University Park, Nottingham NG7 2RD. Telephone: 0602-484848 x2800
Deputy Chairman:	Alan Sykes, Department of Management Science and Statistics, University College of Swansea, Singleton Park, Swansea SA2 8PP. Telephone: 0792-295296

Manager:	Jake Ansell, Business Studies, University of Edinburgh, William Robertson Building, 50 George Square, Edinburgh EH8 9JY. Telephone: 031-667-1011
BAA Representative/ Newsletter Editor:	David Eastwood, MicroAPL Ltd., South Bank Technopark, 90 London Road, London SE1 6LN. Telephone: 071-922-8866
BAA Representative:	John Searle, 13A Mount Ararat Road, Richmond, Surrey TW10 6PQ. Telephone: 081-948-6737 (home)

ASL News and Status - December 1990

I would like to start by reviewing the way in which the ASL project is organised. It should be stressed that with the exception of the part-time input of our Programmer (see below), all the effort put into ASL is done on a voluntary basis. Whilst it is extremely gratifying to have the willing help and assistance of so many people, they all have other things to do and this does impose different timescales to full-time commercial projects.

The Programmer

There are two main ways in which ASL is funded by the BAA. One is by running working conferences or workshops like the one just held in Swansea. The other is by paying for a part-time programmer. She is Tuks Gillespie, who has substantial experience in APL programming and is proving an extremely valuable member of the project. Organising the effective use of Tuks' time is the responsibility of the Manager, Jake Ansell.

Working Parties

The actual specification and planning of ASL software is done by Working Parties, who liaise with the Management Team and the Programmer through Jake Ansell. One key way in which you can support ASL is by serving on Working Parties. So why not write to Tony O'Hagan (address above) and tell him which aspects of statistical software you would be interested in helping to develop? Is there an area of statistical methods in which you have a great interest or expertise? Are you keen to help with the graphics, the numerical algorithms or other utilities that will serve the statistical functions? Or could your experience in developing large non-statistical APL systems be valuable to us in our general operations, or in managing our growing library of functions? WRITE NOW, please.

Shelves and Volumes

ASL is a library, which suggested the word "shelf" to cover a collection of functions dealing with a common subject area. Thus, there will be a Regression Shelf. Within a Shelf, there is a smaller unit called a "volume". A volume is a coherent group of functions that are released together. So shelves grow by the addition of a volume at a time. There is a shelf called the Bottom Shelf, concerned with low-level functions that are likely to be usable both directly and within higher shelves.

The Official Launch

We currently plan to launch ASL by the simultaneous release of three volumes at a special meeting of the BAA on 22 March 1991. The first software to be released will be:

Bottom Shelf Volume 1, Basic Statistics
Bottom Shelf Volume 2, Basic Graphics
Regression Shelf Volume 1, Generalised Linear Models.

The Genesis of ASL (1) April 1988

by Tony O'Hagan

Talk of something like ASL goes back at least three or four years. Jake Ansell had voiced the idea at various meetings of the APL Statistics User Group (which he has almost single-handedly nurtured for some considerable time). In early 1988 it began to be taken seriously and things started to happen.

The spark was a gathering at Warwick University to launch its APL2 service. This was to make APL2 on Warwick's mainframe available over the academic computing network JANET to other universities. (It was beset with problems over translation tables, etc., and was not a success, but that is another story). Several statisticians were present, the idea of a functions library was briefly discussed and they agreed to meet again to consider it carefully.

This meeting was on 18th April 1988 at Warwick University, attended by Jake Ansell, Phil Dawid, Brian Newman, Andy Pole, Ewart Shaw, Norman Thomson and myself. Warwick's APL2 service prompted the natural suggestion of disseminating the library electronically from a repository at Warwick, an idea which has not actually been discussed lately but remains an option.

It is interesting to look at some of the topics discussed at this April 1988 meeting.

Language. The inevitable question arose of whether to work in ISO standard, first generation APL with enhancements over the standard like diamond separators and replicate, or some form of second generation APL. We agreed to use ISO, with extensions that could be converted between dialects automatically. But we also agreed to allow direct definition and to accept functions in APL2. (Norman Thomson was as persuasive as ever!)

Style. A consistent style was recognised as being important. We agreed that "stylistic questions to be considered include naming of arguments and other variables, and how much internal documentation should be provided in the form of comments". A distinctive naming convention was recommended for global variables, and standard "structured programming" functions like IF and WHILE were suggested.

Graphics. The progress made at Warwick, partly driven by its CTI project, on a single consistent graphics interface was recognised at this early meeting. This is the first time that it was given the name of WAGS - the Warwick APL Graphics Standard.

From these headings it can be seen that some of the most persistent topics of discussion in ASL meetings were already established. We had already hit the problems of Language and Naming that have taken a great deal of work to resolve. However, the major preoccupation in the early days was funding. This was item 1 on the agenda in April 1988. Even though the two suggestions made at the time turned out to be non-starters, it is interesting to take a look at them.

- (a) Approach the Computer Board/UGC, whose Computers In Teaching Initiative (CTI) had funded a project at Warwick into the use of APL in Statistics teaching.
- (b) Obtain a Science and Engineering Research Council (SERC) grant, perhaps a Research Grant for a PhD student to develop software.

The CTI initiative at that time was inviting proposals for setting up centres for educational software information and exchange in different subject areas. In Mathematics and Statistics, such a centre was subsequently established jointly at the Universities of Birmingham and Glasgow. We would really only have been interested in an APL service, which would have been too narrow a formulation for CTI, so this approach was dropped. However, it was decided that it would be useful in any such proposal to have letters of support from academic statisticians, so this was done and my files include letters from John Copas, Phil Dawid and Alan Hawkes.

The idea of a PhD project also seemed highly unlikely to have enough academic challenge, so the whole functions library idea lapsed until it was revived again at a meeting in October..... which is where the story will continue in our next Newsletter!

Within ASL - Basic Statistics

The first three volumes of software have been issued for beta test. In this issue of our newsletter, I shall be taking a quick look at what is in one of our first volumes of software - the Basic Statistics volume. Subsequent issues will look at the other shelves.

The Basic Statistics volume owes much to the work of Norman Thomson who made the initial suggestions regarding the scope of the Bottom Shelf at Gregynog in 1989 (and at earlier meetings). Norman's document (called, not surprisingly, 'The Bottom Shelf') contains details of virtually all the statistical routines contained in the Basic Statistics volume, using APL2 as the specification language. The ASL Documentation Working Party is currently aiming to bring Basic Statistics documentation and functions fully into line. As with other ASL documents, copies of Norman's paper are obtainable from the Management Team.

A review of the contents of the Basic Statistics Volume gives the best idea of its scope, as in most cases the function name alone tells us all we need to know. The workspace contains almost 100 functions, with some duplication covering alternate algorithms (coding for speed, elegance, workspace economy with variants using numerical suffixes). Symbol table usage is some 700 symbols.

The contents of the Basic Statistics volume is designed to address the needs of 'A' level or first year university students, with groupings suggested as follows:

- Univariate Statistics
- Distribution Functions
- Elementary multi-variate statistics
- Estimation and significance testing
- Non-parametric statistics
- Times series
- ANOVA
- Combinatorics

The full list of functions in the beta test version of this volume of ASL follows.

FUNCTION CATALOGUE

NAME	Syntax	SIZE	COMMENT
BARChart	R+ DB	780	a FN: Visual representation of frequency table
BINDEF	R+AOB	816	a FN: Values of the binomial probability density function
CHICDF	R+AOB	1550	a FN: Cumulative function values of the chi-square distribution
CHICONT	R+ DB	550	a FN: Calculate chi-square value of a contingency table
CHIDEN	R+AOB	759	a FN: Values of the chi-squared probability density function
CHIGOF	R+AOB	672	a FN: Calculate goodness of fit chi-square value
CHIQUNT	R+AOB	1008	a FN: Calculate chi-square quantiles
CHITAIL	R+AOB	1675	a FN: Upper tail probability of chi-square distribution
COLUMN	R+AOB	550	a FN: Selects columns from a matrix
COMB	R+AOB	994	a FN: All combinations, of a certain length, from a dataset
CORR	R+AOB	819	a FN: Calculates the correlation coefficient or product moment
CORRMAT	R+ DB	1056	a FN: Calculates correlation matrix
COVMAT	R+ DB	845	a FN: Calculates covariance matrix
EWMA	R+AOB	1330	a FN: Exponentially weighted moving average of a time series
FDEF	R+AOB	840	a FN: Returns values of the F probability density function
FREAD	R+ DB	1364	a FN: Read whole native file as characters
FREQ	R+ DB	1403	a FN: Returns the frequencies of unique values in a data set
FREQTAB	R+AOB	2077	a FN: Converts a dataset to frequency table
FREQTABIRREG	R+AOB	910	a FN: Converts dataset to frequency table with irregular intervals
FTAIL	R+AOB	2150	a FN: Calculate upper tail probabilities of F distribution
FWRITE	AOB	2232	a FN: Create and write to a native file
GAMDEF	R+AOB	864	a FN: Returns values of the gamma probability density function
HYPGEDEF	R+AOB	936	a FN: Values of the discrete hypergeometric probability function
INTADAPT	R+AOB	1340	a FN: Adaptive integration
INTSIMP	R+AOB	812	a FN: Simpsons numerical integration
KENDALCON	R+ DB	1587	a FN: Calculate Kendall's concordance, a rank corr'n coefficient
KENDALCONTS	R+ DB	1650	a FN: Calculate Kendall's concordance test statistic
KENDALTAU	R+AOB	1560	a FN: Calculate Kendall's Tau, a rank correlation coefficient
KENDALTAUTS	R+AOB	1525	a FN: Calculate Kendall's Tau, a rank correlation coefficient
KURTOSIS	R+ DB	792	a FN: Calculates kurtosis of a set of values
MANNWHIT	R+AOB	672	a FN: Mann-Whitney rank sum signed rank test
MANNWHITTS	R+AOB	1105	a FN: Mann-Whitney rank sum signed rank normalised test statistic
MAX	R+ DB	756	a FN: Calculates maximum of a set of values
MEAN	R+ DB	720	a FN: Calculates mean of a set of values
MEANFT	R+ DB	505	a FN: Calculates mean of a frequency table
MEDIAN	R+ DB	590	a FN: Calculates median of set of values
MIDPOINTS	R+ DB	600	a FN: Calculates midpoints of increasing pairs of endpoints
MIN	R+ DB	756	a FN: Calculates minimum of a set of values
MODE	R+ DB	969	a FN: Calculates mode of set of values
MOMENT	R+AOB	524	a FN: Calculates a moment about the origin
MOVA	R+AOB	1140	a FN: Calculate moving average of a time series
NORMCDF	R+AOB	1224	a FN: Calculate cum function values of a normal distribution
NORMDEF	R+AOB	1088	a FN: Returns values of the normal probability density function
NORMQUANT	R+AOB	1320	a FN: Calculate normal statistic values or quantiles
NORMTAIL	R+AOB	1224	a FN: Calculate upper tail areas of standard normal distribution
ON	R+AOB	418	a FN: Combines datasets into a matrix
ONEWAY	R+AOB	1876	a FN: Calculate oneway analysis of variance
PCTILE	R+AOB	756	a FN: Calculates percentile of set of values

PERM	R+ OB	1511	a FN: Generates all permutations of order num
POISDEN	R+AOB	768	a FN: Returns values of the poisson probability function
RANDBOOL	R+AOB	768	a FN: Generates random boolean values
RANDEXP	R+AOB	649	a FN: Generates random values from exponential distribution
RANDNORM	R+AOB	966	a FN: Generates random values from normal distribution
RANDNORM3	R+AOB	1449	a FN: Generates random values from normal distribution
RANDPOIS	R+AOB	1624	a FN: Generates random values from poisson distribution
RANDPROBTAB	R+AOB	715	a FN: Generates random values from discrete distribution
RANDWEIB	R+AOB	672	a FN: Generates random weibull distribution values
RANDBIN	R+AOB	636	a FN: Generates random values from binomial distribution
RANDUNIF	R+AOB	737	a FN: Generates random values from uniform distribution
RANGE	R+ OB	732	a FN: Calculates range of a set of values
RANK	R+ OB	450	a FN: Returns the ranks of a data set
REG	R+ OB	798	a FN: Calculates least squares regression coefficients
REGEST	R+ OB	728	a FN: Predicts dependent values using regression
REGLIN	R+ OB	792	a FN: Calculates simple linear regression coefficients
REGLINEST	R+ OB	792	a FN: Predicts dependent values using simple linear regression
REGLINREP	R+ OB	1820	a FN: Reports on simple linear regression
REGLINRESID	R+ OB	828	a FN: Calculates residuals of simple linear regression
REGPOLY	R+AOB	975	a FN: Calculates least-squares regression polynomial coefficients
REGPOLYEST	R+AOB	896	a FN: Predicts dependent values using polynomial regression
REGPOLYREP	AOB	1944	a FN: Reports on the polynomial least squares regression
REGPOLYRESID	R+AOB	896	a FN: Calculates residuals of polynomial regression
REGREP	R+ OB	1736	a FN: Reports on least squares regression
REGRESID	R+ OB	720	a FN: Calculates residuals of least squares regression
RUNS	R+ OB	560	a FN: Calculates the maximum run length in a data vector
SDEV	R+ OB	840	a FN: Calculates standard deviation across first dimension
SDEVFT	R+ OB	804	a FN: Calculates standard deviation of a frequency table
SEASA	R+AOB	1272	a FN: Seasonally adjust time series
SEMIQR	R+ OB	876	a FN: Calculates the semi interquartile range $(Q3-Q1)+2$
SIGN	R+AOB	795	a FN: Calculates the no. of positive signs from the median
SIGNYS	R+AOB	870	a FN: Calculates the normalised sign test statistic
SKEWNESS	R+ OB	804	a FN: Calculates skewness of set of values
SPEARMAN	R+AOB	550	a FN: Calculate spearman correlation coefficient
STANDARDISE	R+ OB	858	a FN: Standardises a set of values
STATS	OB	1566	a FN: Calculates various statistical summaries of a set of values
STRIPMEAN	R+ OB	624	a FN: Subtracts the mean from a set of values
STRIPOFF	R+AOB	870	a FN: Subtracts constant(s) from a set of values
TCDF	R+AOB	1464	a FN: Calculate cumulative function of Student T distribution
TDFN	R+AOB	871	a FN: Returns values of the Student T probability density function
TQUANT	R+AOB	1403	a FN: Calculate T statistic values or quantiles
TSTAT	R+AOB	935	a FN: Calculates the one-sample T statistic
TTAIL	R+AOB	1541	a FN: Calculate upper tail area of student T distribution
TWOSAMT	R+AOB	880	a FN: Calculates the two-sample T statistic
TWOWAY	R+ OB	1809	a FN: Calculate oneway analysis of variance
UNIQUE	R+ OB	624	a FN: Returns the unique values in a data set
VAR	R+ OB	671	a FN: Calculates variance of set of values
VARFT	R+ OB	732	a FN: Calculates variance of a frequency table
VARRATIO	R+AOB	504	a FN: Returns variance ratio of 2 datasets
WILCOXON	R+AOB	780	a FN: Wilcoxon signed rank test (allows for rank ties)
WILCOXONTS	R+AOB	1104	a FN: Wilcoxon signed rank test statistic, normalised form

WAGS: Graphics Specification Language for ASL

by Tony O'Hagan

Introduction

ASL, the APL Statistics Library, has been reported on several times in 'Vector'. The objective is to establish a comprehensive library of APL functions for use by statisticians. APL is a particularly suitable language in which to explore possible analyses of data, but the spread of APL in the statistics community has been hindered by the need for the user to program all the common statistical operations from scratch. ASL will attempt to remove this blockage, making statistics in APL as accessible as the huge wealth of statistical packages currently available. ASL will improve on such packages in two ways. First, it will not be a closed system, but rather the whole power of APL will be available to the statistician. Second, ASL has the support of several eminent academic statisticians. Their input will help to ensure that ASL functions are accurate, reliable and represent the best in current statistical methodology.

Graphical presentations are an important part of modern statistical analysis. Clearly, ASL must meet this need by providing functions that produce graphical output where appropriate. The major problem is the great variety of graphical output devices and the absence of a standard way for APL interpreters to handle graphics. We would like the ASL functions to be as portable as possible. We certainly do not want different versions of every function using graphics, for each different graphics system that might be available to the user.

Warwick University faced the same problem with its 'APL in Statistics' project. That project was supposed to produce software for teaching Statistics using APL which was to be freely available to the academic community. It was not desirable to make the software dependent on a single graphics system, nor to rewrite everything for a range of systems. The answer was to define a core set of routines to perform all primitive graphical functions. This core would need rewriting for each new system, but all the software would then be immediately portable to any system on which the core had been implemented. The resulting graphics specification language was called the Warwick APL Graphics Standard, or WAGS. The first version of WAGS was created by Ewart Shaw, and subsequently many important extensions were added by Andy Pole.

A thoroughly revised version has been adopted as the definitive specification of the ASL graphics primitive language. In preparing this revision the author received enormous help from Alan Sykes (University College of Wales, Swansea), Ewart Shaw (University of Warwick), Andy Pole (Bull Secure Systems) and John Hinde (University of Exeter). All of these people are co-authors of WAGS.

This article describes the main design features of WAGS. Another document - 'WAGS: Function Specifications, Version 1' - presents a full specification of WAGS, and is available from the author. The full specification will be of interest to three different groups of people. First, WAGS must be implemented on a variety of APL systems, so the specification serves as a blueprint for the WAGS implementer. Second, ASL will contain many functions which create graphics at a much higher level than WAGS, but they will all do so by calls to the WAGS primitives: the specification of WAGS acts as a user manual for ASL graphics programmers. Finally, the end user of ASL may have occasional need to understand the WAGS system that underlies the ASL graphics functions. After all, the initial audience for ASL will comprise statisticians who already know and use APL. ASL is deliberately an open collection of functions, not a package. The knowledgeable user is free to modify functions in ASL for his or her own purposes, and will need to understand WAGS in order to modify graphics functions.

The following sections describe the key features of WAGS.

Getting in and out

The basic usage of WAGS for any graph is

```
GOPEN
```

```
code to draw graph
```

```
GINTER
```

```
GCLOSE
```

GOPEN initialises graphics. This is physically necessary in an IBM-PC environment to switch from the text screen to the graphics screen. In many other systems there will be a need to do some initialising of the hardware, and there will always be some software housekeeping to do. *GCLOSE* simply closes down the graph. It may switch back to the text screen on a PC, return the pen to rest on a plotter, etc.

GINTER is a very important preliminary to *GCLOSE*, and it may also be invoked at other stages in drawing the graph. At the very least, it will halt operation until the user presses a key. This is necessary on a PC to stop the graphics screen disappearing before the user has had a chance to look at it, but the pause effect can be useful elsewhere. More importantly, *GINTER* is intended to allow the user to add his/her own things to the graph, interactively. He/she may wish to add a caption, draw an arrow to highlight one point in the graph, delete something, etc. The range of such features provided by *GINTER* is not specified in WAGS. There are several reasons. First, it is no concern of the programmer, since *GINTER* simply gives the user control where the programmer leaves off. Second, the available *GINTER* options will be system dependent. For instance, a screen might allow deletions but a pen plotter will not. Third, specifying *GINTER* would deny implementers of WAGS the freedom to provide the best features which each system can provide. Fourth, WAGS can be implemented quickly on a system by providing a skeleton *GINTER*, and this can then be enhanced at leisure.

Ambiguity in WAGS

There are other places in WAGS where the precise functionality is not specified but is left to the implementer to define. How does this ambiguity affect the three groups of people who might use the specification document? First, it gives the implementer freedom to exploit the system's graphical capabilities to the full.

For the programmer of higher level ASL graphics functions the ambiguities in WAGS are a great simplification. For instance, to distinguish between plotted lines that represent different things, he/she might wish to plot them in different colours or different types of line (e.g. dotted or dashed). WAGS does not require the programmer to know (or to enquire) whether the graphics device on which the program will run has a colour capability, or what kinds of dotted and dashed lines it can draw. The programmer simply asks for colour-type combination *i*. What this produces is left to the implementer, and it will certainly produce a different effect on different implementations. The ambiguity is intentional. To provide facilities for the programmer to enquire about all possible configurations, and then to require the programmer to handle all these possibilities would make WAGS enormously complex, and would make the programmer's task very difficult. Instead, the programmer only needs to know that colour-linetype *i* is different from colour-linetype *j*.

For the user of ASL, the ambiguity is no problem either. No amount of clever programming could be guaranteed to produce graphs that always satisfy the user completely. But if the implementer has been able to set up pleasing colour-

line-type combinations, etc., then ASL graph drawing functions should produce good graphs with minimal effort from the user. And if the user would prefer different colours, shadings, etc., WAGS provides a very simple way of making those changes. A function *GCONFIG* allows him/her to over-ride the default sets of colours, line types, colour-line-type combinations, etc., provided by the implementer. Then running the graph again with the new settings will achieve the desired effect.

Windows and Scaling

A favourite graphical technique is to define different areas of the screen (or page) as windows, each addressed with its own coordinates. WAGS covers this idea very simply by allowing the whole screen to be addressed in variable coordinates. By default, *GOPEN* initially defines the screen so that the lower left-hand corner is (0,0) and the upper right (1,1). We say the screen has coordinates (0,0;1,1). If we then want to refer to the upper left-hand quarter of the screen as a window, with its own coordinates (0,0;1,1), this is equivalent to giving the whole screen coordinates (0,-1;2,1). WAGS allows new screen coordinates to be set up at any time by assigning new coordinates to any pair of points. So instead of defining the above example as rescaling (0,0;1,1) to (0,-1;2,1), we could have said it was rescaling (0, 1/2;1/2,1) to (0,0;1,1). This is done by a function *GSCALE*.

This general approach is very convenient in drawing conventional (x,y) graphs. The screen scaling is simply defined to agree with the natural (x,y) scales of values. So if the x values range from x1 to x2 and the y values from y1 to y2, and if we want to plot these values in a part of the screen whose bottom-left and top-right corners are (a,b) and (c,d) in the current scaling, the new scaling is created by:

```
(A,B,C,D) GSCALE (X1,Y1,X2,Y2)
```

Function *GASPECT* returns the aspect ratio of the screen/page, so whereas *GOPEN* initially establishes the screen coordinates as (0,0;1,1), the statement

```
0 0 1 1 GSCALE 0 0 1,GASPECT
```

will then change the scaling so that equal scale distances in the x and y directions correspond to equal physical distances on the screen. Whereas *GASPECT* always gives the true aspect ratio, an aspect ratio relative to any current scaling can be found using a function *GANGLE*, which reports the true angle formed by any two points given in the current scaling.

Colour

Where colour is available, different systems have different ways of referring to colours; some have only a few available and some have a vast range. In WAGS, *GOPEN* establishes a list of available colours and the programmer simply requests colour *i* in that list. The programmer will not know what colour 'colour *i*' is, simply that it is different from the other colours. Function *GCOUNT* reports how many different colours are available. On a monochrome system this will be one. On a colour system, *GOPEN* will set a default number and sequence of colours, but the user can over-ride these with *GCONFIG*. He/she can therefore change the sequence of colours, or even make a colour system behave as monochrome.

Types of Line

Some systems can draw lines as dots or dashes, or in different thicknesses. WAGS needs to make full use of whatever features are available. The technique is the same as for colour. There is a list of different types of line available, created by *GOPEN*, which may be the default sequence or specified by the user with *GCONFIG*. As a general rule, programmers wishing to distinguish between two lines in a graph can use colour or line type or both. The program might use colours if enough are available, and otherwise use line types. WAGS provides another feature to simplify this process. When drawing lines, the programmer can either specify both colour and line type separately, or give a single number to select a colour-line-type combination.

Shading

Another variable feature of graphics devices is the ability to fill an area of the page or screen with colour. Even if that is not possible, as on a pen plotter, a comparable visual effect is provided by cross-hatching the area. *GOPEN* establishes a default sequence of types of shading. Different WAGS implementations may interpret shading in terms of various forms of hatching, various shades of colour (e.g. various grey shades on a monochrome screen) or various dot-densities. Again, the user can over-ride the default sequence by *GCONFIG*.

Text

Some systems can write text in various fonts in a range of sizes. Now different colours, line types or shadings are generally used simply to distinguish between similar graphical elements. The situation is different with text. Labelling of

graphs should mostly be done consistently in one font and size. Changes of font are not needed just to distinguish between things; the distinctions are contained in the text itself. Using many different fonts or sizes tends to create a very messy picture. Nevertheless the user may want some font changes. The caption to a graph may be preferred in a bold or italic font, or in a larger size, for instance.

The implications of these comments are as follows. First, WAGS recognises a sequence of fonts, in the same way as there are sequences of colours, line types and shadings. (The same font style in a different size is technically understood as being a different font, so the notion of a font encompasses both style and size.) However, *GOPEN* establishes the default sequence as containing only one font. The programmer should nevertheless call for different fonts wherever the user might conceivably want different fonts. Now the convention in WAGS is that the sequences of colours, line types, shadings and fonts wrap around, e.g. asking for colour 6 when only 5 are available yields colour 1, asking for colour 7 yields colour 2, and so on. Therefore, with only one font available this font will be provided whichever font number is requested. This devious technique allows the user to obtain whatever fonts he/she requires simply by over-riding the *GOPEN* default sequence.

Another feature of fonts is that the user or programmer will often need to know how much space a piece of text will occupy in a given font. A function *GSPACE* is provided for this purpose.

Global Variables

A cleaner system results if the programmer is working always through functions, rather than some of the operations being to assign values to pre-defined global variables. For instance, to redefine the current scaling of the screen/page it is neater to use the function *GSCALE* as above rather than assigning a value to a global variable. When some operations are functions and some are not, it is hard to remember which is which. Also, it is easier to work with a hard-copy printout of a workspace when the only things of interest are functions.

Global variables will of course be needed within WAGS (to store the current scaling, for instance), but the implementer is free to define them in the most natural and efficient way. Neither the user nor the programmer need know what they are or what they do.

Redrawing

Graphs frequently display the results of quite extensive calculations. When we wish to redraw a graph it would be tedious to repeat those calculations. If the calculations and graphics are in separate functions, then the graph can be redrawn quickly, but only if a copy of the workspace has been saved with all the necessary variables. ASL provides the following facility for redrawing graphs.

A function *GSAVE* is called to switch on the graph saving feature. Then as each core WAGS function is invoked, a transcript is built up of the graphical functions being used. The transcript is actually a function which, when called, redraws the graph. (Another function *GFIX* finally fixes this function from a canonical representation which is built up by the other functions.) In the transcript, all arguments will be explicit numeric data, so it is not necessary to refer to any other variables: saving the single transcript function saves the graph. Redrawing the graph is quick because the explicit arguments incorporate the results of all necessary calculations. Interactive drawing commands issued from the keyboard to *GINTER* are recorded in the transcript, too. The transcript may be edited before being redrawn, for instance to include changes of line type.

GSAVE and *GFIX* need not be used to save an entire graph. For instance, calling *GSAVE* before and *GFIX* after a complicated sequence of operations to draw a set of axes will yield a function that can quickly redraw those same axes as part of another graph. The graph saving feature is another reason for requiring all operations to be done through functions, because it is not possible to trap and record assignments to global variables.

Graph saving could have been provided as an entirely separate implementation of WAGS. In that case, after the graph saving WAGS implementation had been copied into the workspace, a graphics function in ASL would not actually draw on any device, but would produce the saved transcript instead. However, it is more convenient in practice to have the transcript being saved simultaneously with the graph being produced on the screen. If interactive drawing operations performed with *GINTER* are also to be saved, then it becomes essential to combine saving with drawing in this way. A user is, of course, likely to have two or more different versions of WAGS. The WAGS implementation that draws on the screen, for whatever APL interpreter and hardware he/she is using, will be his/her main version of WAGS, and will include graph saving. Another implementation might drive a laser-printer by producing a PostScript file. After previewing the graph on the screen, saving the final version, and perhaps editing the saved function, the user will simply run the saved function through this other WAGS implementation to produce the hard copy.

Running the saved function through another implementation of WAGS will not necessarily produce an identical graph. This is to be expected because different devices will have different capabilities. It is, however, important to realise that running the saved function on a different device may even produce a different result from running the original function on that device. If the original function calls *GSPACE* to ensure that a piece of text will fit a given space, then the saved function will place it there. On the new device the same text in the same font number may not fit the space, and running the saved version of the function will produce an unsightly result. Running the original function would call *GSPACE* afresh, find that the text will not fit, and could take appropriate alternative action. Saving a graph is a reliable way of rerunning it through the same implementation of WAGS. When using a different WAGS implementation, however, such as for producing a printout, it will often be necessary to run the original function again to achieve the best result.

Second Generation APL

ASL supports second generation APLs to the extent of allowing nested arrays as arguments to ASL functions where appropriate. Therefore in the detailed WAGS specifications there are frequently alternative forms of arguments. A user with a second generation interpreter may use nested arrays, but may also use the same simple array arguments as a user with first generation APL.

Higher Level Graphics

WAGS simply defines the lowest level of graphics operations in ASL. As mentioned before, along with the statistical functions which form the major part of ASL will be other functions to create appropriate graphical displays. These will operate entirely through calls to the WAGS functions, so that such functions will be portable to any system for which a WAGS implementation is available. To make the job of creating those functions easier, ASL will also offer a range of higher level graphics building blocks. For instance, there will be various functions useful for setting up and drawing axes. Axis drawing is a task that almost all higher graphics functions in ASL will need to do, and it is a complex and difficult task to do well. The higher graphics facilities will make axis drawing a simple and routine operation. Other building blocks will be available to draw standard graphical elements like histograms and box and whisker plots. However, those functions are quite separate from the low level WAGS system which is the subject of the present article.

Their design and specification is another story.

dyalog APL

for 386 PCs under MS-DOS

Dyalog APL for 386 PCs under MS-DOS is a complete implementation of Dyadic's popular APL for Unix systems, AND fully exploits the power of a DOS 386-PC.

JUST LOOK AT THE FEATURES . . .

- *An exciting new Screen Manager in which the contents of the screen are 'shared' with a nested system variable □SM (Screen Map). Facilities include:*
 - Pop-up windows by localisation of □SM
 - Scrolling (vertical, horizontal, linked)
 - GENUINE numeric fields
 - Date fields
 - NESTED forms
- Fully integrated window-based Session Manager, Editor and Debugging Environment.
- Compatible with the Draft ISO standard for APL
- Huge Workspaces (16Mb and MORE!)
- Virtual Memory Support (using on-chip demand paging)
- Full 32-bit processing speed
- *APL2 features including:*
 - Nested Arrays
 - Defined Operators
 - Selective Specification
- *Commercial features including:*
 - FMT
 - Component Files
 - Error Handling
 - "AP124-like" Screen Manager
- External Variables
- Interface to GSS*CGI for fast device-independent graphics
- C Interface

. . . THE BENEFITS SPEAK FOR THEMSELVES

Requirements: 386/486-based PC or PS/2, 640K DOS Memory plus 2Mb RAM, EGA or VGA (colour STRONGLY recommended), 80387 coprocessor STRONGLY recommended, DOS 3.3 or later.

For further information contact:

Dyadic Systems Limited

Riverside View, Basing Road, Old Basing, Basingstoke,
Hants, RG24 0AL Tel: 0256 811125 Fax: 0256 811130

APL Product Guide

Compiled by Alison Chatterton

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We do depend on the alacrity of suppliers to keep us informed about their products so that we can update the Guide for each issue of VECTOR. Any suppliers who are not included in the Guide should contact me to get their free entry. We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage.

The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages. Where no UK distributor has yet been appointed, the vendor should indicate whether this is imminent or whether approaches for representation by existing companies are welcomed.

For convenience to readers, the product list has been divided into the following groups:

- * Complete APL Systems (Hardware & Software)
- * APL Interpreters
- * APL Visual Display Units
- * APL character set printers
- * APL-based packages
- * APL Consultancy
- * APL Training Courses
- * Other services
- * Overseas Associations
- * Vendor addresses

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

Note: 'poa' indicates 'price on application'.

All contributions to the APL Product Guide should be sent to Alison Chatterton, at the address on the inside back cover.

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Active Workspace Ltd	AWL486	4,450	486 based 25MHz PC, 140MB Disk, 4MB RAM, VGA Colour. (inc. 1 year on site maint.)
	AWL 386	3,095+	386 based 25 & 33MHz PC, 140MB Disk, 4MB RAM, VGA Colour. (inc. 1 year on site maint.)
APL People	IBM PCs & compatibles	poa	includes PC, mono/colour monitor, APL interpreter, operating system software, plus optional printers, graphics boards, additional memory etc.
Dyadic	IBM RS/6000 MD320	11,736	APL POWERstation (Greyscale) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 19" 1280x1024 Greyscale Graph Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	13,817	APL POWERstation (Colour) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	22,656	Advanced APL POWERstation 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD520	37,114	APL POWERsystem (8-users) 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (2-8 user licence)
	IBM RS/6000 MD530	72,054	APL POWERsystem (16-users) 34.5 MIPS, 10.9 Mflops RISC Processor 32Mb RAM, 1.34Gb Disk, 2.3Gb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (8+ user licence)
	IBM RS/6000 MD540	122,642	APL POWERsystem (32-users) 41 MIPS, 13 Mflops RISC Processor 64Mb RAM, 1.7Gb Disk, 2.3Gb Tape CD-ROM Drive, 32 Ports AIX, Dyalog APL (8+ user licence)
MicroAPL	Aurora	20,000+	Multi-user APL computer using 68020 CPU. Std. configuration 2Mb RAM, 16 RS232 ports, 68 Mb hard disc, 720K diskette
	Spectrum	7,000+	Expandable multi-user APL computer using Motorola 68000. Std. configuration 1 Mb RAM, 12/36 Mb disc, 12 ports.
M.T.I.C.	386sx AT	2600+	Entry level 32-bit compact AT with DIALOG APL, 2MB ram, VGA, 40MB hard disk, 1.44MB floppy disk, 102-key keyboard, mouse, MS-DOS 4.01, WORKS and WINDOWS-386. Many expansion options available.

APL TIMESHARING SERVICES

COMPANY	PRODUCT	PRICES(£)	DETAILS
REUTER:FILE	SHARP APL	poa	International Network application systems and public databases.
Uniware	APL*PLUS	call	STSC's mainframe service

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Active Workspace Ltd	DIALOG APL DOS 386		Dyadics PC 386 APL Interpreter
APL Software	APL*Plus/PC Release 9	535	STSC's APL for IBM PC, PC/AT and PS2. Upgrades from earlier Releases also available.
	Run-time	poa	Closed version of APL*Plus/PC which prevents user exposure to APL.

	APL*Plus II	1600	Incorporates mainframe features & performance in a version of APL for the PC
	Run-time	poa	
	Dyalog APL	1000-10,000	2nd generation APL for Unix systems
	APL2/PC	376	IBM's APL 2 for the PC.
Cocking/Drury	APL*PLUS PC Rel 10	477	STSC's full featured APL for IBM's and compatibles - Version 10 includes the quad-NA facility to interface to non-APL software, support for MS Windows and mouse devices. The User-command processor has been built in to the interpreter. Upgrades to version 10 are available from Version 9 and earlier releases.
	APL*PLUS PC Run-Time	poa	Closed version of the interpreter for developers, prevents user exposure to APL.
	APL*PLUS II System	1600	High powered APL interpreter for the 80386 chip. Price includes one years maintenance and free upgrades - volume discounts VERSION 3 NOW AVAILABLE.
	APL*PLUS II Developer System	poa	All the features of the APL*PLUS II System plus unlimited free run-times to enable developers to distribute their applications.
	APL*PLUS UNIX	poa	STSC's 2nd generation APL for all major Unix computers and workstations. Version 4 for existing platforms and for the IBM RS/6000 available from January.
	APL*PLUS VMS	poa	2nd generation APL for DEC VAX computers running under VMS.
	APL*PLUS Mainframe	poa	Enhances VS APL with many high performance, high productivity features. For VM/CMS and MVS/TSO offers simple upgrade from VS APL.
Dyalic	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and dSM Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL for Unix Systems	995-12,000	Second generation APL for Unix systems. Available for Altos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle interface available for IBM, Sun and Xenix versions.
I-APL Ltd	I-APL/PC or RML Nimbus	4.50	ISO conforming interpreter. Supplied only with manual. (see 'Other Products' for accompanying books)
	I-APL/BBC	4.50	As above
	I-APL/Archimedes	4.50	As above
IBM UK	IBM PC APL2	348	APL2 for the IBM PC. Program 5799-PGG/1, PRPQ number RJ-0411. From all IBM dealers.
MicroAPL	APL.68000 Level I	2000	First generation APL with numerous enhancements. Multi-user version (Unix, Mirage, MCS).
	APL.68000 Level II	2500	Second generation APL. Nested arrays, user defined operators, selective specification etc. Multi-user version (Unix, Mirage, MCS)
	APL.68000 Level I		
	Mac, ST, Amiga, QL	87	First generation APL. Single user, full windowing interface, software floating point support.
	Mac, Amiga	260	First generation APL. Single user, full windowing interface, hardware floating point.
	APL.68000 Level II		
	ST	170	Second generation APL. Full windowing interface, software floating point support.
	Amiga	260	Second generation APL. Full windowing interface. Hardware and software floating point support.

	Mac	520	Second generation APL. Full windowing interface. Hardware and software floating point support.
	APL*PLUS Rel 9	450	
	APL*PLUS II V 3.0	1395	
REUTER:FILE	SHARP APL	poa	for IBM mainframes
Uniware	APL*PLUS/PC	495	STSC's full feature APL for IBM PC/XT/AT, Compaq, Olivetti.
	Run-Time	call	Closed version of APL*PLUS/PC which prevents user exposure to APL.
	APL*PLUS/UNIX	call	STSC's full feature APL for UNIX based computers
	APL*PLUS II	call	STSC's full feature APL for 386 machines.

APL VISUAL DISPLAY UNITS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL People	IBM & compatibles	poa	IBM and 'budget' APL VDUs - monochrome/colour/graphics.
Dyadic	IBM 3151	599	Monochrome APL/ASCII vdu with APL keyboard. Supports downloaded Dyalog APL font.
	IBM 6154	1,228	Colour APL/ASCII vdu with APL keyboard. Supports downloaded Dyalog APL font.
General Software	Mellordata	poa	
Shandell	HDS3200/10 APL	965	15" screen, 8 page memory, windows. 80/132 columns, full overstrike. Multi-host multi-session support. ANSI X3.64, DEC VT100, VT220, Tektronix 4010/4014 1024 x 390 resolution.
	HDS3200/25 APL	1065	As above plus switchable 25 or 50 line screen. 75 Hz refresh. Resolution 1024 x 780 in graphics mode.
	HDS3200/35 APL	1265	As HDS3200/25 plus local pan & zoom.
	HDS3200/50 APL	1395	14" colour monitor. 75 Hz refresh. 8 or 16 colours from palette of 256. Display memory 96 lines of 80 or 132 columns. APL processing & keyboard with full overstrike. Windows, multi-host, multi-session support. ANSI X3.64, DEC VT220, VT100, VT52 emulation.

APL PRINTERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL People	Epson series	200	inexpensive dot-matrix and NLQ printers
	Quen-data & Qume etc	500	Daisy-wheel printers
Dyadic	Various	poa	Range of APL printers available.
MicroAPL	Datasouth DS180+	1,195	See Datatrade entry
	Philips GP480	2,137	Matrix printer with letter & draft quality and APL (480 cps).
	Qume Letterpro20	549	APL/ASCII Daisy-wheel printer

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
Active Workspace Ltd	Syndicate Manager	poa	Lloyd's managing agent's syndicate / company accounting system. Stamp & Personal accounts (Inc. Run offs)
APL-385	APL-385	50(PC), 125(mf)	Including ...
	FSM-385		Screen development
	DRAW-385		Screen design

	DB-385		Relational W.S.
	GEN-385		Miscellaneous Utilities
APL Software Ltd (mainframe)	RDS	poa	Relation Data Base System
	IPLS	poa	Project Management System
	REGGPAK	poa	Regression Analysis Package
(microcomputer)	POWERTOOLS	295	Assembler written replacement function for commonly used CPU-consuming APL functions. Includes a Forms Processor.
	REGGPAK	poa	Regression Analysis Package
	RDS	990	Relational Database System
APL IMPETUS	Impetus	poa	Hierarchical Planning System
Cocking/Drury (for VSAPL)	E'MENTS & SHAREFILE	poa	Component files, quad- functions & nested arrays for VSAPL under VM/CMS & MVS/TSO
	COMPILER	poa	The First APL compiler!
	FILEPRINT	poa	Print APL component files
	FILECONVERT	poa	Converts non-APL files to APL
	FILEMANAGER	poa	Extends APL primitives to database management
	TOOLS + UTILITIES	poa	APL Software development tools
	DATAPORT	poa	Information Centre spreadsheet incorporating data exchange between APL, FOCUS, IFPS, SAS, APL/DI, ADRSII, Lotus123, Visicalc, Multiplan & DIF
(for APL2)	SHAREFILE/AP	poa	STSC's shared access component file system for APL2. Comparable to all APL*PLUS file systems: multi-user storage of APL2 arrays with efficient disk usage
	FMT	poa	Full featured FMT for APL2
	WSDOC	poa	Workspace documentation utilities
	FILEMANAGER	poa	Extends APL primitives to database management
(for PC's)	APL*PLUS PC Tools	275	Utilities including: RAM disk, full screen data entry, menu input, report generation, exception handling and games.
	IRMA Module	90	327x IRMA support.
	FIN & STAT. LIBRARY	250	Financial & Statistical routines
	SPREADSHEET MGR	150	APL-based spreadsheet for APL*PLUS/PC. Cell arithmetic; transfers to ASCII & Lotus
H.M.W.	4XTRA	poa	Front-end Foreign Exchange dealing / pos keeping
	Arbitrage	poa	Arbitrage modelling
	Basket	poa	Basket currency modelling
	Menu-Bar	poa	pull-down menu for APL*PLUS/PC
HRH Systems	APL Utilities	poa	PC Utilities including: APLMAC (windows); Unlock (unlocks functions in .AWS); DTEX (text and spreadsheet imp/exp). Mostly available in English or French.
Interprocess	IEDIT	1900-3200	Full screen APL2 editor with immediate APL execution, and a full-screen debugger
(mainframe)	AFM	8200-9800	High performance component and keyed file system (VS APL and APL2)
	Format	1650	A QuadFMT data formatter for VS APL and APL2
	FSM124	1650	AP124 programming for APL applications without GDDM (APL2)
	PowerCode	1300	External functions for APL2
	CALL/AP	3000	for calling non-APL programs (VS APL and APL2)

	UCF	1800	Inter-user data transfer for VM users via IUVC
(PC)	AFM	115	Single user component and keyed files for APL2/PC
Mercia	STATGRAPHICS 4	684	Integrated statistics/graphics system for the PC. Now with macros. Bulk and educational discounts available.
	Upgrade 3 to 4	215	
	Upgrade pre 3 to 4	375	
	MICROSPAN	250	Comprehensive APL tutor
	LOGOL	poa	Logistics management system for PC and 386. Sales Forecasting, Inventory Control, Master Scheduling, Distribution Requirements, Planning etc
	TWIGS		A modular library of tools to teach and explore state-of-the-art materials management concepts.
	T	299	Time series forecasting
	W	99	Warehouse replenishment
	I	199	Inventory Management
	G	99	Grouping requirements into EOQ's
	S	99	Scheduling production/purchasing
		599	All 5 modules above
		4999	All 5 modules site licence
MicroAPL	MicroTASK	250	Product development aids
	MicroFILE	250	File utilities and database
	MicroPLOT	250	Graphics for HP plotters etc
	MicroLINK	250	General device communications
	MicroEDIT	250	Full screen APL editor
	MicroFORM	250	Full screen forms design
	MicroSPAN	250	Comprehensive APL tutor
	MicroGRID	poa	Ethernet & other networking
	APLCALC	400	APL spreadsheet system
	MicroPLOT/PC	250	For APL*PLUS/PC product
	MicroSPAN/PC	250	APL self instruction for APL*PLUS/PC
	STATGRAPHICS Rel 4	590	
REUTER:FILE	GLOBAL LIMITS	poa	Exposure management for banks
	IPSA/CONNECT	poa	Mainframe to micro link
	MAILBOX	poa	Electronic Mail
	MAILBOX/PC V.2	75	Full screen front end to IPSA mailbox
	upgrade to V.2	15	
	NEWSFLASH	poa	Real time message exchange
	VIEWPOINT	poa	4GL - Info centre product
Un/ware (mainframe)	STSC's ENHANCEMENTS	poa	Quad-functions & nested arrays for IBM VSAPL under VM/CMS and MVS/TSO
	STSC's SHAREFILE	poa	component files for IBM VSAPL under VM/CMS and MVS/TSO amdnd for IBM APL2
	PROGRAMMER TOOLS & UTILITIES	POA	

	FILEPRINT	poa	
	FILESORT	poa	
	FILECONVERT	poa	
	FILEMANAGER(EMMA)	poa	STSC's database package
	EXECUCALC	poa	Mainframe spreadsheet compatible with VISICALC and part of LOTUS 1-2-3 under VSAPL(VM or TSO)
(microcomputer)	STATGRAPHICS	poa	Statistics and graphics for PCs
	STATGRAPHICS UNISTAT	poa	An add-on module to STATGRAPHICS: Data analysis software.
	APL*PLUS TOOLS		
	-VOL1	poa	Incl. 327 AE IRMA support, RAM disk, full screen data entry, menu input, report generation, games
	-VOL2	poa	Incl. File documentor, screen editor, exception handling.
	SPREADSHEET MNGR		poa APL spreadsheet with built-in ASCII, LOTUS and SYMPHONY interfaces.
	APL*PLUS/PC FIN & STAT.LIBRARY	poa	Collection of financial and statistical utilities.
	POCKET APL	poa	Smaller version of APL*PLUS/PC.
	UNIASM	poa	Collection of assembler routines for APL*PLUS/PC users.
	UNITAB	poa	APL*PLUS/PC spreadsheet-like data entry and validation system.
	The APL DEBUGGER	poa	First released APL*PLUS/PC debugger.
	APL2C	poa	Interface between APL*PLUS/PC and DATALIGHT C language
Warwick University	BATS	250	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free	Training program for the above.

APL CONSULTANCY

(prices quoted are per day unless otherwise marked)

COMPANY	PRODUCT	PRICES(£)	DETAILS
Active Workspace Ltd	Consult'	poa	PC Based APL system design, programming and implementation.
Adfee	Consultancy	poa	Development, maintenance, conversion, migration, documentation,,of APL products in all APL environments
APL People	Consultancy	poa	Consultants available at all levels, with experience in: VS APL, APL*PLUS, APL2, Sharp APL, Dyalog APL, APL6900, C/Unix, TSO/MVS, VM/CMS, graphics, Operational Research etc. Expertise in APL system design, project management, prototyping, financial applications, decision support systems, MIS, links to non-APL systems, documentation, etc.
Buckland Management Systems	Consultancy	poa	Business and Technical systems in commerce and industry - designing, programming and implementing applications.
Camacho	Consultancy	poa	Specialising in programming & manual writing.
Chapman	Consultancy	150-300	24-hour programmer: APL, C, assembler, graphics; PC, mini, mainframe, network.
Cocking/Drury	Consultancy	175-275	Junior consultant
		275-350	Consultant

		300-450	Senior consultant
		400-600	Principal Consultant
		450-750	Managing consultant
Peter Cyriax	Consultancy	100-150	Junior Consultant
		120-200	Consultant
		160-300	Senior Consultant
Delphi	Consultancy	poa	APL system development on mainframes and micros.
Dyadic	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
E & S	Consultancy	poa	System prototyping: all types of information system, engineering software, graphics and decision support systems APL*PLUS/PC, APL2, Dyalog APL
General Software	Consultancy	from 120	
H.M.W.	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Ian A. Clark	Consultancy	poa	Computer-based Information Systems implementation where acceptance is critical. APL on PC and Macintosh. Human Factors of HCI; novice ease of use; online assistance; training courses; distance-learning materials.
Intelligent Programs Ltd	Consultancy	175-350	Systems development, enhancements, support.
	Documentation	150-250	Preparation of new manuals, rewriting of existing materials.
	Training	150-250	Training for APL experts through to non-technical system users.
Mercia	Consultancy	poa	APL*PLUS & VSAPL consultancy.
MicroAPL	Consultancy	poa	Technical & applications consultancy.
M.T.I.C.	Consultancy	240-500	Business analysis and APL consultancy
Parallax Systems Inc	Consultancy	\$750	Introductory APL, APL for End-user & Advanced Topics In APL
QB On-Line	Consultancy	250	Specialising in Banking, Financial & Planning Systems.
REUTER:FILE	Consultancy	poa	Consultancy & support service world-wide.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
Rex Swain	Consultancy	poa	Independent consultant, 15 years experience. Custom software development & training, PC and/or mainframe.
Wickliffe Computer Ltd	Consultancy	poa	System design, consultancy, programming and documentation. Especially project management and decision support systems

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Employment	poa	Contractors and permanent employees
APL People	Employment Agency	poa	Permanent employees placed at all levels. Contractors supplied for short/long-term contracts, supervised or unsupervised. Executive Search service available.
HMW	Employment	poa	Contractors and permanent employees placed.
I-APL Ltd	An APL Tutorial	2.50	45pp by Aivord & Thomson
	An Encyclopaedia of APL (2nd Edn)	5.50	228pp by Halzer
	APL in Social Studies	2.50	36pp by Traberman
	I-APL Instruction Manual (2nd Edn)	2.50	55pp by Camacho & Ziemann
	APL Programs for the Mathematics Classroom		(Springer-Verlag)

14.50 185pp by Thomson

** Please add one pound packing charge per order **

REUTER:FILE

Productivity Tools poa Utilities for systems, operations, administration & analysts; auxiliary processors, comms software, international network.

Databases poa Financial, aviation, energy and socioeconomic.

ISI

Tangible Math \$19 An APL Approach to Math - Shareware, includes base Sharp APL

J \$24 Dictionary APL simplified and enhanced - Shareware (Mac,PC)

SharpAPL/PC \$74 Registered Shareware and Reference Manual

ISI APL \$99 Improved APL/PC - enhancements, performance, large workspaces

Renaissance
Data Systems

Booksellers

The widest range of APL books available anywhere. See Vector advertisements.

OVERSEAS ASSOCIATIONS (Sample entries)

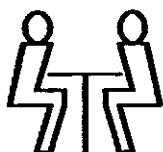
GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.	Visa/Mod
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$15	N/N
Dutch APL U.G.	Holland	-	Mini-congress, APL ShareWare Initiative		

VENDOR ADDRESSES

COMPANY	CONTACT	ADDRESS & TELEPHONE No.
Active Workspace Ltd	Ross D Ranson	Moulsham Mill Centre, Parkway, Chelmsford, Essex, CM2 7PX. Tel: (0245) 252414 ext.240
Adfee	Bernard Smoor	Dorpsstraat 50, 4128 BZ LEXMOND, Netherlands. Tel: 31.3474.2337, fax: 31.3474.2342
APL 385	Adrian Smith	Brook House, Gilling East, York. Tel: 04393-385
APLBUG	Jorge Mezel	117 East Creek Dr., Menlo Park, CA 94025, USA
APL Impetus Ltd	Cedric Heddle	Rusper, Sandy Lane, Ivy Hatch, SEVENOAKS, Kent TN15 0PD Tel: 0732-885126
APL People	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS. Tel: 0225-462602
APL Software	David Allis Jill Moss	The Old Malthouse, Clarence ST, BATH, BA1 5NS. Tel: 0225-462602
Buckland Management Systems	John Buckland	Westwood, 19 Grange Road, Camberley, Surrey, GU15 2DH Tel: 0276 684327
Anthony Camacho		2 Blenheim Road, St. Albans, Herts AL1 4NR. Tel: St. Albans (0727) 860130
Paul Chapman		19, Trevelyan Road, London, SW17 9LN Tel: 081-767 4254
Cocking & Drury Ltd.	Ronilly Cocking	180 Tottenham Court Road, LONDON, W1P 9LE Tel: 071436 9481 Fax: 071-436 0524
Datatrade Ltd.	Tony Checkseid	38 Billing Road, Northampton, NN1 5DQ. Tel: 0604-22289
Delphi Consultation	David Crossley	Church Green House, Stanford-in-the-Vale, Oxon SN7 8LQ. Tel: 03677-384
Dutch APL U.G.	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein. Tel: 03474-2337

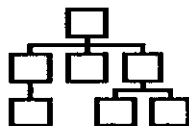
Dyadic Systems Ltd.	Peter Donnelly	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL. Tel: 0256 811125 Fax: 0256 811130
E & S Associates	Frank Evans	19 Homesdale Road, Orpington, Kent BR5 1JS. Tel: 0689-24741
General Software Ltd	M.E.Martin	22 Russell Road, Northholt, Middx, UB5 4QS. Tel:081-864-9537
H.M.W. Computing Ltd.	Stan Wilkinson	Hamilton House, 1 Temple Avenue, Victoria Embankment, LONDON EC4Y 0HA Tel: 071-353 4212 Telex: 926604 HAMHSEG Fax: 071-353 3325
HRH Systems	Dick Holt	Box 4486, Silver Spring, Maryland 20904
Ian A. Clark		9 Hill End, Frosterley, Bp. Auckland, Co. Durham DL13 2SX Tel: 038852-7190
I-APL Ltd	Anthony Camacho	2 Blenheim Road, St. Albans, AL1 4NR. Phone 0727-860130 for queries, order forms, bulk orders
	J C Business Services	56 The Crescent, Milton, Weston-super-Mare, Avon, BS22 8DU N.B. PAID ORDERS ONLY
IBM UK Ltd	Nat.Enq. Centre	414 Chiswick High Rd, London W4 5TF Tel: 081-747 0747
Interprocess Systems	Stella Chamberlain	9040 Roswell Road, Suite 690, Atlanta, Georgia 30350-1131 Tel: (404) 992-8400
Intelligent Programs Ltd	Mike Bucknall	Unit 7, Hermitage Court, 6-10 Sampson Street, London E1 9NA Tel:071- 481-4813
ISI	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9 Tel:(416) 925-6096
	Orders	3512 Cameron Mills Road, Alexandria, Virginia, USA 22305-1103 Tel:(703) 548-1799
Mercia Software Ltd.	Gareth Brentnall	Aston Science Park, Love Lane, Birmingham B7 4BJ. Tel: 021-359 5096
MicroAPL Ltd.	David Eastwood	South Bank Technopark, 90 London Road, LONDON SE1 6LN Tel: 071-922 8866
M.T.I.C.	Ray Cannon	7 Pine Wood, Sunbury-on-Thames, Middx. TW16 6SH Tel: Sunbury(09327) 80948
Peter Cyriax Systems	Peter Cyriax	213 Goldhurst Terrace, London NW6 3ER Tel: 071-624 7013 (Answerphone) 0660-377963 (Mobile)
Parallax Systems Inc.	Kevin Weaver	Avery Road, Box 319, Garrison, NY 10524, U.S.A. Tel: 914-424-4265
QB On-Line Systems	Philip Bulmer	5 Surrey House, Portsmouth Rd Camberley, Surrey, GU15 1LB. Tel: 0276-20789
Renaissance Data Systems	Ed Shaw	P.O. Box 20023, Park West Finance Station, New York, NY 10025-1510, U.S.A.. Tel: (212)864-3078
REUTER:FILE	Paul Jackson	7th Floor B Block, Coventry Point, Market Way, Coventry CV1 1EA Tel: 0203 256562
The Rochester Group	Robert Pullman	50 S.Union St., Rochester NY 14607, U.S.A. Tel:716-454-4360 or 716- 454-4641
Shandell Systems Ltd.	Maurice Shanahan	Chiltern House, High Street, Chalfont St. Giles, Bucks., HP8 4QH. Tel: 02407-2027. Fax:02407-3118
Sugar Mill Software Corp.	Lawrence H. Nitz	1180 Kika Place, Kailua, Hawaii 96734 Tel: (808) 261-7535
Rex Swain		8 South Street, Washington, CT 06793, U.S.A. Tel:203-868-0131 or 212- 242-5816
Uniware	Eric Lescasse	15 Rue Erianger, 75016 Paris, France. Tel:(1) 45-27-20-61. Fax:(1)45-27- 20-61. Telex: 648348F UNIWARE
Wickliffe Computer Ltd	Nick Telfer	76 Victoria Rd., Whitehaven, Cumbria, CA28 6JD. Tel:0946-892588
Warwick Univ.	Prof.Jeff Harrison	Dept of Statistics, University of Warwick, Coventry, CV4 7AL Tel:0203-523369

A single source for APL



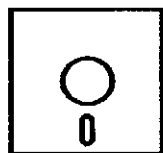
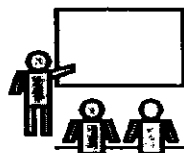
Advice

Design



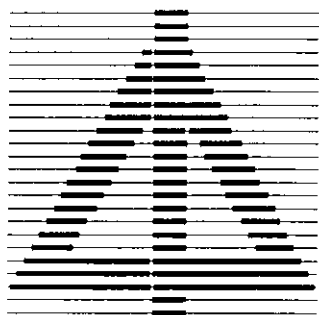
**Application
Development**

Education



APL software

Support



COCKING & DRURY LTD.

THE APL PROFESSIONALS

180 Tottenham Court Road

London W1P 9LE

Tel: 01 - 436 9481

(From May 1990, our STD code will be 071)

Windows 3: the Implications for APL

by Adrian Smith

Background

These notes are a brief attempt to explain why I think Windows 3 matters a lot to APL. I will start by outlining some of the things you can do **now** with Windows and APL*PLUS/PC or APL2/PC; the rest of this note is a look a little further ahead to where I would like to get to in (say) 18 months' or 2 years' time. The assumption behind all that follows is that you have at least a 16 MHz 386SX with about 4Mbyte of memory; Windows will also run in 'standard' and 'real' mode if you insist, but both of these effectively cripple it, and should be disregarded unless you are absolutely desperate!

What you can do with Windows

The most obvious thing is task-switching. I never added up all the time I lost whenever the phone rang:)SAVE and)OFF followed by a quick dive into the mainframe or Word or a spreadsheet, followed by APL and)LOAD to get back to where I was. To leave my APL*PLUS/PC session sitting on the screen, kick out into something else, and recover it **exactly** as I left it is a huge time saver.

There seems to be no practical limit on the number of DOS applications you can fire off alongside each other; I have often had 6 sessions running and Windows has never fallen below 75% free. Switching is literally instant, you use Alt+Tab to toggle round the sessions, and your screen flicks back the instant you take your thumb off the Alt key. There is only a minimal cost in memory; I lose 17K off my APL workspace as a consequence of running under Windows, and the e=8192 option works fine.

The next thing you might try is setting one or more sessions to run in the background. This is especially handy when that session is doing something slow and wasteful of processor resources, like copying a big file across a slow network. For example to upgrade our MTP system to a remote site requires about 8 minutes over a 2400baud line; I can run this in background at a low priority and get on with something useful in the meantime. Not recommended with things like GoScript which really hammer the CPU! You either slow your printing to a crawl, or make APL so jerky you can't use the editor ... set the print running in foreground and go for lunch!!

You can get more ambitious, and try cut and paste between sessions! For example I can write a line of code in APL, copy it into the Windows clipboard, and paste it straight into Word. This saves a deal of wondering whether rho is really alt-251 (ρ ... so it was!) and should speed up Vector production quite a bit. The only snag is that you can only use the clipboard when the application is running windowed, and this switches it to a graphics character set, hence losing all the APL symbols (at least in versions prior to 10). However you can toggle between windowed and full-screen with Alt+Enter, so this is quite bearable in practice.

Amazingly enough, cut and paste works fine in graphics mode too. For example I have an APL function to draw the Rowntree logo (any size, any colour etc.); I copied the logo from APL into Windows Paintbrush, and hence was able to use it to tile my desktop, as well as being able to scribble graffiti all over it with the mouse!

What you can't do with Windows

You can't run APL2/PC 32bit, APL*PLUS II or Dyalog APL. APL*PLUS II and Dyalog use the same memory management as QEMM, which will only co-operate with Windows if you have QEMM version 5.1. However this still limits you to the Windows 'standard mode', in other words applications get swopped out to disk, instead of being stacked away in extended memory. One of the great joys of Windows is the instant switching, so I personally still tend to close Windows down and run Dyalog APL from the DOS prompt.

Where APL needs to be Right Now

Obviously, APL has to co-operate with the Windows memory management, at least to the extent that it will run under Windows as a standard DOS application. Windows is quite happy to allocate extended memory as old-style LIMS/EMS memory, which is why APL*PLUS/PC still gets to use e=8192 to load 64K of the interpreter high. However if you want true, flat, 32-bit addressing, you need to use a new interface called the Dos Protected Mode Interface (DPMI for short) ... see the recent PC edition of Byte (from page 113) for details.

At this point, I go way outside my technical knowledge; all I can say is that it would appear that all the fancy Protected-mode support given by interface software like Phar Lap now comes free with the system, in fact it will probably be in DOS 5.0 even without Windows.

This leaves vendors like Dyalog and STSC in the happy position of being able to wave goodbye to Phar Lap, and hence to the hefty royalties they are paying on every copy of the interpreter they sell. The implications would seem to be:

- cheaper development systems, and very much cheaper runtime.
- more straightforward development of APs, ideally in common or garden C, rather than the super-whizzo Meta-ware Hi-C that Dyalog currently demands.
- you should get as much memory on your system as you can afford! If you want to run 32-bit APL alongside Aldus Pagemaker and MS Word, you are looking at 12 Mbyte minimum.

That will do for starters ... there is one more goody that deserves detailed explanation, and offers APL the chance of a giant step back to the future ...

Dynamic Data Exchange (DDE)

How many of you remember `□SV0` and `□SVC`? They actually gave their name to APLSV ... and I don't mean the perversion of a brilliant design concept into a lousy mechanism for reading and writing files!! I mean genuine user-user shared variables, which are a superb mechanism to manage concurrent processing, and deal with all the problems that arise when you need to organise co-operating tasks.

I last tangled with `□SV0` under VSPC, when we used it to set up a simple teleconferencing system between our planners. Essentially, it allowed two or more logged-on users (in York and Edinburgh) to update the same data, and see what each other was doing. With DDE under Windows, you have a defined interface whereby any Windows application can share variables with any other, using the Clipboard to pass messages, and shared memory to exchange data.

In other words, you could imagine a situation where you were running an Excel spreadsheet alongside an APL session, with the values in some range of cells shared with an APL array. As you modify the array in APL, the spreadsheet reflects the changes instantly (possibly on a chart); alternatively when you update the spreadsheet you find that the values in the array automatically stay in step. This sort of thing makes particular sense when one of the partners is a background task, such as an SQL server. Rather than setting up an APL-specific database link through SQL*NET, you just use a standard windows task (probably also used by Excel) via DDE. It all looks so much simpler!

Where I would like to be in 2 years' time

I want an APL system that is a real pukka Windows application!! That means a lot of work for somebody, but the benefits must surely be enormous for APL in general, and in particular for the first vendor to get there! Here are a few reasons why:

- if you know one Windows application, you know them all! No more session managers or editors to learn ... just use the Notepad to edit functions and the Clipboard to cut and paste! At last I will be able to throw away all my code for pop-ups and pull-downs and dialog boxes! How much simpler it would be if I let Windows do all this for me.
- I want to write systems which look for all the world like other Windows applications. No-one need know I wrote them in APL in 1/10th of the time it took some poor hack to write the same thing in C. If anyone asks, I just say that I used this fancy set of C macros, like `p` and `o`. = ... it may help.
- true integration of text and graphics. I want to be able to edit the data in one window, and see the graph change in another. Alternatively, I want to drag the graph around with the mouse, and see the numbers change to match. I want to be able to put a map of Europe on the screen, and see the production plan at Halifax by pointing at it ... and so on.
- DDE. This should go without saying. I want to share data between background server tasks, APL, and any other Windows applications I happen to be running, like Pagemaker or MS Word.
- no more hanging on the coat-tails of DOS-extendors like Phar Lap. Assuming Windows really has got the memory management right (and Microsoft aren't stupid) we should be clear to run up to 15Mbyte and beyond. We may at long last shake the notion of runtime licences out of the system.
- a whole lot of intangible things, like the far richer visual environment that Windows gives me. Some of the games are quite good too ... I suspect that proficiency at Reversi has taken over from proficiency at Billiards as a sign of a mis-spent youth.

I am sure St Paul felt the same on the road to Damascus ... MicroAPL and their Mac users must be laughing like drains at all this; they have been shouting it from the rooftops for years. I have nothing to say in my defence except that I was blind and now I can see! Please don't take my mouse away!!

First Catch your Function: A Newcomer's Thoughts on the STSC Mainframe Compiler

by Ray Cannon

Let me state from the outset, I am, as the title suggests, a newcomer to the STSC APL compiler. I have compiled about 6 functions so I'm no expert on the subject. However, only about half a dozen people in the UK have used the compiler so, "In the land of the blind, the one-eyed man is king".

The compiler turns APL functions into machine-code functions which one uses like a locked function within the workspace. The compiler does NOT compile whole workspaces. In theory, you compile the slower functions in the workspace. In practice, it's not that easy. For a start, you have to add declarations. Then, the compiler only handles old style (non-nested) APL. Lastly, if it is too complex, the compiled function itself "calls the interpreter".

How to Compile a Function.

"First, catch your function."

Since compiling 10% of the code in the average workspace can often save 80% of the CPU time, there is little point in compiling the whole workspace. So the first step is to discover which function or functions it is worth while compiling.

The best tool for this job is the STSC $\square MF$ (Monitor Function); this allows one to locate where all the CPU is being used at run time. (For more information on $\square MF$ and efficiency in general, may I refer you to Dave Ziemann's excellent article "Efficiency in APL". Having applied the points learnt from that article, you may find you don't need the compiler at all!)

OK, you have written efficient code, and $\square MF$ still says that 80% of the CPU is being taken up by 1 function AND the system still runs too slow. Now you know which function to work on. The next stage is to check where the CPU is being used within that function. If only a few lines use most of the CPU, then it may well be worth putting those lines into a sub-function and just compiling that rather than the whole thing.

"Next, add the seasoning."

Before a function can be compiled, some declarations may need to be added.

Declarations are required for:

User-defined functions

Global variables

Shared variables

To aid the compiler, other declarations are recommended for:

Arguments to the function

Results returned by subroutines

Variables assigned atypical values (integer or reals assigned Boolean 0 or 1)

Local variables being used internally (under some conditions).

Getting the right level of declarations is important, and studying the supplied "Compiler reference manual" will help here. Too few declarations will not allow the compiler to produce the best code. Too many or too detailed will burden the compiler and may lead to run time errors due to errors in the declarations.

"Then, place in a pre-heated compiler for 2-3 minutes."

Cooking your function is easy, simply copy it into the compiler workspace and run the compiler function with the name of your function as its right hand argument. The compiler produces lots of screen output, so it's worth having a session manager turned on (this is also written to a special file from where it can be retrieved) as you may need to examine it rather closely.

The resulting output (assuming the compilation worked) is the "object code". This is stored in a character vector and when fixed via `□FX` or `□DEF` produces the compiled function in the active workspace. (Note that `□CR` or `□VR` on the compiled function return the source code NOT the object code.)

"The test of the pudding is in the eating"

Having fixed the compiled function in the workspace, you can now rerun the original test you did with `□MF` to see how successful you were. Speed-ups of between 2 and 200 times are achievable, but some functions even run slower!

If you are happy with the results "bon appetit", if not refer back to the compiler listing to find out why the compiler is unable to speed up the execution. The reference manual gives a lot of help in this area, telling you how to rewrite your

function in a way the compiler can take an advantage of. (It might be useful if you can still remember how to write Fortran!)

"How fast is fast food?"

As an example, may I present some relative timings produced from a test on a real problem. It involved a looping numerical (non-integer) calculations on arrays. The original un-compiled code ran in 8.54 CPU time units. On the same problem the modified and compiled code ran in 1.51 CPU time units. The overall effect was to reduce the total CPU used in the application by 50%.

"Here is one STSC prepared earlier"

STSC are now providing many "fast functions" with their interpreter, so you may not need the compiler at all!

"Health warning"

The compiler is unable to deal with:

Nested array features or syntax

Error trapping via `□ELX` or `□ALX`

Large functions. (Large being defined in terms of the number of "tokens" used, rather than number of lines of code.)

Pregnant women, young children or APL programmers of a nervous disposition.

Final Thoughts

My overall impression is that you end up rewriting the function in pseudo-APL that looks (and runs) like Fortran. There are times however, that it pays off.

References

Efficiency in APL. Dave Ziemann. Vector Vol.4 No.4. pp.111-144 (April 1988).

*APL*PLUS Compiler Reference Manual*. Jim Welgang. (June 1986).

APL*PLUS/PC Release 10

reviewed by Jonathan Barman

Introduction

STSC have come out with a new release of APL*PLUS/PC almost exactly one year after the previous release. As in previous reviews, I will concentrate on the new features rather than trying to cover the entire product.

The major new feature is the introduction of `□MA` to access programs written in C or FORTRAN, which solves a lot of the hassle that was necessary previously. The remaining enhancements are relatively minor, but I suspect that the better colour support and the mouse control will prove to be very popular.

Running non-APL routines

I wish this enhancement had come out earlier, it would have saved hours of work battling with assembler to provide those functions where APL is inherently slow. There are five components in the enhancement:

Interface Programs

When you want to run a non-APL program you have to install an Interface Program before going into APL. There is a special Interface Program for each different compiler and language combination. STSC have provided three, Microsoft C, Microsoft Fortran, and Turbo C. You can load as many Interface Programs as necessary to run the different flavours of non-APL programs. The source assembler is provided so that experienced programmers can write their own interface programs for other compilers and languages.

Stack Allocation

Compiling is as normal for the non-APL program, but at the linking stage it is necessary to include an STSC provided object deck with allocates room for the stack. The source assembler is provided so that one can change the size of the stack as required.

`□MLOAD`

`□MLOAD` reads .EXE and .MAP files and returns an integer vector of machine code which you save in your workspace. The vector is in a special internal format which STSC say is deliberately not documented.

□NA

□NA creates a special function in the workspace which will run the non-APL program. You have to specify which Interface Program to use, the name of the variable holding the module created by □MLOAD, the name of the entry point of the program, and the parameters together with the data types required by the program.

)DEBUG

If APL has been started under the control of the DOS DEBUG program, then)DEBUG takes a □NA'ed function as an argument and will hand over control to DEBUG when the function is run.

To try out the new feature I created a little program in Turbo C to manipulate dates. My initial effort resulted in a spectacular crash in APL with garbage written all over the screen. Tracking down the error was fairly easy. First I made sure that the C program was behaving properly by adding a 'main' calling routine and using the Turbo C debugging facilities. When this did not cure the problem, I looked more carefully at the sample C programs supplied and found that I had omitted to declare the C functions as 'far'. When this was corrected all worked beautifully. I never got)DEBUG to work but I must confess that I did not try very hard as its usefulness is limited when Turbo C has such magnificent debugging facilities.

The automatic data type conversion facilities are very useful. Some years ago I wasted hours trying all sorts of tricks to force the APL interpreter to pass floating point numbers to an assembler routine. As well as being able to specify F8 for long floating point numbers, one can also specify I4 for long integer, I2 for the normal integers used by APL, and C1 for character. If coercion to the specified data type is not possible you get a *DOMAIN ERROR*.

The problem of multiple arguments and results has been handled by extending APL for those functions which have been created by □NA. Multiple arguments are passed to a □NA'ed function as a right argument in the same format as the right argument to □FMT, with a set of statements separated by semicolons and the whole enclosed in parentheses. Multiple results must be assigned to a strand of variable names. For example, a program which takes 4 arguments, two of which are modified and returned as results, would be called with *A B ← FUN (A;B;C;D)*.

The whole thing looks to be a really useful extension which enables one to integrate APL and non-APL programs easily and quickly.

Mouse Support

`□MOUSE` allows one to control the mouse, such as positioning it and specifying whether the clicks do anything. It is now possible to design mouse interaction with `□WIN` and `□WKEY`, with yet more special feature numbers such as one to move the cursor to the mouse position. I was unable to play with the facilities as I have not got a rodent, but the implementation looks comprehensive and reasonably easy to use.

File System

The file system has always been plagued by the expanding replace problem. When `□FREPLACE` is used with an object which is bigger than the original object, then the new object is placed at the end of the file and you have to use `□FDUP` to recover the space taken up by the original object. Rather than doing a proper job of reusing old space automatically, STSC have extended `□FAPPEND` and `□FREPLACE` so that extra space can be reserved for a component. There is also a new system function `□FCSIZE` which enables one to find out what space has been reserved for a component. `□FDUP` does not take any notice of the reserved space, and removes it all when the file is cleaned up. Old hands at the game have been reserving extra space by writing large dummy variables to file on the first `□FAPPEND`, and the 'enhancement' just makes the process a little more convenient. One day STSC might provide a proper file system to eliminate all this nonsense.

User Command Processor

The User Command Processor has been properly integrated so that the interpreter now recognises commands such as `]LIB` or `]MYFN` directly, rather than the original messing around with Function Key 5. The `□UCMD` system function replaces the previous `ΔUCMD` function. The file formats have changed and a conversion function is supplied to upgrade. I found the processor very slow in use as there is a lot of disk activity, and my disk is relatively slow, but it is sometimes worth waiting for the results of the more exciting distributed functions.

PostScript Support

A PostScript APL font is supplied, together with a special printer driver which does the normal Port 10 stuff. Not having a PostScript printer was tantalising, as I would have dearly liked to check this one out. If possible, a full review will appear in a later issue of Vector.

Other Enhancements and Changes

The colour support for the APL session should be popular, and I cannot think why it was not done before. The changes required must have been relatively simple. The colours can be specified on entry, or `POKE` can be used to set them during the session.

Calling the **Extended System** has been simplified as the session manager has been bundled up with its interpreter. There are now three favours of .EXE files, APLX for Extended, APL for standard, and APLNOG for no graphics.

Font management has been improved so that you no longer have to remember the whether you have an EGA or VGA, and you can use the extended depths of 28, 43 or 50 lines with APL characters. It no longer matters how many times the program is run, and the same program can be used to remove the APL characters when the APL session ends.

There is now an extra step in the familiar routine of copying all the relevant files from the supplied disks. Many files are distributed in a compressed format, so one now has to unpack them before use. The instructions are clear and the simple process only has to be done once. I liked having each packed file separate so that one can pick and choose which to unpack by looking at the directory. One often comes across software packed up into one huge file which has to be unpacked in total.

The Smith Benchmarks

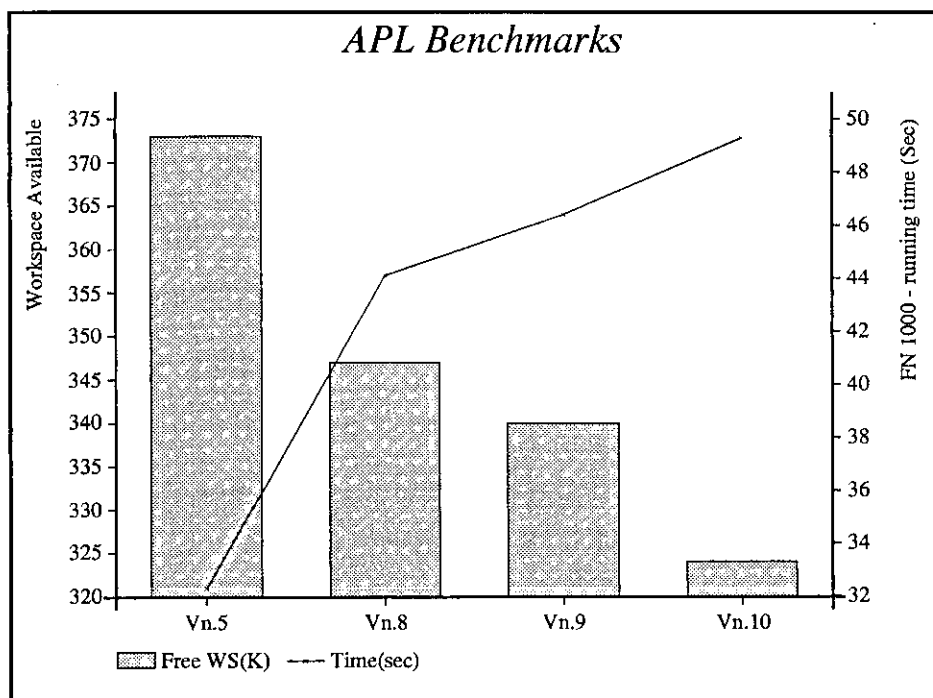
Yet again the benchmarks show up a slightly larger interpreter and slower performance. `%WA` is down 16K from Release 9, which inevitably increases running times as the garbage collect process has to be carried out more frequently. Nothing come free in this world, and the extra features are bound to take up more space for the code. Prospective purchasers will have to decide if the additional features are worth the degradation in performance.

I carried out the benchmarks on an IBM XT with a co-processor for floating point arithmetic, but only 576K of memory. This gives different absolute results from those shown by Adrian in Vector 6.4, but the differences between the releases seem to be consistent.

<i>APL Expression</i>	<i>Type</i>	<i>Vn.5</i>	<i>Vn.8</i>	<i>Vn.10</i>
$\lfloor \square WA \div 1024$	α <i>WA</i>	373K	347K	324K
$T+\square TS \diamond +/V+V \diamond DO \square TS-T$	α <i>INT</i>	380	380	380
$T+\square TS \diamond +/V[I] \diamond DO \square TS-T$	α <i>INT</i>	50	50	50
$T+\square TS \diamond V+. \times V \diamond DO \square TS-T$	α <i>RL</i>	380	440	490
$T+\square TS \diamond +/V \times 2 \diamond DO \square TS-T$	α <i>RL</i>	2080	2310	2310
$T+\square TS \diamond X++/M \diamond DO \square TS-T$	α <i>RL</i>	490	490	500
$T+\square TS \diamond V+. \times, M \diamond DO \square TS-T$	α <i>RL</i>	1200	1260	1260
$T+\square TS \diamond +/S='T' \diamond DO \square TS-T$	α <i>CH</i>	3790	3950	3950
$T+\square TS \diamond 'FAT' \epsilon S \diamond DO \square TS-T$	α <i>CH</i>	50	50	50
$T+\square TS \diamond X+S \epsilon 'FAT' \diamond DO \square TS-T$	α <i>CH</i>	2800	5660	5660
$T+\square TS \diamond X+B \vee B \wedge \sim B \diamond DO \square TS-T$	α <i>BL</i>	5000	5650	5650
$T+\square TS \diamond \wedge / B \diamond DO \square TS-T$	α <i>BL</i>	110	160	160
$T+\square TS \diamond X \vee \wedge B \diamond DO \square TS-T$	α <i>BL</i>	220	270	270
$T+\square TS \diamond FN \ 10 \diamond DO \square TS-T$	α <i>FN</i>	330	490	490
$T+\square TS \diamond FN \ 100 \diamond DO \square TS-T$	α <i>FN</i>	3240	4560	4830
$T+\square TS \diamond FN \ 1000 \diamond DO \square TS-T$	α <i>FN</i>	32.3s	46.5s	49.3s
$T+\square TS \diamond X+FF\wedge. = 'EJD..' \diamond DO \square TS-T$	α	50	50	50
$T+\square TS \diamond X+FF\wedge. = \#20 \ 5\rho 'EJD..' \diamond DO \square TS-T$	α	35.1s	39.5s	39.5s

The standard system was used (APL.EXE), and all the timings shown are the minimum values that were returned. Higher times were assumed to be due to the garbage collect job that has to be carried out when there is no free memory for a new object.

The two most interesting lines are those timing the expressions *FN 1000* and *X+FF $\wedge$. = #20 5 ρ 'EJD..'*. The function *FN* (see Vector 6.4 for a listing) may cause the garbage collect process to occur more frequently, but I suspect that the interpreter is a mite slower as it has to check for $\square NA$ 'ed functions with their special syntax. The big inner product has very little interpreter overhead as once everything is set up the majority of the time is taken up with doing all those thousands of equality tests, and there is no change in that process.



The benchmarks were not carried out on the Extended System, as this is intended for developers and would not normally be used for production systems. Speed is less important than good editing and debugging facilities.

Conclusion

The same story yet again, more features, bigger size, and slower - all products seem to get like this towards the end of their life. I liked the $\square/\Lambda$ and the mouse support features, and can see many uses for them. I am less sure of the utility of some of the other features in both this and previous releases. It would be nice to be able to pick and choose the features that are needed so that a smaller faster interpreter can be used for production systems, without having to buy the minimum quantity of Run Time systems.



APL PEOPLE

Serving the International APL Community

Consultancy advice and assistance with all aspects of APL systems development

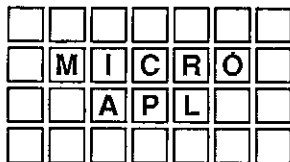
Recruitment unique placement service for companies and individuals

Software Competitive prices on all APL software tools and interpreters

Contact: Jill Moss, APL People Ltd.,
The Old Malthouse, Clarence Street,
Bath BA1 5NS

Tel: 0225 462602 during office hours
0225 333618 evenings and weekends

APL.68000 Level II



APL.68000 Level II is a full specification APL2 for the 68000 family of processors. It incorporates all the user-oriented enhancements of APL.68000 Level I as well as MicroAPL's acclaimed windowing interface. APL.68000 Level II is available for computers such as the NCR Tower, Sun, HP, Bull, Nixdorf, Wicat, Mac, Amiga and ST. Features include:

- Nested and mixed arrays
- User-defined operators
- Selective specification
- Upwards compatibility with APL.68000 Level I

For more details contact:
MicroAPL Ltd.,
South Bank Technopark,
90 London Road, London, SE1 6LN.
071-922-8866

RECENT MEETINGS

This section of VECTOR is intended to document the seminars given at recent meetings of the association; it is of particular value to members who live away from London. It also covers other selected events which may be of general interest to the APL community.

If you would like to speak at one of the regular British APL Association seminars, please ring the Activities Officer (address on inside back cover) who will respond enthusiastically to your offer.

APL and PostScript Graphics

by Adrian Smith (Rowntree Mackintosh Ltd)

Background

The roots of this talk go way back to Vector No.1 Vol.4, which includes my notes on "Getting the Best out of GDDM". One of the points I made then was of the benefits of using IBM's Presentation Graphics Facility (PGF) rather than the APL Graphpak workspace. PGF ran at least 5 times faster than Graphpak, and had the added virtue of requiring a mere 543 lines of code (of which about half were comments) instead of over 3600. It also led to pretty trivial APL code like:

```

▽ CHΔPIE MAT
[1] ⍝ Form data into multiple pie charts ...
[2] ⍝ one row makes each pie.

[3] MAT+(⌈2+ 1 1 ,⍐MAT)⍐MAT
[4] CTLs←796,(⍐MAT),,MAT
[5] 'FAILED TO DRAW PIE CHART' FSMΔERR CTLs
▽

```

PGF was remarkably good at giving nice defaults for colours and shadings, and its algorithm for picking 'good' scales and tick intervals is still the best I have found. This meant that in the vast majority of cases you could simply say "Hi PGF ... take that and gizza barchart" without a lot of preliminary setting up. Of course you needed to know that 796 was PGF-speak for "gizza piechart", but that was easy enough to bury in a cover function.

As things moved towards the PC, I kept thinking that someone somewhere must be working on this ... all I had to do was sit around until a decent library of C routines came on the market, and I would be able to call these from APL. Occasionally things popped up like "The Turbo Pascal Graphics Toolbox", but invariably these stopped far short of the PGF functionality I was after. Nothing in the APL world went much beyond the basic "draw a polyline" functionality of `⌈Gxx`, or `GSS*CGI`. OK, this is fine for drawing ducks with (as long as you have Dyalog APL!), but leaves all the hard work of designing an effective graphic to the application. "Bother, said Pooh."

PostScript Basics

Your starter for 10 ... name a computer language which:

- has no silly precedence rules
- has a matrix-inverse primitive
- allows any array as an array element
- allows assignment of both functions and variables, with no prior type declarations
- makes no distinction between integers and reals, and automatically converts between them as required.

Hands up those who said APL!! The more perceptive amongst you might have taken note of the title, and said "PostScript", but I feel I ought to discount that as cheating ... although you were quite correct. Obviously, there is no way I can teach you PostScript in half a page - all I want to do here is to leave you with the impression that PostScript is a lot like APL, and that the idea of coding in the "intersection" of the two languages is not as daft as it sounds.

Here is the PostScript to draw a square somewhere near the middle of a sheet of A4 paper (all measurements are in points, or $\frac{1}{72}$ "):

```
newpath                % clear the current path
200 400 moveto          % bottom left corner
300 400 lineto 300 500 lineto 200 500 lineto
closepath              % back to the start
2 setlinewidth          % use a 2-point line
stroke                 % draw round the path

showpage               % print it!!
```

PostScript is entirely stack-based, i.e. all functions are niladic, and pull their input from a stack. If they produce results (none of the above do) the results are pushed on to the stack. You need to think 'reverse Polish' to do arithmetic, but otherwise you can get along quite well by regarding the syntax as 'just like APL', except that it executes from left to right! All functions are of the form:

```
∇ ARG LINETO DUM
[1]  A Draw line from current point to ARG
[2]  .... etc
∇
```

The other things you need to know are the syntax for assignment, for function definition, and for representing an array. Assignment is coded like this:

```
/PPI 72 def      % move the value 72 to the literal PPI
/TEXT (Hello!) def % strings get () round them
```

```
PPP+72      A Move the value 72 to the name PPI
TEXT+'Hello!' A Strings get '' round them
```

The def operator takes a literal (the leading '/' flags literals) and a value off the stack, and stuffs the latter into the former. It also allows function assignment:

```
/M {moveto} def % for lazy typists
/inch {72 mul} def % allows us to work in inches

4 inch 6 inch M % .... etc.
```

Obviously, functions can call other functions (including themselves), and can refer to defined variables. Everything is global by default, although you can manipulate contexts to give the effect of localisation (see [1] and [2] for details). Arrays are very similar to APL:

```
/ARRAY [1 2 3 4] def % 4-element array
[23 34 45 56] 4 3 % 3-element nested array

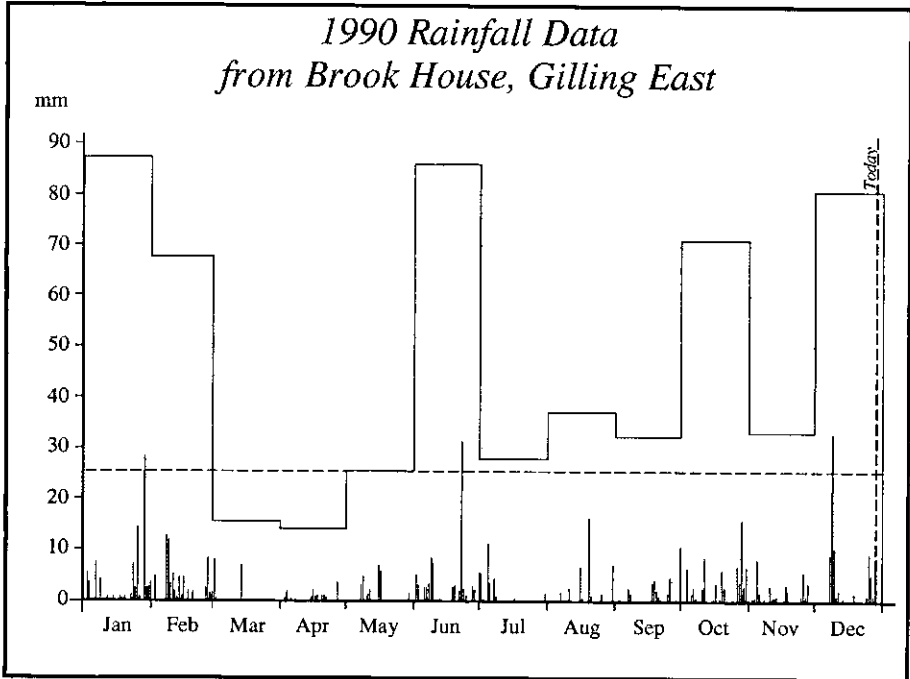
ARRAY+1 2 3 4      A 4-element vector
(23 34 45 56) 4 3 A 3-element nested vector
```

I think that should be enough to start with ... if you really want to know PostScript properly, you should get the Blue Book [1] and the Red Book [2] and a PostScript printer (or some software like GoScript which will put the results on the screen ... much cheaper!) and play with it.

Motivation

As Vector moved gradually up-market in its presentation, I began to feel the need of some 'publication-quality' graphics for things like benchmark timings. The charts in question were extremely simple, typically just line-graphs or multiple barcharts, but it was important to get the typesetting details right if the result was to look good on the page. It was pretty obvious that PostScript had just what I needed when it came to detailed control over type sizes and text rotation and placement, so I began to play with some basic routines for things like axis-drawing and labelling.

Roughly at the same time, various parts of my user-community began to agitate for the same graphics support on the PC as they had been used to on the mainframe with PGF ... clearly I could no longer afford to wait for someone else to implement PGF as an AP (or in STSC's case `BAR`CHART, `PIE`CHART and so on!!). The trigger was provided by a weather station which Richard was given for Christmas ... we were accumulating daily records of rainfall and temperature, and soon wanted to produce graphs like this:



As always, when you finally summon the energy to start hacking, things are much easier than you thought they would be. Essentially the exercise splits into two parts:

1. Write the APL bits for the things PostScript does, but APL lacks (like text handling, scaling and rotation).
2. Write the PostScript bits for the things APL does (like Polylines, Polybars - `GSHADE`, and polymarks).

You can then code in the comfortable region where the languages intersect, with appropriate supporting routines to cover the remainder. As a bonus, it turned out that the HP plotter language HPGL also maps nicely to this region, and it took about half a day to get my functions driving an HP7550! I suspect that the same is true of any vector graphics language (e.g. the new AP207 that comes with APL2/PC); certainly having skimmed the booklet on GSS*CGI that comes with Dyalog APL, I am quite sanguine about implementing all this lot on any APL you can think of!

Stage-1: The APL End

The examples which follow are all from APL*PLUS/PC, using $\square G$ -style graphics. I have not attempted complete coverage ... if any of this code would be any use to you, APL-385 will oblige for £10 to cover the cost of the disk and postage..

To begin at the beginning ... PostScript handles translation, scaling and rotation by holding a 'Current Transformation Matrix' in the classical form [3]:

```

1      0      0
0      1      0
tx     ty     1      ... translation

sx      0      0
0      sy     0
0      0      1      ... x/y scaling

cosθ    sinθ    0
-sinθ   cosθ    0
0      0      1      ... rotation counterclockwise by θ
```

The matrix starts as $CTM \leftarrow 3 \text{ } 3 \text{ } 4 + 1$, and successive transformations are applied by matrix multiplications, for example:

```

▽ X ROTATE DUM;D2R

[1]  * Catenate rotation to current transformation matrix

[2]  D2R←(○2)÷360
[3]  X← 3 3 + 2 2 ρ 1 1 -1 1 × 2 1 1 2 ○X×D2R ○ X[3;3]÷1
[4]  CTM←X+.×CTM
▽
```

When it comes to drawing lines, we follow the PostScript philosophy of building a path, and deciding later whether to draw round it (with stroke) or shade it in

(with fill). Both these clear the path automatically, but the nervous can use `newpath` to make sure they start with a clean slate:

```

▽ NEWPATH DUM
[1]  ⍝ Set up an empty path. Cols are [x,y,0/1]
[2]  PATH← 0 3 ρ 0
▽

▽ A LINETO DUM
[1]  CPT←A ⋄ PATH←PATH,(3+A,1)+.×CTM
▽

▽ X MOVETO DUM;NP;LEN
[1]  ⍝ Add / update pen move to current path.
[2]  CPT←X
[3]  NP←(3+X,1)+.×CTM ⋄ LEN←(ρPATH)[1] ⋄ →LEN+Add
[4]  →PATH[LEN;3]+Add
[5]  Rpl:PATH[LEN; 1 2]←2+NP ⋄ →0
[6]  Add:PATH←PATH,3+2+NP
▽

▽ STROKE DUM;SEG;LEN
[1]  ⍝ Draw line segments in PATH, and clear it.
[2]  Loop:LEN←(1+PATH[;3])⋄0
[3]  SEG←(LEN,2)+PATH
[4]  GSΔDRAW SEG
[5]  PATH←(LEN,0)+PATH ⋄ →(1+ρPATH)+Loop
▽

```

`GSΔDRAW` is a very thin cover on `GLINE`. Similar functions will do for the PostScript basics like displaying text:

```

▽ TXT SHOW DUM;XY
[1]  ⍝ Dummy version of () show
[2]  XY←2+(CPT,1)+.×CTM
[3]  (XY,FontSCL)GSΔCHAR TXT
[4]  (TXT STRINGWIDTH 0)RMOVETO 0
▽

```

... but when it comes to working out the space occupied by graphics text, things get nastily device-dependent, not to mention the fact that in APL we are limited to integer multiples of the basic font size:

```

▽ R←TXT STRINGWIDTH DUM;WID;GT
[1]  ♂ Find width of text string in current font.
[2]  ♂ Note that this varies for different adaptors!
[3]  WID← 5.16 6.88 6.34 5.42 ♂ Herc, CGA, EGA, VGA
[4]  GT←4 ♂ →(0=□NC 'ΔGTYPE')→Skip ♂ Assume VGA
[5]  GT←4\ 'HCEV' 11+ΔGTYPE
[6]  Skip←R+2+WID[GT]×FONTSCL×ρ, TXT
▽

▽ SIZE SCALEFONT DUM
[1]  ♂ Rough mimic of scalefont
[2]  FONTSCL←1\ [0.5+0.1×SIZE×CTM[1;1]
▽

```

So much for the basics ... of course APL is much more efficient when dealing with vectors, so we would really like to work with a polyline:

```

▽ VEC PL DUM;LEN
[1]  ♂ APL version of polyline, as used by CHΔLINE etc.
[2]  ♂ Adds line to path ... it will still need stroking!
[3]  ♂ Last elements give colour, style and thickness

[4]  VEC[~2+ρVEC]SETCOLOUR ♂ ♂ VEC[~1+ρVEC]SETLINESTYLE ♂
[5]  VEC[ρVEC]SETLINEWIDTH ♂ ♂ VEC+~3+VEC
[6]  LEN←\0.5×ρVEC ♂ VEC+(LEN,2)ρVEC
[7]  VEC+(VEC,1)+.×CTM
[8]  PATH←PATH,VEC[; 1 2],~LEN+1
▽

```

This is just like **lineto**, except that it expects a vector of x/y pairs, with some extra bits on the end giving the line attributes. These just get bunged into a rat's nest of globals (except for the line width, which is a dummy). Now let's have a look at the PostScript end of all this!

Stage-2: The PostScript End

Another respect in which PostScript is like APL is in its orthogonality. As long as we disregard □functions, APL has almost no redundancy in the primitives; you add new ones to suit your convenience. You might expect to see PostScript functions for right-aligned and vertical text, but (as with APL) you must make your own:

```

/cshow      % centre text round current point
{ dup stringwidth pop
  neg 2 div 0 rmoveto show
} def

```

```

/vshow      % vertical text ... for y-axis titles
{ gsave 90 rotate
  dup stringwidth pop
  neg 0 rmoveto show
  grestore
} def

```

You might also expect to see a whole string of parameters to specify the writing direction of text ... again there is no such nonsense! You simply save the Current Transformation Matrix with `gsave`, rotate the paper, write your text normally, and restore afterwards with `grestore`.

Now for something more interesting: the PostScript version of the APL function to tackle Polyline. I shall try to explain this one in detail, because it illustrates a lot of useful concepts. If you are still struggling over the logic of `cshow`, all should shortly become clear!!

```

/pL % Polyline: stack is array of [x,y] pairs; colour style width
{ setlinewidth setlinestyle setcolour
  /xy exch def
  /npts xy length 2 idiv 1 sub def % halve the shape
  xy aload pop                    % disclose it
  moveto                          % pen to 1st point
  npts { lineto } repeat          % loop for each pair
} def                             % will need stroking!

```

The top line is mostly a comment, describing what our function expects to find on the stack: this is always good practice when documenting PostScript code. We then peel off the last three elements (in backwards order remember) to set the line attributes. The functions `setlinewidth` etc. are all primitives.

The next line assigns the remaining element (the array of x/y pairs) to a variable `xy` (the `exch` primitive exchanges the top two things on the stack). In order to find out the number of line segments to draw, we need one less than half the shape (remembering the initial `moveto`):

```

NPTS+~1+[(pXY)÷2      n /npts xy length 2 idiv 1 sub def

```

We then disclose (or `aload`) the array; this dumps the array contents onto the stack, plus the array itself, which we discard with a `pop`! The stack now holds an even number of x/y pairs, so we pick up the first set with a `moveto`, and then hoover up the rest by looping `npts` times. At the end of this, we have the entire polyline defined in the current path, so we can either draw round it with `stroke`, shade it in with `fill`, or even make it the outline of a clipping region with `clip`.

```

/wedge      % shaded pie slice: stack is x,y,r,a,s,pattern,width
{ setlinewidth setpattern
  /beta exch def /alpha exch def /radius exch def
  /y exch def /x exch def
  x y moveto                                % start at sector root
  x y radius alpha beta arc                 % line to edge, and arc
  closepath                                % back to the middle
  gsave currentpatn grestore               % shade it
  [] 0 setdash stroke                      % and solid edge
} def

```

Here is another example ... this time PostScript is quite close to APL in the way we draw arcs, so the code is almost trivial. I'm sure I could get away without all those variables, but my brain starts to hurt as soon as I have to deal with anything not on the top of the stack, so I got lazy and just snaffled the lot at the start! Shading patterns are quite interesting:

```

/setpattern % set current graphics pattern: stack is int [1..12]
{ /case exch def
  case 0 eq { /currentpatn {} def } if % No fill
  case 1 eq { /currentpatn { .98 setgray fill } def } if % Light grey
  case 2 eq { /currentpatn { .9 setgray fill } def } if % Medium
  case 3 eq { /currentpatn { .8 setgray fill } def } if % Darker
  case 4 eq { /currentpatn { .5 setgray fill } def } if % Half tone

  case 5 eq { /currentpatn % diagonal shading
    { gsave 1 setgray fill grestore
      clip newpath 0.2 setlinewidth [] 0 setdash
      0 6 patscale mul 600 { -60 moveto -240 360 xlineto } for
      stroke } def } if
}
.... etc

```

What we are doing is dynamically defining the `currentpattern` function (analogous to fixing a function with `□FX` before using it elsewhere) to fill the current path with the selected pattern. Pattern 0 does nothing, patterns 1 to 4 are progressively darker greys, then the fun begins! To get a diagonal shading, we first fill the path with white (using `1 setgray fill`), then set it to be the clipping region, and simply shade the whole page! The same strategy works for all the obvious cross-hatchings, which is why I haven't bored you with the rest of the code.

Markers are dealt with in a very similar way. First we need a polymark routine:

```
/pM % Polymark: stack is array of [x,y] pairs; colour marker-type
{ setmarker setcolour
  /xy exch def
  /npts xy length 2 idiv def          % halve the shape
  xy aload pop gsave                 % disclose it
  0.3 setlinewidth                   % set correct weight
  [] 0 setdash newpath               % and solid lines
  npts { moveto currentmark } repeat % loop for each group
  grestore
} def
```

Then we need to fix ourselves a **currentmark** function for repeated use at each x/y pair:

```
/setmarker % set current graphics marker : stack is int [1..15]
{ /case exch def

  case 1 eq { /currentmark % cross
    { -1 -1 rmoveto 2 2 rlineto 0 -2 rmoveto
      -2 2 rlineto stroke } def } if

  case 2 eq { /currentmark % plus sign
    { -1 0 rmoveto 2 0 rlineto -1 -1 rmoveto
      0 2 rlineto stroke } def } if
```

.... and so on

Note that the markers are careful to use *relative* moves and lines; this is a typical PostScript device to make such functions re-usable. Finally, font selection and size:

```
/settext % set text parameters: colour, Size in pts, SSID (0-34)
{ /ssid exch def /size exch def setcolour
  ssid 0 eq { /Times-Roman findfont } if
  ssid 1 eq { /Times-Italic findfont } if
  ssid 2 eq { /Times-Bold findfont } if
  ssid 3 eq { /Times-BoldItalic findfont } if
  ssid 4 eq { /Helvetica findfont } if
  ssid 5 eq { /Helvetica-Oblique findfont } if

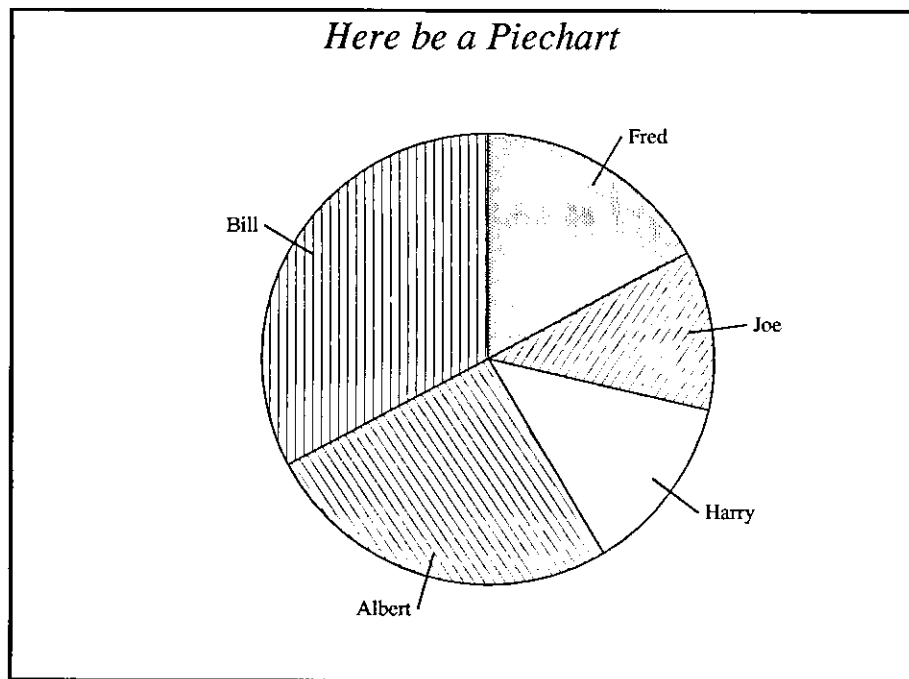
  ....

  ssid 32 eq { /Palatino-BoldItalic findfont } if
  ssid 33 eq { /ZapfChancery-MediumItalic findfont } if
  ssid 34 eq { /ZapfDingbats findfont } if
  size scalefont setfont
} def
```

This has the effect of burying the font names (which you have to get exactly right, including the capitalisation) where I need never see them again.

Putting it all to Work

I would like to end by showing you a selection of examples, where possible with explanations of anything I have so far failed to cover. First, a simple piechart, with spider labels and a heading:



The APL code which generated this undoubted work of art:

```

▽ R←VPIC
[1]  ⍝ VECTOR PIE DEMO
[2]  CHHEAD 'Here be a Piechart'
[3]  CHXLAB 'Fred;Joe;Harry;Albert;Bill'
[4]  CHSET 'CBOX' ⋄ 'HMAR' CHSET 0 1
[5]  CHPIE 12 8 9 18 23
[6]  CHTERM
[7]  R←'PS PG  ⍝ TO SEE IT'
▽

```

... old PGF hands will recognise some of the syntax, and also the mnemonics like 'CBOX' to get the chart are boxed! The output from this function is a simple text vector of PostScript code:

```
newpath gsave % before we translate the origin
[ 4 4 422 4 422 316 4 316 ] 15 1 1.5 pL
closepath # stroke
/
40 4 translate

11 18 1 settext # 177 294 M # (Here be a Piechart) cshow

15 10 0 settext
185 150 105 28.3 90 2 0.5 wedge
[233.5 231.1 246.9 253.7] 15 1 .6 pL # stroke
249.9 250.7 M # (Fred) show

185 150 105 -12.9 28.3 6 0.5 wedge
[278.6 162.7 304.7 166.2] 15 1 .6 pL # stroke
307.7 163.2 M # (Joe) show

185 150 105 -59.1 -12.9 1 0.5 wedge
[261.5 94.5 282.7 79] 15 1 .6 pL # stroke
285.7 76 M # (Harry) show

185 150 105 -151.7 -59.1 5 0.5 wedge
[159.9 58.9 152.9 33.6] 15 1 .6 pL # stroke
149.9 30.6 M # (Albert) rshow

185 150 105 -270 -151.7 10 0.5 wedge
[103.9 198.5 81.3 211.9] 15 1 .6 pL # stroke
78.3 208.9 M # (Bill) rshow

grestore
showpage
```

... note that the '#' characters have nothing to do with PostScript ... they get treated as diamonds by my APL code, and are mapped to $\nabla TCNL$ when I write out the file for printing. With a bit more trouble I could get APL to infer these line breaks, and save myself the trouble of putting them in by hand.

... the bulk of the hard grind having taken place in *CHAPIE*:

```

▽ R←CHAPIE VEC;lab;CT;LAB;TG;TGS;CEN;RAD;ARC;HA;YL;XL;EXP;PEXP

[1]      a Piechart from given sectors

[2]      TGS←CHXLLAB ◊ TGS+((ρVEC),-1+ρTGS)ρTGS
[3]      R←' ' ◊ →(v/'VALU'CHSET)÷Cum

[4]      a Add sector values to pie tags ....

[5]      TGS←Lj(Rj TGS),((×*/ρTGS)/(ρVEC),2)ρ': ' ,Ljv((ρVEC),1)ρVEC
[6]      Cum:VEC←+VEC ◊ VEC←VEC×360÷-1+VEC ◊ VEC←0,VEC
[7]      PEXP←0[0,01×(ρVEC)+CHPEXP
[8]      XL←(-/CHAREA[3 1])-+/CHVMAR
[9]      YL←(-/CHAREA[4 2])-+/CHHMAR
[10]     R←(vCHLATT), ' setttext', □TCNL,R

[11]     a Find the centre, and a sensible radius ....

[12]     CEN←0.5×(XL,YL) ◊ RAD←0.35×XL\YL

[13]     a Build segments and tags as we go round .....

[14]     Loop:←lab+1+((-1+ρVEC)ρLoop),End,CT+1
[15]     HA←0.5×+/90-VEC[CT+ 1 0]
[16]     EXP←PEXP[CT]×RAD× 2 1 ◊HA×(◊1)÷180
[17]     ARC←(CEN+EXP),RAD,90-VEC[CT+ 1 0]
[18]     R←R,(CHFMT ARC), ' ',(v1+CHPATN), ' 0.5 wedge ', □TCNL
[19]     TG←(2 2 ρARC[1 2])+ 0.9 1.15 ◊.×ARC[3]× 2 1 ◊HA×(◊1)÷180
[20]     LAB←Lj TGS[CT]; ◊ LAB←(+/v\φ' '×LAB)+LAB ◊ →(◊ρLAB)+Skip

[21]     a ===== SPIDER TAGS =====

[22]     R←R,'[', (CHFMT TG), ']' , (vCHLATT[1]), ' 1 .6 pL # stroke ', □TCNL
[23]     →(180<90-HA)+Lft
[24]     Rt:R←R,(CHFMT 3 -3 +TG[2;]), ' M # (' ,LAB,') show', 2ρ□TCNL ◊ +Skip
[25]     Lft:R←R,(CHFMT -3 -3 +TG[2;]), ' M # (' ,LAB,') rshow', 2ρ□TCNL
[26]     Skip:CHKDEF←CHKDEF+5+2,CHPATN[1] ◊ CHPATN+1φCHPATN
[27]     End:←lab[CT+CT+1]
▽

```

The only utility function I need to describe is CH_{FMT} ...

```

▽ R←CHFMT X
[1]      a Format data to 1dpl and subst APL negatives.
[2]      X←0.1×\0.5+X×10 ◊ R←v,X ◊ R[(Rε'"'')/;ρR]+'- '
▽

```

Because my printer only resolves to $\frac{1}{300}$ of an inch, I reckon I am quite safe rounding distances to $\frac{1}{10}$ of a point. I then can just use the APL primitive format (actually it is ideal as it only shows decimal places if there are any, thus saving space), being careful to map the APL high minus to an ordinary dash.

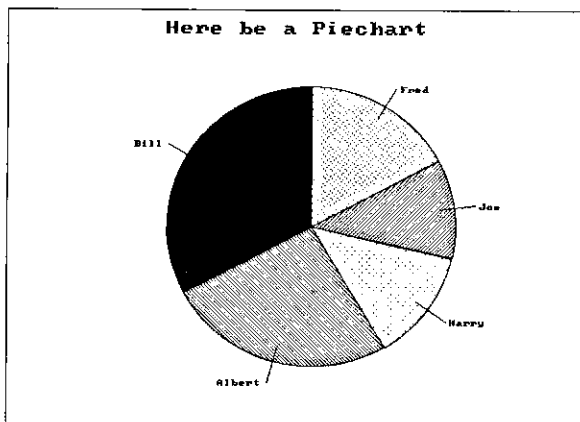
To put it on the screen, I simply run my DIY PostScript interpreter (I don't think I showed you *WEDGE* in APL):

```

V SPEC WEDGE DUM;XY
[1]  a Cover PostScript Wedge routine ... x,y,r,from,to,pat,width
[2]  1 GSAUSE SPEC[6] o SPEC[7]SETLINEWIDTH 0 o SPEC+~2+SPEC
[3]  XY+SPEC[1 2] o SPEC+(2+SPEC),1
[4]  a X-scale determines size of pie ... ratio is used as the ellipticity.
[5]  SPEC[1]+SPEC[1]*CTM[1;1]
[6]  SPEC[4]+CTM[2;2]÷CTM[1;1]
[7]  XY+2+(XY,1)+.×CTM
[8]  15 GSAWEDGE XY,SPEC
V
V CLR GSAWEDGE ARC;D2R;SEED;R;ASP
[1]  a Pass vector of pie wedge detail,...
[2]  a ARC: x, y, r, init angle, final angle (in degrees), aspect ratio
[3]  a The colour was set up by GSAUSE in GSADEF[1].
[4]  a
[5]  a{0=NC 'CLR')/'CLR'+15[8+GSADEF[1]' o a(2=GSADEF[6])/'CLR'+×CLR'
[6]  R+[]GMASK ~1 o D2R+180÷01 o ARC+6+ARC,1
[7]  ASP+1,ARC[6] o ARC+ARC÷ 1 1 1 ,D2R,D2R,GSADEF[5]
[8]  a Find a safe seed point in the middle of the sector!
[9]  SEED+ARC[1 2]+ASP×0.8×ARC[3]× 2 1 00.5×+/ARC[4 5]
[10] ARC[1 3]+GSADEF[5]×ARC[1 3]
[11] CLR []GCIRCLE 1 1 6 pARC
[12] a Fills with GSADEF[1] ... edges with CLR
[13] GSAFPAINT SEED,CLR
V

```

... and the result looks a bit like this (what I get on my DeskJet when I hit *QPRINT* from APL*PLUS):



I get something rather similar from my HP plotter, only here the functions build an HPGL string instead of a PostScript string, for example:

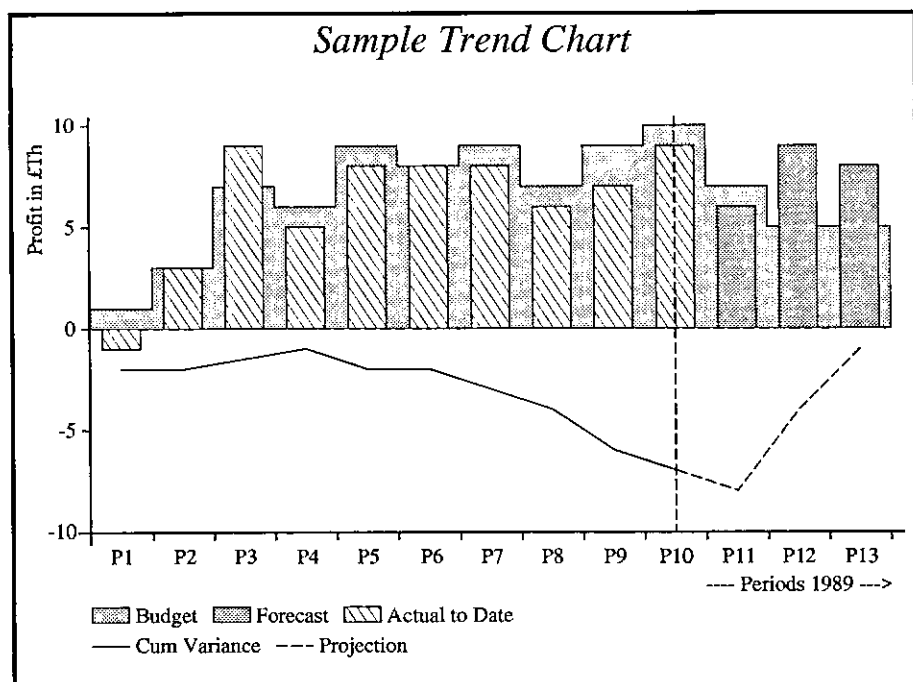
```

▽ TXT RSHOW DUM;XY
[1]  * HPGL version of () rshow
[2]  XY+2*(CPT,1)+.*CTM
[3]  HPGL+HPGL,'PA ',(▽XY),';'
[4]  HPGL+HPGL,'LO7;LB',TXT,'_ ',n1
▽

```

... and of course the colours work rather better!!

One final example, just to prove to those guys at MetaPraxis that anything they can do, I can do too....



Summary

I am now reasonably convinced that it will be hard to buy a printer without PostScript in 5 years' time. I suppose it will have to do Epson and HP emulation as well, but the simplification of having a single standard approach to both text and graphics is overwhelming. Just think how many fewer disks WordPerfect would come on if it only supported one type of printer! Accordingly, it makes sense to target your graphics on PostScript as the primary device, and let the rest take care of themselves. As it happens, it is pretty straightforward to convert the PostScript code to HPGL anyway (I think I have seen some public domain stuff which claims to do the whole job), but there are other benefits:

- the resulting graphic is amazingly compact. The whole of that MetaPraxis chart takes up a mere 2856 bytes of disk. If you saved it as a monochrome bit image, it would be well over 1Mbyte ... multiply by 16 for colour! As well as saving on disk space, this cuts down on network traffic, and gets the print done very much faster. It took GoScript a lot less time to print the high-quality version of my piechart than it took APL to do the screen dump!
- you can easily pull it back from disk and reshow it on the screen. In fact it is pretty trivial to define functions *ABOVE* and *BESIDE* to let you tile the screen with as many charts as you like ... obviously a handy thing for slide shows.

```
▽ R←A ABOVE B
```

```
[1] A Put two charts above and below
```

```
[2] R←'gsave # 1 .5 scale # ',B
```

```
[3] R←R,'# 0 316 translate # ',A,' # grestore '
```

```
▽
```

```
▽ R←A BESIDE B
```

```
[1] A Put two charts side by side
```

```
[2] R←'gsave # .5 1 scale # ',A
```

```
[3] R←R,'# 422 0 translate # ',B,' # grestore '
```

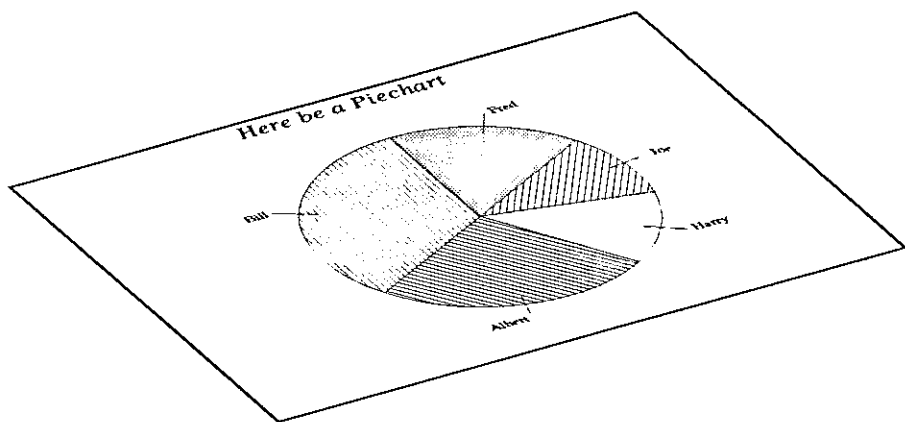
```
▽
```

- the results really are publication-quality! If you have a half-way decent word-processor, you can link them in to your final copy, and again the print file is far more compact than it would be if you link in bitmap graphics (such as the output from PC-Paintbrush).

Of course, there are one or two snags! PostScript is quite happy with the idea of rotating anything you put on the page, but of course APL fails to rotate the text,

and you also come adrift on things like polybar, as this calls `GSHADE`, which expects to be given the corners of a rectangle, not a parallelogram!! As a final tour de force, here is my piechart again, this time prefaced with:

```
130 150 translate
.8 .4 scale
30 rotate
```



This works beautifully in real PostScript, but I would advise you not to attempt it with my APL version, except possibly for its amusement value at BAA meetings!

References

- [1] *POSTSCRIPT Language Tutorial and Cookbook*. Adobe Systems Inc. Addison Wesley, ISBN 0-201-10179-3
- [2] *POSTSCRIPT Language Reference Manual*. Adobe Systems Inc. Addison Wesley, ISBN 0-201-10174-2
- [3] *Fundamentals of Interactive Computer Graphics*. J.D. Foley and A. Van Dam. Addison Wesley 1982.

The BAA Visit Rowntree Mackintosh

reported by Sylvia Camacho and Jonathan Martin

The decision to hold some meetings away from London was amply justified by the success of this visit to York. About 20 chose to arrive mid-morning to take a tour of the factory and found themselves kitted out in white overalls, caps and hairnets before being allowed into the world of Kit Kat, Smarties, Polo mints and Black Magic. Attendance at the afternoon meeting also exceeded expectation and the speakers were very well received.

Why we use APL by Tom McGuffog (Purchasing, Logistics and IT director)

The theme running through this talk was that computers and the software which runs on them are too valuable to be monopolised by Data Processing departments. Rowntree has backed this judgement by moving its Operational Research facility away from IT management into an area now called Logistics. They argue that Rowntrees are looking for people interested in the problems they need solving and not those whose primary interest is in computers.

They look upon computer facilities as a way of adding value to bright people and use APL to help them articulate and prototype their problems and solutions. Experience shows that people who can make computers sing are often not good at messy business problems, and they also find that IT departments have inappropriate career structures.

Supply Chain Management

Mr McGuffog continued by describing how Rowntrees use computers in supply chain management. He pointed out that whereas the goods flow from the suppliers through warehouses and on to the customer, the data flow is naturally in the reverse direction from the (probably EPOS) checkout, through goods packaged in 'outers' by pallet loads, to the orders (often on Electronic Data Exchange), to the invoices and delivery instructions and so through to contract and production plan.

As the parties involved in these activities become more automated, the control demanded becomes more stringent. Tesco, for instance, will not only specify a day for delivering but a precise time to fit with their own warehouse throughput schedule.

The hardware Rowntrees use is becoming increasingly distributed, nationally and internationally. Mainframes are seen as file servers to mini computers (VAX) which service networked (Ethernet) micro computers (mostly IBM PCs). Data is held on a relational database (Oracle) from which it is pulled down and uploaded using SQL. This allows a fast and flexible access to data on 30-40 Brands presented in about 1000 varieties of pack. The packs change continually so that there may be 3000 in the course of a year.

Chocolate Monitoring demonstrated by Adrian Smith

Adrian was able to bring the factory into the lecture room by plugging into the live, real time chocolate making system enquiry and scheduling packages which monitors production for 6-8 ingredients and displays the activity of the various machines in 1/4 hour schedule slots.

The data is brought down to the PCs via SQL from a Vax ORACLE database. This delegates all problems of data integrating to the ORACLE software. It is a matter of horses-for-courses, ORACLE to handle large amounts of data but APL to manipulate it as arrays. Postscript is used for the graphics because it has a defined public standard. It is very important not to get locked into any particular vendor.

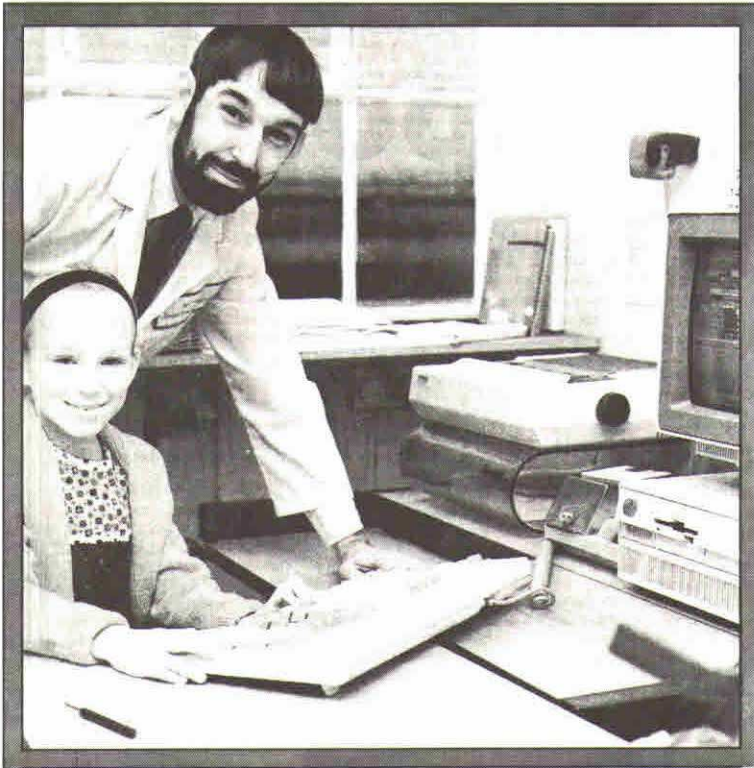
Medium Term Planning by Vicki Wright Interactive Planning Board by Andy Gibson

There has been a system from 1983 onwards. The object of medium term planning is to reduce stock-holding and improve service to customers and thereby increase value added to the raw materials. The system provides an interface between otherwise incompatible systems. In the spirit of the time the mainframe is gradually giving way to PC based systems.

Products are organised hierarchically, and frequent packaging changes impose many restraints on planning. The system provides an immediate response to any data edited. Colour is used extensively and standards for its use have been devised.

Data is taken weekly from the mainframe via a local VAX. The planners maintain their preferred product tree structure. They find that Export production tends to require bottom up planning, but the home market is better suited top-down. The front end is a short term schedule into which actual data is linked as it becomes available.

An unconsidered but intriguing trifle of information emerged from this talk - some Smartie tubes are abbreviated to 'Tooth' which is short for 'toothpaste tube', but they aren't tubes, they're cardboard boxes - quite square; they're for Irish Smarties! (Andy said "That's not a joke. Its true.")



It was also pointed out that for the first time an APL system had **made the front page** of the Company Newspaper! (captioned "Daddy - I want your job").

APL Training at Rowntrees by Richard Taylor (OR Manager)

The first contact Rowntrees had with APL was via time-sharing in 1976. The second system they set up (1979) after bringing it in house is still running. They used VM CMS and then VSPC and AP124, going in 1983 to GDDM & PGF graphics and occasionally to APL*PLUS/PC & APL 68000 on a SAGE IV. By 1986 they had over 150 mainframe users and by 1987 were beginning a serious move

to PC systems integrated with the company Ethernet and Dec's PCSA. By 1989 they could develop major systems on 386 machines as a consequence of the facility to copy the code and data easily (and quickly) over the network.

Rowntrees have a recruitment policy biased towards numerate graduates. They offer 8 weeks structured training, after which graduates are eased into existing systems via routine maintenance and simple changes. Where possible, graduates are given little 'stand-alone' projects as soon as possible, so that they can grow in confidence (and so that their inevitable mistakes cannot cause too much damage!).

Staff turnover averages 16% with an average service of 4 years. About one third of the recruits tend to stay in the Company, with a good spread about the different functions (e.g. the current head of I.T. is ex-OR, and there is ex-OR representation in Marketing, Production, and Product Development).

Don't Even Think About It!

a (sore) footnote by Jonathan Martin

Quite possibly those words don't mean anything to you. They are taken from the back page of the notice for the meeting at Rowntree's concerning methods of getting to York, in this case by air. But, hang on a minute, I work for an airline - flying is the most obvious (and most convenient for me) way of travelling north. So that's what I did.

I set off from home early Friday morning and arrived at Heathrow Terminal 1 in plenty of time, only to find that (because of bad weather up north) the plane was late arriving and would consequently be about an hour late departing. This was not good news - I had worked out my timetable (fly to Leeds, pick up a hire car and drive up to York) so that I didn't have a great degree of leeway. However, there was nothing I could do about it.

Eventually we boarded the plane to the wonderful smells of a British Midland cooked breakfast wafting around. Just before we pushed back, the Captain came on and announced that we had a problem - there weren't enough breakfasts to go around! How could he possibly resolve this situation? - send all the breakfasts back and make us do without!

Thirty minutes later and we began our descent into Leeds. The captain immediately warned us that the weather at Leeds had deteriorated and that we may not be able to land. Sure enough, after two attempts and not even a whiff of a view of the airport, we diverted to Teesside airport (near Darlington).

Things were now getting desperate as it was 11:30 and I was due in York at 10:30. So, off the plane, skip the coaches back to Leeds and straight to the Hertz desk to beg for a car. Fortunately I was one of the first there and they had some spares. So into the car and off into the wilderness of North Yorkshire, not altogether knowing where I was going except that I wanted to go South. Ninety minutes later I finally arrived at Rowntrees, just in time to catch the end of lunch. A very interesting but thankfully (with respect to the morning) uneventful meeting followed.

Back to the journey - the meeting finished at about 5:15 and my plane back from Leeds was at 7:05 - plenty of time. For a change, I made it quite easily, even managing to call in at my sister-in-law's for a cup of tea before going to the airport.

At the airport, all was still gloom and doom (in that order) - the fog had well and truly settled in and they had cancelled the evening flight to London. Instead they were going to bus everyone down to Heathrow on coaches! Now the M1 to London on a Friday night in the fog is not my idea of a good time - so back to my sister-in-law's I went and spent a very comfortable evening with a glass of gin and tonic...

First plane Saturday morning was 7.05! I woke just about in time and glanced out of the window - no prizes for guessing the state of the weather. So the options open were either to sit in Leeds and wait for it to clear (anyone who knows where Leeds/Bradford airport is will know that this could take a long time) or to drive over to Manchester and get on the shuttle. Manchester it was, over the M62, over the pennines.

Now if Leeds/Bradford is not the best place to put an airport, then the top of the Pennines is not necessarily the best place to put a motorway - needless to say, the fog was as bad (if not worse) on the M62 as it had been at the airport. So I crawled my way, in visibility down to about 20-30 feet in places, over to Manchester, where surprisingly there was bright sunshine (a rare occurrence!) and some flying aircraft. Thankfully, from then on normal service was resumed and I arrived home somewhat later and a touch more crumpled than I had expected.

Hindsight is a wonderful thing.

GENERAL ARTICLES

This section of VECTOR is oriented towards readers who may neither know APL, nor be interested in learning it. However we hope you are curious about how, under the right conditions, such impressive results can emerge so quickly from APL programmers

APL91 UPDATE

from Charles Schulz (December 1990)

Summary

Plans for the APL91 conference, to be held at Stanford University 4-8 August 1991, are well under way. We are meeting regularly to build this conference into a major event, conferring with our planning firm, APL user group contacts, vendors, and media contacts. Interest so far has been very encouraging: we have received over 250 interest cards to date, from more than 25 countries, and these continue to come in daily as distributions are reaching APL users worldwide. We have received nearly 60 papers, and are expecting over 90 on a wide range of topics and from a wide range of locations. 10 draft papers have been submitted from the USSR.

Call for Participation

Nearly 20,000 copies of the "Call for Participation", which included an interest card, were circulated at APL90 in Copenhagen, mailed directly to ACM SIGAPL members and SIGAPL local chapter members, and continue to be distributed by several user groups and vendors' mailings. Contact the conference office if you need copies.

Papers

About 100 "Information for Authors" instructions have been mailed to prospective authors indicated on the interest cards, and many of the papers have already been received. The official deadline for paper submission has passed, if you have special circumstances, please contact the conference office.

Program Committee

We are fortunate to have assembled an international and qualified program committee who will travel to California in February 1991 make final paper selections.

Speakers

We are exploring several possibilities for keynote speakers. We may also have room in the program for invited speakers on particular topics or applications. If you have particular ideas or nominations, please forward them immediately to the conference office.

Exhibits

We have had an amazingly strong exhibitor response, with over 35 vendor request for booth information! What new products are they working on?

Publicity Help Requested

Help spread the news about APL and APL91. Please send us media addresses and contacts from your industry or field, where APL91 announcements would be noticed.

APL: A Productive Language?

We hope to publish a bibliography of APL productivity references for the conference. APL is frequently cited for productivity advantages over other languages, now more clearly illustrated with function point analysis measures. However for most users considering using APL, actual references are harder to come by than general impressions.

We encourage you to send your specific and favourite references of APL productivity, describing comparative results with other methods, solutions not possible without APL, and for good measure, cases where APL turned out not to be appropriate. References can be from published papers or articles, public remarks, or results of company internal studies, with permission. Send your references to the conference office today!

Travel Negotiations

Our conference planning firm is actively negotiating conference travel discounts:

- Sub-compact rental cars will be available through Hertz for \$109.00 per week with unlimited miles.

- Delta Airlines has offered the conference 45% off of full coach fares and 5% off of the lowest super-saver fares on domestic travel. United and Pan Am have extended the conference 5% off first and business classes, 40% off full coach, and 5% off excursion classes for both international and domestic travel. Plan on booking all your travel arrangements through the conference travel office as this will benefit your conference on bonuses and further negotiations.
- Conference ID numbers for the above mentioned companies will be forthcoming in future news, or contact the travel office with desired itineraries for quotations: by telephone, toll-free in the USA, at 800-777-8630, or by Fax at 415-391-8634.

We are not yet ready to receive calls about travel, but will contact you when we are.

Spouse Tours

Spouse tours have been arranged for 5, 6 & 7 August. Monday, 5 August - California Wine Country Tour. Tuesday, 6 August - San Francisco Tour and Bay Cruise. Wednesday, 7 August - Muir Woods and Sausalito Tour.

Suggestions

As members of SIGAPL and the international APL community, this is YOUR conference. Please send us suggestions of how to make this conference even better. Postal or electronic submission is best.

Important

If you have not already, please use the interest cards from the "Call for Participation" to let us know of your plans. We are depending on those for much of our planning.

APL91 Conference Office
883 Santa Cruz Avenue
Menlo Park, California 94025-4658 USA
Tel: (415) 326-7781
Fax: (415) 326-3945

SEE YOU AT STANFORD FOR APL91!

Soviet APL: a Historical Outline

by Alexei I. Miroshnikov

This paper is a personal survey of the significant events in Soviet APL history. It is intended to give the audience a general view of the evolution and implementation of the Soviet APL interpreters, as well as outline the successive applied systems.

The Beginning

Academician N.N.Moiseyev was the first Soviet scientist to get acquainted with APL in 1975. At the University of Toronto he was astonished to see how easily and speedily his Canadian colleagues could produce programs and obtain necessary results using APL. On his return Moiseyev initiated the creation of the first Soviet APL interpreter.

The APL/BESM realization(implementation) on a powerful Soviet BESM-6 computer was undertaken by S.P.Prokhorov, who was at that time working at the Computer Centre of the Academy of Sciences of the USSR [1]. When Prokhorov left the Centre in 1980, the APL/BESM system was on the whole completed. It underwent test running at several computing centres including the Leningrad Research Computing Centre of the Academy of Sciences. Although APL/BESM was not to become a finished programming product, its development was the starting point for the Soviet APL interpreters.

Moiseyev's visit to Canada had one more effect, *viz.* the APL interpreter was incorporated into the CDC CYBER-172 software bought in 1978 for the former Leningrad Research Computer Centre, now the Leningrad Institute for Automation and Informatics of the Academy of Sciences of the USSR (LIAS). The APL 2.0 version, developed under the direction of J.H.Burrill at MIT, had for a long time been the only legally imported APL interpreter in the Soviet Union [2]. Since 1979 intensive application of the APL 2.0 version has been undertaken by the North-West Branch of the Central Research Economic Institute at the State Planning Committee of the RSFSR, at the mathematical simulation department headed by V.V.Ponomaryov [3]. Research associate A.I.Miroshnikov of the same department developed an intelligent graphic APL terminal, emulated on the mini-computer Wang-2200 for the CYBER-172 incorporated into the Leningrad computer network of the Academy of Sciences.

The APL story is closely linked with IBM. The Soviet Union was no exception in that respect. It was along with the IBM/360 computer system that APL found itself in this country. One of the largest computer development centres, the Computer Research Institute in Minsk, together with the Institute for Computer Machinery (Sofia, Bulgaria) used APL as a tool for analysis and synthesis of digital systems [4]. Inclusion of APL into the standard software for the Soviet computer series ES (compatible with IBM/360 and IBM/370 systems) was being considered. Unfortunately this upsurge came to nought, mostly due to the lack of adequate terminal equipment in the USSR and

conservative attitudes of the leading scientists. This was the period of general enthusiasm about structured programming in the USSR. Production of a series of APL-terminals for Bulgaria by the Minsk Computer Works was to become the *chord finale* at this stage of the APL history.

Independent Realisations of APL

Frederico Fellini (the great film producer) once said that there is always somebody in the world who is shooting Anna Karenina. In the Soviet Union at different times several groups of developers quite independently attempted to realize APL.

In the Moscow Energetics Institute (MEI) under E.T.Semyonova, APL was realized in the late seventies. APL/MEI was based on the DOS ES batch mode and involved APL-program preparation on cards [5]. Later A.Sokolov developed an APL terminal with an APL character set on the basis of the Soviet 7920 terminal. APL/MEI was one more variation of APL/360 and, as distinct from the latter, had a great number of errors. It is for this reason perhaps that this version was not applied anywhere apart from the one organization which had invested money in its development. Later the developers team from MEI rewrote APL anew in ... LISP. This APL was used only by the developers themselves.

About the same time at NIIMM in Yerevan (Armenia) an interactive system with APL as a command language was realized [6]. This system was to solve the problem of designing microprogram support for the central processor of the ES1046 computer that was then being developed. Again, for this APL version APL\360 was the prototype.

In the early eighties a group of specialists from the Siberian Department of the Academy of Sciences (Novosibirsk) was developing special software for image processing on the multiprocessor complex SM2/PS-2000 [7]. Having realized on parallel architecture a set of primitives for processing data arrays, the developers discovered all of a sudden that they had effectively realized a subset of APL. The report on this work was a great success at the first Soviet APL seminar in Obninsk in 1986. The developers were quite optimistic but further information on their work is lacking.

The town of Obninsk, not far from Moscow, had for a long time remained quite obscure. Intensive research in the field of nuclear energetics carried on in the town lent secrecy to any information about it. It is for this reason perhaps that nobody outside Obninsk knew about the APL application there. In the late seventies A.I.Kononov had adapted an APL\360 version to the ES series computers replacing APL characters by the Russian letters resembling the former in their outlines, similar to APL/BESM [8]. This version differed from the others in that it provided a multi-user mode of computer application. All the APL interpreters developed in the Soviet Union are single-user ones. Since the early eighties the APL SV system has been used in the teaching process (in the Central Institute for Skill Improvement) [9], which is an IBM VS APL version adapted to the Soviet computers [10].

In 1983 the "Radio and Communication" publishers issued a book by N.A.Magariu "The APL Programming Language" [11]. With no relation to the latter a team of

researchers and students from the Kishinyov state University undertook development of an APL interpreter for SM-2 mini-computers, compatible with HP computers. This version differs from APL/360 principally in that it makes it possible for the user to determine in advance the set of APL primitives he will use during the session, similar to the auxiliary processors specification by the user in IBM APL.

Publicity for APL

In 1984 the annual international conference on APL was held in Helsinki, Finland. After the conference ended K.Iverson, R.Bernecky and T.Seppala visited Moscow [13]. At that time Soviet APL users were isolated from each other and the visit of APL veterans remained unnoticed by most of them. Iverson was not to meet like-minded persons, but rather laymen of no great capabilities, which did not exactly make a favourable impression upon him.

After the journal "Vycheslitelnaya Tekhnika" had been published in 1990 "Computer Machinery" - the Russian "BYTE" with a circulation of 73 thousand copies) that was wholly devoted to APL [12], in the Soviet Union there made its appearance a team from the joint venture "Ural" that had developed its own APL-like language for IBM PC compatibles.

New Wave

The new APL wave in the Soviet Union is again connected with IBM. As had happened before, APL got into the USSR together with the IBM hardware. But this time it was a "strange" programming product APL*PLUS/PC developed by the American STSC, which was unknown in the USSR altogether. The position of APL*PLUS/PC was further consolidated due to the incredible popularity of the statistical graphics system STATGRAPHICS, realized in APL*PLUS among the Soviet users.

The first APL presentation for IBM PC was arranged by Professor P.I.Medow (the University of Toronto, Canada) at the Computer Centre of the Academy of Sciences in Moscow, and also LIAS in Leningrad in 1983. Moreover, Professor Medow presented his colleagues in Moscow and Leningrad with two APL computers MCM-70, which inspired lots of people to learn APL. Since 1983 the APL*PLUS/PC system, though illegally, has been applied at the North-West Branch of the Central Research Economic Institute [14].

The problem of intellectual property has a most dramatic influence upon computer science development in the USSR. The notion of intellectual property is not to be found in the Soviet legislation as yet, and thus in the USSR neither a Soviet citizen nor a foreign company have any actual rights to their own programming product. It goes without saying that the Soviet scientists and programmers are aware that by using illegal programming products they are breaking the law, but under the present conditions they are forced to violate the universal legal and moral standards as they are not able to acquire the necessary programs officially. In summer 1989 an international forum was held in Pereslavl-Zalessky near Moscow devoted to the intellectual property

issues in computer software. The world's leading firms, such as IBM, Microsoft, ICL, Xerox, Lotus, Ashton-Tate and others took part. At the organizers' invitation A.I.Miroshnikov represented the Soviet APL community - and this was to be the first act of international recognition of APL in the Soviet Union.

When in 1985 S.P.Prokhorov had at his disposal two new IBM XT computers, he had no problem in choosing the programming system - it could be only APL*PLUS. Prokhorov and N.Zejulinsky, working 16 hours a day during 3-4 weeks developed a system for effective control over the Stavropol region's agriculture (the same region where Gorbachev, the current president of the USSR, had worked). This successful development was reported in the central newspapers and in an all Union TV broadcast. Nevertheless the system did not gain popularity - there were too few organizations owning IBM PCs at that time.

In 1985 at the Computer Centre of the Academy of Sciences in Moscow A.V.Kondrashev realized an APL-interpreter version for SM-4, the Soviet version of DEC PDP-11 [15]. This APL interpreter was the first Soviet APL version to gain popularity and to be commercially successful. Similar to APL/BESM (Kondrashev had begun as a student in Prokhorov's team) APL/VTCAN makes use of the Russian spelling of system commands, functions and the names of variables. Seeking to make maximum use of the potentialities of a micro-computer with only 64K RAM M.Koganov and A.V.Kondrashev realized APL/VTCAN on a ROM-card inserted into the frame of the Soviet micro-computer DVK (the desktop version of the PDP-11) [16].

Having thus changed from swapping with a disk to using immediate ROM they managed to improve considerably the interpreter's properties. APL/VTCAN supports a 32-36K workspace and a symbol table of 64 names including local variables and labels, and is more suited to teaching, or to functioning as an intelligent terminal. Nevertheless it was used to carry out a few software projects: a modelling system for the BN-600 nuclear reactor diagnosis (A.Scomorokhov, Obninsk), the DIASTAT statistics program package (V.Vladimirov, Obninsk), the interface to RBD-MICRO (V.Neradovsky, Leningrad), and at least three different accounting systems (Sverdlovsk and Leningrad).

Birth Of The Soviet APL Community

In 1986 in Obninsk the first seminar-school was held, which has now become annual. A working team on APL, as yet informal, is being formed, that aims at popularizing APL and coordinating the development work dealing with it. The 1986 seminar was marked by great enthusiasm - many of the participants, working in the same town, actually didn't know each other. In 1988 the first Soviet organization dealing with production and popularization of Soviet APL products came into being - the scientific technical co-operative "Dialog".

In 1989 Kondrashev transferred APL/VTCAN to a more modern Soviet micro-computer Elektronika-85, compatible with DEC Professional 380. On the APL/85 version the workspace was enlarged four times (64K for functions and 64K for variables) and it became possible to use a 20M hard disk [17].

In 1989 a decision was taken in the Soviet Union governmental circles to reorganize four ministries dealing with computer development and production into one ministry with a unified policy in the field of computer machinery: following IBM. Consequently financing of projects based on computers compatible with HP and DEC products was limited, and the interest in any software except that for IBM-compatible machines fell dramatically. The only Soviet APL-interpreter found itself in a difficult situation.

Though there were a great number of attempts to realize the APL programming system, there were relatively few successful ones. At present there are practically only three teams in the USSR that are specifically engaged in developing applied programs in APL. The principal direction of the Obninsk team's work is developing teaching programs for APL/VIDEO, statistical programs (in the Central Institute for Skill Improvement), prototyping of algorithms and modeling system development for nuclear reactor diagnosis (in the Institute for Physics and Power Engineering). The team of scientists from the Computer Centre of the Academy of Sciences (Moscow), which is using APL*PLUS/PC, specializes in developing programs for simulating economic processes, with wide application of graphics.

At the Central Research Economic Institute (Leningrad) and LIAS (Leningrad), APL is used as a practical tool for developing DSS and algorithm models of economic systems and processes [18]. These models serve as a basis for the Institute's work, providing the leadership of the GOSPLAN of Russia and the Planning Committee of Leningrad region with results of the analysis and forecasts of economic development. The Leningrad team of users has always had at its disposal powerful hardware and quite modern software: CDC APL 2.0 and STSC APL*PLUS.

It has developed drivers that make it possible to use Cyrillic in different APL versions, as well as realizing full adaptation for application in the USSR of the STATGRAPHICS statistical graphics system. The experience they have acquired enables the team, together with the system for preliminary planning, to prepare for object construction designed for commercial use on the Soviet Union territory [19].

Nowadays two APL systems are mainly used in the USSR: APL/VTCAN and APL*PLUS/PC. APL/VTCAN versions are used on SM-4 series minicomputers (about 40 copies of it have been installed) and on different versions of DVK-3 microcomputers (more than 100 copies have been distributed). Unlike the Soviet APL version, which is rather difficult to copy, the APL*PLUS/PC system is copied and used uncontrollably. At least 20 users of this popular APL version are known to the author, half of whom work in Leningrad. Besides these, two multiuser APL systems are being used on mainframes: APL/VIDEO in Obninsk on an IBM/370 compatible and APL 2.0 on CDC Cyber-172 in LIAS in Leningrad. One more single-user APL/MEI system is being used in Moscow.

A Breakthrough

The year 1990 will go down into the history of the Soviet APL community as the year of the breakthrough. That year two Soviet representatives read their papers at the annual international APL90 conference in Copenhagen for the first time and joined SigAPL.

"The Russian Advance", supported by Finnish and Dutch national APL associations, was received rather optimistically. The world APL community was faced all of sudden by a strange APL user community, who managed to retain the APL-pioneers' enthusiasm while working in isolation and in rather strained conditions. It was this factor as well as the general interest towards the USSR that made ACM SigAPL support the idea of holding the APL92 conference in Leningrad, USSR.

In September the Union of the Soviet APL users was registered in Obninsk. ACM came forward with an initiative of establishing five free ACM Chapters (or Local SIGs) in the USSR, with the view to organizing the Soviet SigAPL as well.

On October 11-12 the first international meeting on APL in the USSR - the Finnish-Soviet seminar "APL and Application Software" - was held in Leningrad, which was a big success. 50 Soviet APLers and 26 participants from Finland and Sweden took part.

At the end of 1990 the first state company - InfoStroy - was established in Leningrad, whose aim is to develop and distribute APL-based application systems, and also to supply APL systems for different computer classes. At the same time the Leningrad APL Users Group was formed which is contributing to APL development in the USSR along with the Union of the Soviet APL-users.

Though APL can hardly be called a widespread programming language in the USSR, the great way it has grown in this country during the last year with the help of the international APL community gives us hope for success in the near future.

Bibliography

- [1] Prokhorov S.P., Prokhorova T.V., *System APL/BESM, Software for BESM6*, Moscow, CC AS USSR, 1976.
- [2] Miroshnikov A.I., Trojan A.I., *APL 2.0*. Publications on software, Leningrad, LSRCC, 1981, 170 p.
- [3] Golubeva S.V., Miroshnikov A.I., *APL: Programming Tools*, LIAS, Leningrad, 1982, 39 p.
- [4] Petkov A., Janev K., Bojanov K., *Programming System APL - the Effective Tool for Analysis and Synthesis of Digital Systems/Computing Machinery of the Socialist Countries*, Moscow, Statistics, 1979, Vol.5.
- [5] Semernova E.T., Semernov A.L., *APL/Programming Languages*, Moscow, MEI, 1979, Vol.2.

- [6] Shukurjan S.K., Topchjan S.A., Manukjan L.E., *Dialog System Using APL, Questions on Radioelectronics*, Series: Computing Machinery, 1983, No.8, pp.51-59.
- [7] Scholnik K.M., *Technological Support of the PS-2000 in the OBRAZ-System*, Theoretical and Applied Questions on Data Parallel Processing, Novosibirsk, CC AS USSR, 1984.
- [8] Malashinin I.I., Kononov A.I., *Implementation of APL on ES Computer*, Applied Informatics, Moscow, Finance and Statistics, 1981, Vol.1, pp.155-170.
- [9] Luksha O.P., Rejmarov G.A., *Instruction in Research Using Dialog Computing System, Progressive Forms of Work and Production Organizing*, Moscow, VIMI, 1988, pp.21-23.
- [10] Kononov A.I., *Concepts and Application of the APL Programming Language in SVM ES Operating System*, Moscow, CSRIatominform, 1989, 104 p.
- [11] Magariu N.A., *APL Programming Language*, Moscow, Radio and Communication, 1983, 104 p.
- [12] *Language for Personal Calculations*. Computing Machinery, Moscow, Knowledge, 1990, No.4.
- [13] Meetings and Conferences. APL Quote Quad.-1984.- Vol. 15, No.1.
- [14] Emanovskaja T.N., Miroshnikov A.I., *Using of PC in DSS for Long-Term Planning: in the Proc. of III Seminar on PCs and LANs*, Tbilisi, 1986, pp.227-228.
- [15] Kondrashev A.V., *Dialog Computing System APL/VTCAN*, Moscow, CC AS USSR, 1985, 66 p.
- [16] Barmin A.V., Koganov M.I., Kondrashev A.V. *DVK as a personal APL computer*. Microprocessor devices and systems. 1990, No.4, pp.15-20.
- [17] Kondrashev A.V., *The Family of Soviet APL Systems.: APL90 International Conference Proceedings*. ACM, 1990, pp.214-216.
- [18] Emanovskaja T.N., Miroshnikov A.I., Hanenko V.N., *Design Organizing of Information Support System for the Scientific and Technical Development in a Region: Problems of Integral Production Automation*, Moscow, Science, 1988, pp.47-57.
- [19] Miroshnikov A.I., *Algorithm Alterable Models and APL: APL90 International Conference Proceedings*, N.Y., ACM, 1990, pp.272-274.

Object Oriented Programming and APL

by Zdenek V Jizba

It has been frequently suggested that software development lags behind the fast advancing technology of hardware. The chief reason for such statements is the difficulty in development of new, ever more complex applications. Our software tools often fail to take the fullest advantage of state of the art in our hardware.

To be sure, a great deal of effort is being placed on development of our software tools. Improvements in programming languages, as well as new techniques in programming are constantly being introduced. These often receive much publicity. Examples of improvements in software efficiency, or breakthroughs in problem solving include the following: "Structural programming" "Prolog and Artificial Intelligence", "Neural nets" and more recently "Object Oriented Programming (OOP)".

Object Oriented Programming in particular, has been of considerable interest to the APL community. This is attested by a number of published papers and by proposed conference themes. Before we turn our attention to OOP, it is useful to refresh our memory about the evolution of computer programming languages.

Most programming languages are invented by an individual or a group of individuals, who have some concept of how one should communicate problem solving to a computer. The commonly used name for this concept is "Paradigm". The table below is an incomplete list of distinct paradigms for a variety of languages.

PARADIGM	CHARACTERISTICS	LANGUAGE
VonNeuman architecture	efficient translation to machine language	Fortran, C
Compact notation	array processing	APL
Object Oriented Programming	Class Hierarchy	Smalltalk
Tree structures	List processing	Lisp
Predicate Logic	Horn clause	Prolog

When a programming language becomes established it seldom remains unchanged. As new demands demonstrate weaknesses, it begins to evolve. What originally may have been a simple concept, becomes more complex as more features are added to it. Compare, for example the first version of APL to the most recent releases of APL with nested arrays.

If there are those who feel that once a language has proven itself, it should become cast in concrete, they should realise that this is the path to oblivion. Programming languages have been invented to solve specific classes of problems. Today, many of these problems have been solved, as a visit to any PC software store will demonstrate. The problems that have not yet been solved, and these are many, pose a great deal of challenge. We need the sharpest possible tools for their solution. Perhaps the best way to improve our tools is to continue evolving the power and productivity of our programming languages.

Looking at the table of paradigms, one should note that each one tends to have its strength in one particular domain of problem solving. VonNeuman languages are good for efficient usage of existing hardware architectures. APL is good for compact expression of solutions to mathematical and business problems. Smalltalk excels in the handling of graphical displays. Lisp and Prolog are strong in problems dealing with logic.

A significant trend in recent times has been the merging of two or more paradigms into a single language. A good example of this is C++, which is a VonNeuman type language, that has been enhanced by the addition of Object Oriented Programming features. Even APL has not escaped this trend, as we could note by studying Nial.

If there is a weakness to APL, it would be its lack of expressiveness to display graphical images. This shortcoming as well as the need to extend features that ease development of AI applications has prompted me to explore the potential of Object Oriented Programming as a possible paradigm to augment current APL.

In this communication we will explore three questions. What is an Object Oriented language? Can APL be used in an Object Oriented programming style? What would it take to make APL an Object Oriented Programming language?

What is an Object Oriented Programming Language?

The concept of Object Oriented programming was developed by the creators of Smalltalk. This paradigm is rooted on four fundamental ideas:*

1. Abstraction
2. Encapsulation (information hiding)
3. Inheritance
4. Polymorphism (operator overloading)

We will try to explain each of these concepts with simple examples. However, these turn out to be only properties of a more fundamental concept, namely that of an object. The abstract concept of an object cannot exist in a vacuum. Every object must belong. Indeed the name of the item to which an object belongs is called a class.

The definition is circular. A class is also an object, and as such it must belong too. This approach leads to what is known as a "Class hierarchy". At the highest level of this hierarchy is the class known as Object. (To complete the circularity of the definition the class Object in Smalltalk belongs to a metaclass, but this principle can be ignored in elementary expositions.)

How does one develop programs in Smalltalk? Every object, or a class (there is a distinction between these two concepts, which we will for the time being ignore), can receive messages, and act upon them. Indeed messages are the ONLY means of communication with objects. A message can be in the form of a request, or can be a command to do something.

At this point it might be useful to pause in the abstract explanations, and to give specific examples. In APL the following expression is simple and straight forward:

FOOTNOTE: Object Oriented

Programming has its own vocabulary of technical words. We will use underlined text to emphasise these. Smalltalk, in turn also uses special conventions to describe its components. Classes, for example are expressed as proper names with upper case first letter. As an example, object and Object are related but distinct concepts.

$$5 + 2$$

7

We have a dyadic primitive function that takes a left and a right argument. The function then evaluates the expression producing the result (7). In Smalltalk, this same expression produces the same

result, but the interpretation is completely different:

```

      5      +      2
object selector parameter
      [ message ]

```

The object (integer 5) receives the message to add two to itself. The primitive function + is called a method (or selector in this special case), which the object called an integer understands, and knows how to process. Why is such a distinctive point of view so important? Because it leads to a totally alien (from the point of view of APL) way of problem solving. This is illustrated by the next two examples:

In APL the following expression is very similar to the first one:

```

      5.2 + 3.1
8.3

```

We have the same primitive function in a dyadic expression. In Smalltalk, we are dealing with a different object, namely (real 5.2). The method has the same symbol, but is evaluated in a different way. The property of assigning the same symbol (name) to a method in different contexts is called polymorphism. As a Smalltalk user would put it, we are overloading the plus operator. (Overloading is but one example of polymorphism). The next example is even more revealing:

```

      5 > 2
1

```

In APL we are dealing with another dyadic function, but in Smalltalk things are totally different. The object (integer) does not know how to evaluate >. The reason is that the symbol > is evaluated in exactly the same way for an integer as for a real number. So why should there be two copies of a function that does the same thing to objects in different classes (integer and real)?

The solution in Smalltalk is to recognise that both integer and real are subclasses of another class called Number. Therefore when the object 5 sees the method > and fails to recognise it, it seeks to find that method in its superclass namely Number. As it turns out, the class Number also fails to find this particular method. It seems that there are classes other than numbers (e.g. date) for which the > symbol has a meaning.

Not finding the method > in class Number, Smalltalk goes to the next superclass which is Magnitude, and discovers that this class knows how to proceed. This

process of searching up the higher levels in the class hierarchy is called inheritance.

We should not leave this brief and admittedly sketchy description of Object Oriented paradigm without mentioning the other two attributes, namely Abstraction and Encapsulation.

In principle, at the most fundamental level, Smalltalk could be defined by only one class, namely that of an Object. But now if we want to solve arithmetic problems, we must define those methods (addition multiplication ...) that are needed for this purpose. Since there are many types of problems where these primitive functions are not used, it does not make sense to define addition in the class Object. What Smalltalk allows is to define a new class, a subclass of Object.

This subclass is to contain all of the methods that may be needed to solve the problem. However, before doing that, it is important to evaluate the possibility of other similar problems that might not be of arithmetical nature, but which may need some of the same primitive methods. This is the rationale for defining Magnitude as a subclass of Object, Number as a subclass of Magnitude, and Integer as subclass of Number. This feature of segregating and classifying objects by common denominator methods (and sometimes even variables) is called abstraction.

The attribute called encapsulation is a natural consequence of abstraction and the usage of messages as the only means of communication. An object will respond only to a valid message. That message may request access to either a function (method) or data. Neither the method nor the data (there may be exceptions but this is not relevant for this elementary discussion) is available outside the object. This idea that functions as well as information is strictly local to each object is called encapsulation.

Earlier we treated objects (not to be confused with the class Object) and classes as the same. This is not quite true. When a new class is defined, it contains a special method, called the class method, that has the capability of generating specific instances of that class called objects (there are also class variables). Each instance is in some ways a copy of the class, in the sense that it can respond to messages, store data and produce results. Instances are like leaves on a tree. They exist, do a useful function, and fall off when no longer needed. Leaves, unlike branches (classes), however, cannot bud into new branches (subclasses).

Can APL Be Used in an Object Oriented Programming Style?

The brief answer is: certainly. It is always possible to develop a workspace (or a collection of workspaces) together with suitable files to simulate any particular paradigm. However, some characteristics of Object Oriented Programming might be more difficult to produce than others. Consider encapsulation: we would have to be able to generate arrays that contain local functions and variables. Clearly APL does not have such facility.

On the other hand, by allowing functions and variables to be stored in files, and by providing suitable cover functions (also to be stored in files) we can simulate that kind of an effect. If this approach is inefficient, we could release the requirement of encapsulation, by allowing class functions and variables to be global. However, the price of that would be a restriction on other aspects of OOP such as polymorphism. Consider the following example:

We define an APL (APL2 would be better) function called *PLUS*. This function behaves in every way the same as the primitive +:

```
5 PLUS 2
7
PLUS/2 3
5
'F' PLUS '1'
10
```

The last example shows that *PLUS* is overloaded in the sense that it also evaluates hexadecimal character strings. But now, because *PLUS* is a global function, and not local to the class hexadecimal, we are no longer able to use it to define addition of character strings in systems with base other than 16.

With true encapsulation, we could generate objects (number systems) to arbitrary base such that function *PLUS* would be evaluated correctly. Let us sketch briefly how true encapsulation might be implemented in APL. To illustrate the approach, let us continue with the example of function *PLUS*. Let us assume that we desire to also work with octal notation. We will do it by creating objects using only a current version of APL.

What would such numeric objects look like? Since objects must have characteristics of functions as well as data, we have to decide whether to use defined functions or arrays. For the numeric example, we will use defined

functions, but later we will also show an example where a nested array can be used more efficiently. We start by defining the monadic function *OCTAL*:

```
OCTAL 8 10
```

We would like to use function *PLUS* to do addition in this number system:

```
(OCTAL 7) PLUS OCTAL 7 16
```

Since function *PLUS* is global, the result of *OCTAL 7*, the character '7' would be interpreted as hexadecimal '7' which is the same as decimal 7. The answer we would actually get is 'E' which is incorrect. In fact the symbol 'E' is not even defined in the octal system.

The APL OOP solution is to create class *Octal*, and to transfer the expression as a message to that class. (Note that function *OCTAL* is in caps, but class *Octal* is a proper name).

So far so good, but how does one go about insuring that the correct class method (APL function) will evaluate this expression? It is done under a special function called a "driver". A simple driver called *GO* is illustrated below:

```

      ∇ GO ;E
[1]  ⍝ Driver for OOP expressions
[2]  ⍵←' '
[3]  E+⍵
[4]  EVAL E
[5]  +2
      ∇

```

This niladic function is an infinite loop that simulates an APL prompt. Whatever is entered at the keyboard, becomes the character string (*E*) which is evaluated by function *EVAL*. The objective of function *EVAL* is threefold:

1. It has to determine the "domain of expertise" of expression *E*.
2. It has to create the object capable of solving the problem.
3. It turns over expression *E* to that object as a message.

Note that if *EVAL* merely executes *E*, function *GO* is "invisible" to valid APL expressions. This means that as far as APL expressions go, *GO* does absolutely

nothing, except use up extra computer time. (It would cause an error halt for any System Commands though.)

Let us take a closer look at function *EVAL*:

```

      ▽ EVAL E;D;DRIVER
[1]  A Driver for evaluation of non-APL expressions
[2]  D←GETCLASS E
[3]  GETDRIVER D
[4]  DRIVER E
      ▽

```

To see how this function works, consider the following expression:

```
(OCTAL 7) PLUS OCTAL 8
```

Function *GETCLASS* extracts first the tokens from that expression. (Here we use the word "token" to mean any character strings not enclosed in quotes.) The token *PLUS* is recognised as a function, but the token *OCTAL* is not identified. With the help of a dictionary, *EVAL* determines that *OCTAL* is associated with class *Octal*. This label is returned by *GETCLASS* to local variable *D*.

Next, function *GETDRIVER* loads into the workspace function *DRIVER*, a class method in class *Octal* (a polymorphic function). This driver contains in its header the names of all methods as well as variables that are associated with that class. *DRIVER* is then given the string *E* for evaluation. The details of the algorithm in *DRIVER* vary from class to class. Here we show only a portion of the header that is pertinent to the current problem:

```

      ▽ DRIVER E;...;OCTAL;PLUS....
[1]  A Driver for messages in class Octal
[...].
      ....
      ▽

```

Function *DRIVER* first retrieves from component files whatever methods will be needed to solve the expression. At this point expression *E* becomes an executable APL expression. On completion of its task, *DRIVER*, which is local to *EVAL* disappears together with all associated methods and variables.

In practice, this example is much too simplistic. Our next expression may also belong to the same class. It would make sense than to write the code in such a

way that a series of messages default to the same class (or instance of a class) until a new "domain of expertise" is called for. The simplest way would be to invoke function *GO* within *DRIVER*. This approach may be adequate for specific situations, but does not address the general problem.

Let us now look at another problem. In this next example, the OOP solution involves dealing with arrays. Consider the property of inheritance and the following nested arrays:

```

      JOHN
'NAME' 'John'
'CLASS' 'Relative'
...
'ISA' 'PERSON'
...

```

```

Person
...
'ISA' 'Human'
...

```

```

Human
...
'HANDS' 'two'
...

```

These arrays have been generated in a previous session when a particular family tree was stored under class Relatives. The arrays do not reside in the workspace, but will be loaded (from appropriate component files) in response to the message described below by function *DRIVER* which is specific to class Relatives.

To borrow terminology from the field of Artificial Intelligence, these arrays are called frames. The array called *JOHN* (an instance), has many rows and two columns (only those pertinent to this example are shown). The array *Person* (a class) shows only the row associated with class variable *ISA*, and the array *Human* (another class) shows the row associated with class variable *HANDS*. These arrays have been made available through functions *EVAL* and *DRIVER* as a result of examination of the following expression:

JOHN HANDS

FOOTNOTE: Since my work to date has been done under APL*PLUS, the above arrays would be stored in Component files as segmented strings.

The allocation of a proper domain of expertise may not be obvious in this case. Function *EVAL* may handle that with the following dialogue:

```
JOHN HANDS
What are we talking about? Relatives
two
```

(Before solving the expression, the user is asked to specify the appropriate domain of expertise. Entering 'Relatives' should be adequate.)

If John is an instance of class Relatives, we may find that the variable *HANDS* is not defined in that class. However the label *ISA* points to possible inheritance in class Person. The search for a response to message *HANDS* eventually will lead to the contents of line 'HANDS' in class Human.

The practice of naming first the object, followed by the message is unwieldy and too restrictive. With the power of APL, some additional parsing functions should enable us to provide a more human-like communication:*

```
How many hands does JOHN have?
two
```

Clearly, the array Person is more complex. Since it defines a class, it should contain additional information:

```
Person
'CMETHOD' '...'
'CARIABLE' '.....'
'ISA' 'Human'
'FUNCTION' '...'
....
```

CMETHOD, for example may specify an APL function to be invoked whenever an instance of class Person is to be generated. (There may be more than one class method, since Person may require responding to more than one type of message.) *CVARIABLE* may specify one or more names (and possibly contents) of class variables. Some of the class components such as *ISA* may also be transferred to each instance. These would be called instance methods and instance variables. (Instance methods and variables

FOOTNOTE: The attendees of my APL87 tutorial on the subject of Expert Systems had the option to take home with them a workspace and associated component files that illustrated this expanded notion.

may also be defined within an existing instance). *ISA* in instance *JOHN* is an example of a variable that points to a possible inheritance path. *FUNCTION* in *Person* could contain a list of local APL functions (methods) that will respond to messages sent to instances of class *Person*.

There is no implication that the approach illustrated in these two examples is the only or even the best way to handle Object Oriented programming with APL. It merely demonstrates the feasibility of using APL in that way.

Although the workspace approach to Object Oriented Programming may seem adequate, it is not. In a WINDOWS environment, such as that pioneered by Smalltalk, it is possible to re-enter a suppressed object by a mere click on the mouse. This would be roughly equivalent to being able to re-enter a defined function, and to reuse previously generated local variables. This is clearly not feasible with current versions of APL. The examples illustrated in this section could be made re-entrant, but only at prohibitive costs in execution speed.

What Would it Take to Make APL Object Oriented?

Developing Object Oriented programming style in APL is feasible, but requires building a superstructure of defined functions in a workspace. It also may require the generation of special component files to implement true encapsulation. The natural next step is to ask, if it would be possible to enhance APL so that the Object Oriented approach is built into the language.

In this section, we will consider such a modification, but do it with the caveat that it is not a proposal. At this stage, it is not clear whether the evolution of APL should take such a radical direction. We shall call the new version of this language APLOOP.

Since an object must have characteristics of both data and functions, it makes sense to use the $\emptyset$ symbol to define it. To distinguish an object (or class) from a defined function or operator, let us use the square bracket in the header:

```

      V RESULT ← [Human] MESSAGE
[1]   A Sample class Human
[2]   THUMB←'opposable'
      V

```

The header defines the usage of *Human* in a simple way. It is not adequate for applying the full range of Object Oriented programming as it stands, but is a

good start for a discussion. So far Human looks like a defined function. It is monadic and produces a result. Let us see how it would work:

```
Human THUMB
opposable
```

Even though Human looks like a function, we already see some differences. The right argument is a character string, but is not in quotes. In OOP terminology, the right argument is a message. This means that the right argument could be a query or an assertion about the object. In the above simplistic example, the right argument merely calls for the contents of variable *THUMB*. Is *THUMB* a local or a global variable? Well it is neither. If we write:

```
INC 'THUMBS'
0
```

Yet as we will see, *THUMB* must be available to objects outside the local domain of Human. This property of making data or functions unavailable except through messages is encapsulation. As indicated in the comment line, Human is not an object but a class. The variable *THUMBS* is a class variable.

Suppose now that we desire to add more characteristics to the class Human. We could do it by entering a new line:

```
[3] VISION + 'stereoscopic'
```

But wait, what is the domain of expertise of this application? If we are dealing with the subject of Natural History, should we not recognise that "stereoscopic vision" is possessed by all primates?

```
∇ RESULT + [Primate] MESSAGE
[1]  a Sample class Primate
[2]  VISION + 'stereoscopic'
∇
```

Certainly this is not only valid, but also a more general statement. We had to create a new class called Primate. How can we relate class Human to class Primate? We could do it by defining Human as a subclass of Primate. This could be done in the header for Human:

```

    ∇ RESULT ← [Human] MESSAGE;[Primate]
[...]
```

In OOP terminology, Human inherits VISION from Primate. Furthermore, Primate is a superclass of Human, and Human is a subclass of Primate. The structure tying the two classes together is class hierarchy. With this new definition, the APLOOP interpreter should work:

```

    Human VISION
    stereoscopic
```

Suppose now that we broaden the scope of discourse to cover individuals. To describe people, anatomy alone is inadequate. We are embedded in a civilisation and therefore we need another class hierarchy:

```

    ∇ RESULT ← [INSTANCE ← Culture] MESSAGE
[1]    a Class Culture with instance objects
[2]    FORM1
```

Culture belongs to yet another class hierarchy. It may have its own class variables, or even class methods. FORM1 is an example of a class method. It is encapsulated in Culture:

```

    [NC 'FORM1'
0
```

Before we can see what FORM1 does, we must explain the assignment in the square brackets of the header. The class Culture is a prototype of instances. The header merely describes the manner in which a new instance is to be generated. Here is how it would work:

```

    american ← Culture

    [RELIGION .....]
    [GOVERNMENT .....]
```

When Culture is invoked and assigned to "american", a new instance of class Culture will be generated, triggering the class method named FORM1. FORM1 is a defined APLOOP function that generates an object (instance of class) Window,

and prompts the user to fill out the form. Suppose that we enter "mixed" under *RELIGION*, and "democracy" under *GOVERNMENT*. Then we can enter:

```
american GOVERNMENT
democracy
```

Given this approach, how could one define function *FORM1* so that it will be available ONLY as a method in class *Culture*? One possible way would be to allow the following header in the function definition:

```
▽ FORM1;[Culture]
[...]
```

To instantiate an individual, we need to associate two distinct class concepts: anatomy and civilisation. To implement this complex structure we require a complex header:

```
▽ RESULT ← [INSTANCE ← Person [Culture]] MESSAGE;[Human]
[1]  a Sample class Person
▽
```

We could have expanded this definition by adding more lines. But in this example, let us assume that we desire to describe each individual in a unique way. We are trying here to show that to each instance it is possible to add any number of class variables (or class methods):

```
JOE ← Person american
```

This expression instantiates *JOE*.

```
JOE PHONE ← (222) 122 3333
```

We assign an instance variable named *PHONE* to *JOE*. To access the *PHONE* belonging to *JOE*, we enter:

```
JOE PHONE
(222) 122 3333
```

```
JOE VISION
stereoscopic
```

The message *VISION* asks for the contents of variable *VISION*. The search proceeds up the class hierarchy, and eventually finds it under Primate. (This might be difficult to do in Smalltalk, since we are dealing with two distinct paths of search: Culture and Human. The concept that supports a solution is called multiple inheritance. Smalltalk does not support multiple inheritance, but C++ does.

JOE VISION + right eye only

Note that the message, a character string, has no quotes. The variable *VISION* now becomes an instance variable. Since it is contained in the instance called *JOE*, it will take precedence over any value that may exist in the hierarchical chain:

JOE VISION
right eye only

This exploration of possible expansion of APL to include Object Oriented characteristics could be extended with even more details. However, the purpose here is merely to open a possible door for a sketchy view of what might be. It is conceivable that the price for extending APL in this direction is too steep, when compared to potential benefits. Whatever the future of programming, we who love to use APL should not ignore such developments as OOP; on the contrary, we should strive to become more knowledgeable in them.

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Hackers Corner: Check that Disk!	Adrian Smith	125
Technical Correspondence		126
Mice do it on the Mat	Ray Cannon	130
Problems for APL Buffs (3)	Zdenek V Jizba	136

Hackers' Corner: Check that Disk!

by Adrian Smith

One of the penalties of running your APL systems across a network is that you never quite know what is happening at the other end of the wire. I could be sitting quite happily in my planning system, having loaded up my data hours ago ... meanwhile someone has pulled a plug on the Vax that I fondly imagine holds my 'P:' drive! If I don't spot the situation until APL reports 'DRIVE NOT READY ERROR', I need to do some fancy error recovery to give the user the chance to save things temporarily on the PC hard disk.

A much better way is to ask DOS to check out the disk first, and thus to avoid having to back out of my save routine halfway through:

```

▽ R←CHECKΔDRIVE DRV;⊞ELX
[1]   A Use INT 33H to verify drive status ...
[2]   A Returns R[1]←-1 or 65535 for a dead disk
[3]   A     else R[1]←bytes per allocation unit
[4]   A     (e.g. 512 for floppy, 2048 for typical HD)
[5]   A
[6]   A Other values of interest are: [2]=disk size
[7]   A                                     [3]=bytes used
[8]   A                                     [4]=bytes free
[9]   A
[10]  DRV←'ABCDEFGHIJKLMNPOQRSTUVWXYZ\|'⊞DRV
[11]  ⊞ELX←'R+4+~1 ⊞ +0'
[12]  R←(4+ 13824 0 0 ,DRV)⊞INT 33 ⊞ +(R[1]⊞ -1 65535)⊞Err
[13]  R←(×/R[1 3]),(×/R[1 3 4]),(×/R[1 3],-/R[4 2]),×/3+R ⊞ +0
[14]  Err:R+4+~1
▽

CHECKΔDRIVE 'C'
2048 42366976 34584576 7782400

CHECKΔDRIVE 'E'
-1 0 0 0

```

The other returned information could be quite useful too ... for example to check that there is sufficient space on a floppy to hold the file you are about to save. Note also the allocation size ... it is quite pointless to set the APL*PLUS/PC file blocking to less than this; you are simply wasting time and saving no space at all. Many network drives allocate in units as high as 16K, so all those tiny little files are really taking 16K bytes at a time off someone's Vax! Be warned!!

TECHNICAL CORRESPONDENCE

From: Peter Branson

7th Nov 1990

A couple of final remarks, please, on David Piper's reply in Vector 7.2.133 to my letter of Vector 6.4.95.

He is partly right about the reason the original workspace search function ran so slowly; it was full of 'each pepper', but that was not the only reason. The author of the original had tried to please everybody by looking for all occurrences of the string in everything. By restricting this to functions and operators, and looking for only the first occurrence on each line, a further big saving was made. I find that this suits my needs 99 per cent of the time, and I rarely need to use a fuller version.

My second point takes us back again to the pornography, or otherwise, of dynamic function fixing, and I certainly can't agree with his use of the word 'ridiculous' here. However, I will agree that this technique can only safely be used on very rare occasions, and I contend that mine was one of these. A classic case of the wrong way of doing this occurs in IBM's ADRS2 code (particularly ADRS2BG) where, in some places, functions are redefined dynamically to 'fix' otherwise undetectable bugs!!! (Have you ever tried maintaining that code?)

David's suggested alternatives are fine in their way, but don't suit me. I don't use indirect copy for utilities because I do not want all the utilities cluttering every workspace I am working on; I have spent too much time in the past creating different lists and then forgetting to maintain them.

As regard packaging, I think this is the best way in the long term, especially for central maintenance, and we will probably move this way when (if?) we get some time to clean our act up a bit.

In the meantime I shall continue doing it my way, and I see that I am not the only one. Have a look at Tibor Horvath's *CYCLE* function on page 2 of APLBUG Newsletter, June 1990.

Packaged Workspaces in APL2/370

From: Nicholas Small

3rd December 1990

At the recent meeting at which packaged workspaces were discussed, one question (or, rather, a pair of questions) that was given only fleeting consideration was: are there objects that cannot or should not be packaged?

These questions cannot be answered without consideration of how packaging is to be done - if whole applications are packaged into single workspaces then the answer to both questions is, trivially, "no". If, however, utility functions are to be packaged separately, into a number of workspaces, then the answer is not so obvious.

To ensure that it is possible to package utilities, one must organize one's packaged workspaces so that no function which requires the use of *EXP* when invoked from a namespace different from its own is called from its own namespace (when the use of *EXP* could result in an incorrect reference). The reason for this restriction is that, so far as I am aware, there is no way in APL2 to determine from within a function the name of a calling function and hence whether or not the calling function is in the current name scope (not that this would be desirable, because of the complexity it would introduce); the alternative of using *QNS* and passing the identity of the calling name scope to the utility is hardly to be countenanced, as this would entail changes to the calling function.

Turning now to the desirability of packaging an object, I feel that any feature that requires more than a simple application of *EXP* should militate against packaging, because simplicity should be the order of the day. Consider, for example, trying to package the function *ISPEXEC*, which is supplied by IBM to provide access to ISPF Dialog Manager services (using AP317). Requests are passed as character strings, so calls to *ISPEXEC* to browse or edit a file are not a problem, but attempts to use *VPUT* or *VGET*, to transfer variables to or from the ISPF variable pools are a different matter - AP317 will expect to find variables in the namespace of *ISPEXEC* and will return values by creating variables there. A packaged *ISPEXEC* must detect that a *VGET* or *VPUT* request has been made and determine appropriate use of *EXP*, by analysing the request. I think that this is enough to indicate that *ISPEXEC* provides an example of a function that ought not to be packaged, because of the complexity of the changes that would be necessary.

Another topic of interest is that of utility functions and operators that need extending for use within packaged workspaces, rather than just modification to ensure that they will refer properly to objects in a calling namespace.

A simple example of such a function is the following, which endeavours to ensure that a variable is available, with an optional default value, and provides a result indicating whence the value came.

```

V Z←DEFAULT pWFINDVAR NAME
[1]  R Look for var NAME in current namespace; if not found, look in prev n/s
[2]  R If still not found, assign value DEFAULT if provided (else undefined).
[3]  R Z is set according to the source of any value for the variable:
[4]  R -1 - from prev n/s; 0 - undefined; 1 - from current n/s; 2 - default
[5]  + (0=⊖NC NAME)/0, Z+1
[6]  ± (0=EXP '⊖NC' NAME)/'+0, (Z+1), 0', NAME, '+EXP<NAME'
[7]  + (0=⊖NC 'DEFAULT')/Z+0
[8]  ±NAME, '+DEFAULT'
[9]  Z+2
V

```

This function is useful, for example, in a print utility for determining whether or not a printer has been defined.

PostScript Support for APL under Windows

From: George MacLeod

28 November 1990

There has been a lot of interest of late expressed in Vector on the subject of APL fonts and how to mix APL characters with other type faces in a word processing environment. These have ranged from Adrian Smith's elegant PostScript solution, to Rex Swain's technique for using the STSC APL*PLUS PC font in Microsoft Word 5. This latter is totally effective and very professionally documented.

We use Word for Windows for *Impetus* documentation as this gives much of the benefit of a Desk Top Publishing package with a lot less hassle, and is in any case, better suited to large documents. Word for Windows is substantially different to Word 5 in many ways including the fact that it lets Windows look after fonts and printers. This meant that Rex Swain's code did not help us.

Microsoft pointed us at a company called **Fontware Ltd** who are experts in these matters and who built an APL font some years ago for one of their customers. This font was built according to the customers specification and is mostly not very good. We were only interested in a subset of 12 APL characters, 10 of these were OK, and we had the other two modified. Fontware then mapped these on to key tops of our choosing.

The characters look very good when printed on a Laserjet. The screen font has, I think, been produced by some automatic process from the printer font, and is horrible. As the screen font is only for our internal usage, its quality is not a problem.

In summary, if anybody needs an APL font in a Windows environment, then Fontware have the rudiments of what is necessary and the knowledge to build something much better. I think all they need is a decent specification and a paying customer.

George MacLeod,
Impetus Ltd, Greymantle House,
Hempstead Road, Bovingdon,
Herts HP3 0HF

Ed: See Vector 7.1, July 1990, pp 96-107 for Adrian Smith's PostScript solution, and APL Quote Quad Vol 20, No. 3, March 1990, for Rex Swain's Microsoft Word 5 solution.

Mice do it on a Mat

or "How to make friends and influence rodents"

by Ray Cannon

All you need to know to use a mouse from within APL on a PC with examples using the STSC `□INT` system function. This will work on any version of APL*PLUS/PC, unlike `□MOUSE`, which is only available on release-10.

On an IBM (or compatible) PC with an attached mouse, special system software called the **mouse driver** provides an interface between the application program and the mouse hardware. The mouse driver can be installed (attached to DOS) at the time the system is configured. Alternatively the mouse driver can be installed in the form of a TSR (terminate and stay resident) program at run time. In either case, the driver should be installed before APL is invoked.

On the PC, system functions can be invoked from application software via interrupts. All that needs to be said is the CPU chip can support up to 255 different interrupts, and the standard (Microsoft) mouse driver is controlled and interrogated via interrupt number 51 (33hex). All parameters needed for this exercise can (and must) be passed to and from the mouse driver via the PC's registers.

So a method of setting the CPU's registers and then executing interrupt 51 is required to use the mouse. The `□INT` system function in STSC's APL can provide all the interfacing needed between APL and the mouse driver.

Using `□INT` is dead easy. (It took me less than 5 minutes to find out how, without a manual, by trial and error!) It takes two arguments: the right hand argument is the interrupt number (51 for the mouse); the left hand argument is a list of register values. The result `□INT` returns is a vector of the register values after the interrupt.

The facilities provided by the mouse driver allow you to position the mouse, set boundaries of mouse travel, read the mouse position, count the number of button presses (and releases), and show and hide the mouse. (Many other facilities are also provided, see Appendix 2 for the full list.)

But first you need to initialise the mouse. (Note: initialising the mouse is **not** the same as installing the mouse driver.) Initialising the mouse determines whether a mouse is attached, it resets the mouse driver (on reset, the mouse cursor is

hidden and placed in the centre of the screen), and returns the number of buttons on the mouse. It may be initialised from APL by executing:

```
0 □INT 51
```

The first element of the result should contain -1 and the second element the number of buttons on the mouse (2 or 3). All other elements will be zero.

The equivalent assembler code would be:

```
MOV AX,0 ; Put function 0 request in the AX register  
INT 33H ; Call mouse interrupt 51 (33 hex)
```

If successful, the driver will return -1 in the AX register and the number of buttons on the mouse in the BX register. The other registers should not be affected.

Next, to show the mouse cursor execute the code:

```
1 □INT 51
```

And in assembler:

```
MOV AX,1 ; Put function 1 request in the AX register  
INT 33H ; call interrupt 51 (33 hex)
```

To hide the mouse cursor (useful when scrolling text or between screens):

```
2 □INT 51
```

And in assembler:

```
MOV AX,2 ; Put function 2 request in the AX register  
INT 33H ; call interrupt 51 (33 hex)
```

Now to get the current mouse position and button status.

To the mouse, the screen is 640 pixels wide by 200 pixels deep. So all x coordinates are in the range 0-639 and all y coordinates 0-199.

Note these ranges apply to screen modes 0 to 16. Some newer high resolution modes return the actual screen pixel numbers. For example, VGA mode 18 returns 0-639 and 0-479.

3 `□INT 51`

The x coordinate will be returned in the 3rd element of the result, the y coordinate in the 4th element. The current button status is returned in the 2nd element.

The meaning of the button status value is as follows:

- 0 no buttons pressed
- 1 left button pressed
- 2 right button pressed
- 3 left and right buttons pressed
- 4 centre button pressed
- 5 left and centre pressed
- 6 right and centre pressed
- 7 left, right and centre pressed!

And in assembler:

```
MOV AX,3 ; Put function 3 request in the AX register
INT 33H ; call interrupt 51 (33 hex)
```

BX contains the button status, CX the x coordinate and DX the y coordinate.

If you want to set the mouse position to x:y, set the 3rd element to the x coordinate, and the 4th element to the y coordinate viz:

4 0 x y `□INT 51`

And in assembler:

```
MOV AX,4 ; Put function 4 request in the AX register
MOV CX,x ; put x position in the CX register
MOV DX,y ; put y position in the DX register
INT 33H ; call interrupt 51 (33 hex)
```

To get the button press information ...

```
5 0 □INT 51      a Left button
5 1 □INT 51      a Right button
5 2 □INT 51      a Centre button
```

These calls will return four values, the status of the requested button (see above), the count of the number of button presses (reset to zero on each call), the x coordinate at the last button press, and the y coordinate in the 4th element.

So in assembler for the left button only:

```
MOV AX,5 ; Put function 5 request in the AX register
MOV BX,0 ; left button request 0, right 1, centre 2
INT 33H ; call interrupt 51 (33 hex)
```

On return the AX contains the button status, BX the count of the number of presses, CX the x coordinate and DX the y coordinate.

For the button release information use function call 6 e.g. Left button:

```
6 0 □INT 51
```

To set the mouse movement limits use functions 7 (x coordinate) and 8 (y coordinate). This function is useful to "trap" the mouse within a window on the screen by "boxing" it in. (Hence the expression "window-box" or is it a "box-window"?):

```
7 0 xmin xmax □INT 51
8 0 ymin ymax □INT 51
```

And in assembler:

```
MOV AX,7 ; Put function 7(8) request in the AX register
MOV CX,xmin ; set x(y) limit minimum
MOV DX,xmax ; set x(y) limit maximum
INT 33H ; call interrupt 51
```

Lastly, did you know that the international unit of mouse movement is called a "Mickey", one mickey being about $\frac{1}{200}$ inch!

This should be all you need to know to build a simple mouse trap!

Appendix 1: Function Listings

▽ *MSΔINIT CU;RC*

```
[1]  # Initialize mouse ... sets global Δmouse+1 if OK ... else 0
[2]  # CU is the cursor type ... 0=mouse block; 1=hardware cursor
[3]  RC+0 □INT 51 ◊ Δmouse+0≠1+RC ◊ →Δmouse+0
[4]  →CU+0 ◊ RC+ 10 1 □INT 51
▽
```

▽ *MSΔSHOW;R*

```
[1]  # Show mouse cursor
[2]  →Δmouse+0 ◊ R+1 □INT 51
▽
```

▽ *MSΔHIDE;R*

```
[1]  # Hide mouse cursor
[2]  →Δmouse+0 ◊ R+2 □INT 51
▽
```

▽ *R+MSΔQCU;RC;XY*

```
[1]  # Query row/col for cursor
[2]  R+ 0 0 ◊ →Δmouse+0
[3]  RC+3 □INT 51
[4]  XY+RC[3 4]
[5]  R+φ[0.125×XY
▽
```

▽ *MSΔCU POS;XY;RC*

```
[1]  # Place mouse at row,col
[2]  →Δmouse+0
[3]  XY+8×φ2+,POS
[4]  RC+(4,0,XY)□INT 51
▽
```

▽ *R+MSΔQ BTN;RC;XY;CT*

```
[1]  # Query number of presses and row/col for button <btn>
[2]  # BTN+0 ... left; BTN+1 ... right; BTN+2 ... both together
[3]  R+3p0 ◊ →Δmouse+0
[4]  RC+(5,BTN)□INT 51
[5]  CT+RC[2] ◊ XY+RC[3 4]
[6]  R+CT,φ[0.125×XY
▽
```

▽ *R+MSΔQR BTN;RC;XY;CT*

```
[1]  # Query number of releases and cursor row/col for <btn>
[2]  # BTN+0 ... left; BTN+1 ... right; BTN+2 ... both together
[3]  R+3p0 ◊ →Δmouse+0
[4]  RC+(6,BTN)□INT 51
[5]  CT+RC[2] ◊ XY+RC[3 4]
[6]  R+CT,φ[0.125×XY
▽
```

▽ *MSΔWIN POS;XY;RC;SIZE*

```
[1]  # Set Mouse Window .... as per □WIN i.e. offset,size
[2]  →Δmouse+0
[3]  POS+4+,POS
[4]  XY+8×φ2+POS ◊ SIZE+8×φ2+POS
[5]  RC+(7,0,XY[1],XY[1]+SIZE[1])□INT 51
[6]  RC+(8,0,XY[2],XY[2]+SIZE[2])□INT 51
▽
```

Appendix 2

Summary of all known (to me) mouse interrupts (in HEX):

- 00 Initialize mouse
- 01 Show mouse cursor
- 02 Hide mouse cursor
- 03 Get mouse position
- 04 Set mouse position
- 05 Get "button press" information
- 06 Get "button release" information
- 07 Set mouse X limits
- 08 Set mouse Y limits
- 09 Set graphics cursor shape
- 0A Set text cursor type
- 0B Read motion counters
- 0C Set user defined event handler
- 0D Start light pen emulation
- 0E Stop light pen emulation
- 0F Set mickey-to-pixel ratio
- 10 Conditional cursor off (a "mouse hole" on the screen)
- 13 Set double speed threshold
No, I don't know what happened to 11 and 12
- 14 Swap user event handlers
- 15 Get "save state" storage size
- 16 Save mouse driver state
- 17 Restore mouse driver state
- 18 Set alternate mouse user handler
- 19 Get user alternate interrupt vector
- 1A Set mouse sensitivity
- 1B Get mouse sensitivity
- 1C Set interrupt rate
- 1D Set CRT page number
- 1E Get CRT page number
- 1F Disable mouse driver (the mind boggles!)
- 20 Enable mouse driver
- 21 Software reset
- 22 Set message language
- 23 Get message language
- 24 Interrogate mouse "We have ways to make you squeak")
- 2A Life, the universe and everything.

Reference

DOS Programmer's Reference, 2nd edition. Terry Dettmann. QUE.
ISBN 0-88022-458-4

Problems for APL Buffs (III)

by Zdenek V Jizba

The newsletter of our Southern California SIGAPL organisation is a one page sheet announcing the place and time of the next meeting. Since there is usually some space left, we try to include some APL puzzle or problem for our members to solve.

For the October 1990 meeting, I found an interesting reference [1, p.97] to a method that the ancient Greeks used to find an approximation to the square root of 2. They generated a table, or ladder as they called it.

1	1
2	3
5	7
12	17
29	41
:	:

The rule given for finding each new row is as follows. For the first column, add the numbers from the previous row. For the second column, add the number from the first column to the previous number from the first column.

The ratio of the second column to the first column converges to the square root of two. This follows from the relation:

$$\begin{pmatrix} 1 & 3 & 7 & 17 & 41 \end{pmatrix} * 2 = 2 * \begin{pmatrix} 1 & 2 & 5 & 12 & 29 \end{pmatrix} * 2$$

$$\begin{pmatrix} -1 & 1 & -1 & 1 & -1 \end{pmatrix}$$

which can be easily proven as shown in the APPENDIX.

An APL function was defined to generate this ladder. The function uses the recursion rule stated above to generate the $N \times 2$ array. Our SIGAPL members were asked to find another APL function that would produce the same result more efficiently. It was suggested to look for an APL function that would not be recursive.

In the debate that ensued, some including myself, thought that a non-recursive algorithm might be difficult to find. Others including Roy Sykes felt that there must be a "closed" form. As an argument he pointed out that Jim Brown (et al) in his textbook on APL2 [2, p.321] shows a simple formula for the Nth term of the

Fibonacci progression. This might have been the end of the discussion, since no one had any other comment.

About ten days before the November meeting, Roy who is our president, called me to ask if I could take his place since he was not going to be able to make it. My assigned task was to find a speaker, or to come up with a subject for discussion. He thought that perhaps I could come up with new information about the Greek ladder.

Not being able to find a substitute speaker on such a short notice, I decided to tackle the ladder problem. Right away, with some algebraic manipulation (see the APPENDIX), it can be shown that each of the two columns can be evaluated separately. Indeed, the formula is the same. If we call the array *LADDER*, we have:

$$LADDER[N+1;] \leftarrow LADDER[N-1;] + 2 \times LADDER[N;]$$

Armed with this information, one can easily define a general APL function called *PROGRESSION*:

```

      VU←V PROGRESSION W;I
[1]  ⍝ Calculate ladder with factor V[1], V[2] columns
[2]  ⍝ and seed vector W
[3]  I←1
[4]  U←1,[0.5],W
[5]  L0:←(V[2]≤I+I+1)/0
[6]  U←U,[1]U[I-1;]+V[1]×U[I;]
[7]  →L0
      V

```

The express `2 5 PROGRESSION 2 3` reproduces the Greek ladder. When the arguments to this function are `1,N` and `1`, the result is the Fibonacci progression of *N* terms:

```

      ,1 10 PROGRESSION
1 1 1 2 3 5 8 13 21 34 55

```

The reference given by Roy also can be used to produce function *FIB*:

```

      VU←FIB N;A;B;C
[1]  ⍝ Generate N terms of the Fibonacci series
[2]  C←1-A+0.5×1+5×0.5
[3]  B←5×-0.5
[4]  U←B×(×\NpA)-×\NpC
      V

```

Even though the values of A , B and C are irrational, function *FIB* generates a vector of integers:

```
FIB 10
1 1 2 3 5 8 13 21 34 55
```

Where can we go from here? We can look at the ratio of two consecutive terms. That ratio converges, and for the Fibonacci series we have:

```
-1+2÷/,1 50 PROGRESSION 1
0.6180339887
```

This number can be written as $(-1+5*0.5) \div 2$. Why stop just here? We can also evaluate:

```
-1+÷/,2 50 PROGRESSION 2
0.4142135624
```

And this number can be written as $-1+2*0.5$. With this as a clue it does not take much effort to find a closed solution to the Greek ladder problem:

```
∇U←LADDER N;A;B;C
[1]  A Generate N rows of the Greek ladder
[2]  C←2-A+0.5*2+8*0.5
[3]  B←8*-0.5
[4]  U←B*(×\NpA)-×\NpC
[5]  U←U,[1.1]1,2+U
∇
```

We do not have to stop here. Before we can proceed, however, we should note the following: the constants A and C in *LADDER* are the roots of a quadratic polynomial. (This was pointed out by Harry Bertucelli at the November meeting.)

Also note that the difference between a column of $(N \ R) \text{ PROGRESSION } W$ and $(N \ R) \text{ PROGRESSION } N$ is $(0, -1+, (N \ R) \text{ PROGRESSION } N) \circ . \times (W-N)$.

This can be seen best with:

```
(2 5 PROGRESSION 2 2 2)-2 5 PROGRESSION 13
```

We can therefore write a general ladder function:

```

      VU←N GLADDER W;A;B;C;D
[1]  a Generate N[2] rows of ladder with multiplier N[1]
[2]  a and seed W
[3]  D←4+N[1]*2
[4]  C←N[1]-A+0.5×N[1]+D*0.5
[5]  B←D*0.5
[6]  U←B×(×\N[2]ρA)-×\N[2]ρC
[7]  U←(ϕ(ϕN[2],ρW)ρU)+(¯1+0,U)×W-N[1]
      V

```

The property of the Greek ladder that can be used to estimate the square root of 1 can be written as the following APL expression:

```

      ((2 10 GLADDER 3)*2)-2*(2 10 GLADDER 2)*2
¯1 1 ¯1 1 ¯1 1 ¯1 1 ¯1 1

```

This is not generally true for arbitrary pairs of seeds. With some experimentation we found the following expressions:

```

      ((1 10 GLADDER 3)*2)-5*(1 10 GLADDER 1)*2
¯4 4 ¯4 4 ¯4 4 ¯4 4 ¯4 4

```

```

      ((3 10 GLADDER 8)*2)-13*(3 10 GLADDER 2)*2
¯12 12 ¯12 12 ¯12 12 ¯12 12 ¯12 12

```

One can also generalise the Fibonacci progression as an array that can be generated with function *GENFIB*:

```

      VU←ROWS GENFIB COLS;I;S
[1]  a Generalised Fibonacci array
[2]  I←1
[3]  U←1,[0.5]S+1 COLS
[4]  L0:→(ROWS≤I+I+1)/0
[5]  U←U,[1]U[I-1;]+S×U[I;]
[6]  →L0
      V

```

There is more but that is for you, the APL buffs to discover.

Consider the following problems:

1. The expression

$$^{-1} \uparrow, ((1 \ 50 \text{ GLADDER } 3) * 2) \div (1 \ 50 \text{ GLADDER } 1) * 2$$

generates the factor 5. Write an APL function that will generate the vector of integers $(F \ N \ M \ K)$ such that

$$((F \ P \text{ GLADDER } N) * 2) - K \times (F \ P \text{ GLADDER } M) * 2$$

produces $S \times ^{-1} * 1 \ P$. What would be the argument(s) to such a function?

2. The expression $,R \ P \text{ GLADDER } R$ generates a progression of real numbers when R is not an integer ($R > 0$).

Can you define a non-recursive function that will accept real R ($R > 0$)?

Will the resulting progression contain complex numbers?

3. In general, the expression

$$X \leftarrow ^{-1} \uparrow (((F, 50) \text{ GLADDER } N) * 2) \div ((F, 50) \text{ GLADDER } M) * 2$$

does not produce an integer, and the vector

$$((F \ P \text{ GLADDER } N) * 2) - X \times (F \ P \text{ GLADDER } M) * 2$$

does not produce a vector of the form $Y \times ^{-1} * 1 \ P$. Is X irrational?

References

- [1] *The world of Mathematics*. James R Newman Editor; Simon and Schuster, New York 1956
- [2] *APL2 At a Glance*. James A Brown, Sandra Pakin and Raymond P. Polivka; Prentice Hall, New Jersey 1988

Appendix

The original rule for building the Greek ladder can be expressed by the following relations:

$$\begin{aligned} [1] \quad & f(N) + f(N-1) + g(N-1) \\ [2] \quad & g(N) + f(N) + f(N-1) \end{aligned}$$

where $f(N)$ is the value in the N th row first column, and $g(N)$ is the value in the N th row second column. Replacing $g(N-1)$ in [1] with [2] we get:

$$\begin{aligned} & f(N) + f(N-1) + \{f(N-1) + f(N-2)\} \\ [3] \quad & f(N) + f(N-2) + 2f(N-1) \end{aligned}$$

To evaluate $g(N)$ in terms of $g(N-i)$, we first substitute [1] in both terms of [2]

$$\begin{aligned} & g(N) + \{f(N-1) + g(N-1)\} + f(N-1) \\ & + f(N-1) + g(N-1) + \{f(N-2) + g(N-2)\} \end{aligned}$$

but since $f(N-1) + f(N-2) = g(N-1)$, we get

$$[4] \quad g(N) + g(N-2) + 2g(N-1)$$

The property that makes the ladder useful in estimating square root of two also follows from [1] and [2]. Replacing $f(N)$ in [2]

$$[5] \quad g(N) + g(N-1) + 2f(N-1)$$

Replacing [2] and [5] in $g^2(N+1) - 2f^2(N+1)$

$$\{g(N) + 2f(N)\}^2 - 2\{g(N) + f(N)\}^2$$

and after simplification we get the recursion

$$g^2(N+1) - 2f^2(N+1) = -\{g^2(N) - 2f^2(N)\}$$

And since

$$g^2(1) - 2f^2(1) = 1 - 2 \times 1 = -1$$

we can see that

$$g^2(N) - 2f^2(N) = (-1)^N$$

A^{PL} Statistics Library

The ASL Project, sponsored by the British APL Association, and directed/supported by leading British Academic statisticians, announces the launch of three major volumes:

1. Elementary Statistics
2. Graphics
3. Regression and Generalized Linear Models

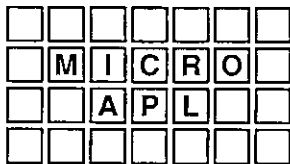
... available for most of the popular APL interpreters. Come and learn more at the Association meeting on Friday 22nd March at the IEE.

Talks and demonstrations of ASL will be given, and you will be able to try ASL for yourself, and to talk to members of the ASL team.

MicroAPL for ... Consultancy and Training

MicroAPL has 10 years' experience with small-computer APL systems - stand alone, networked or multi-user - and using all the major versions of APL. We provide a full range of consultancy services including feasibility studies, project management, programming, documentation and support services. Our programming skills cover all areas of APL and systems programming. MicroAPL also offer a comprehensive range of APL training courses.

For more details, contact David Eastwood at:



MicroAPL Ltd.
South Bank Technopark
90 London Road
LONDON, SE1 6LN
Telephone: 071 922 8866

Index to Advertisers

APL Statistics Library	142
APL People (half)	74
Cocking and Drury	60
Dyadic Systems Ltd	49
MicroAPL (2xhalf)	74,142
APL Booklist (Renaissance Data Systems)	32

All queries regarding advertising in VECTOR should be made to Alison Chatterton, at the address on the inside back cover.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (with diskette as appropriate) to the Editor:

Jonathan Barman,
Hill Top House,
East Garston,
NEWBURY, Berks RG16 7HD
Tel: 048839-575

(not after 10.00pm please!)

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members news) should be sent to Vector Production, c/o Adrian Smith, Brook House, Gilling East, YORK Tel: 04393-385 (6.00pm - midnight).

Product Guide updates should continue to go to Alison Chatterton, as should requests for advertising space.

BAA: Membership Application Form

Membership of the British APL Association is open to anyone interested in APL. The membership year runs from 1st May to 30th April.

Name: _____
Address Line 1: _____
Address Line 2: _____
Address Line 3: _____
Post or zip code: _____
Country: _____
Telephone Number: _____

Membership category (please tick box): 90/91

UK private membership	£10	☐
Overseas private membership	£18	☐
Airmail supplement (not needed for Europe)	£8	☐
Corporate membership	£85	☐
Corporate membership overseas	£140	☐
Sustaining membership	£360	☐
Non-voting student membership (UK only)	£5	☐

For student applicants:

Name of course: _____
Supervisor: _____
Signature of supervisor: _____

If you would like to pay by credit card ...

Card no (Access/Visa): _____

Signature (and Expiry date) _____

PAYMENT

Payment should be enclosed with membership applications in the form of a UK Sterling cheque to "The British APL Association", or you may quote your Access or Visa number. Please send the completed form to:

BAA Administration,
Alison Chatterton, 9 Oak Grove,
HERTFORD SG13 8AT
England

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1990/91 Committee

Chairman:	Peter Donnelly 0256-811125	Dyadic Systems Ltd., Riverside View, Basing Road, Old Basing, BASINGSTOKE, Hants RG24 0AL
Secretary:	Anthony Camacho 0727-860130	2 Blenheim Rd., ST ALBANS, Herts AL1 4NR.
Treasurer:	Nicholas Small 081-980 7870	8 Cardigan Road, LONDON E3 5HU
Journal Editor:	Jonathan Barman, 048839-575	Hill Top House, East Garston, NEWBURY, Berks RG16 7HD
Activities:	Dr Peter Branson 081-848-8989	Electronic Data Systems, Stockley Park, UXBRIDGE, Middx UB11 1BQ
Education:	Dr Alan Sykes 0792-295296	European Business Management School Swansea University, Singleton Park, SWANSEA SA2 8PP
Publicity:	Jonathan Martin 081-562 5697	(S499) British Airways, PO Box 10, Heathrow Airport, HOUNSLOW, Middlesex TW6 2JA
Technical:	David Eastwood 071-922 8866	MicroAPL Ltd South Bank Technopark, 90 London Road, LONDON SE1 6LN
Recruitment:	Jill Moss 0225-462602	APL People Ltd, The Old Malthouse Clarence St., BATH, Avon BA1 5NS
Projects:	John Searle 081-948 6737	13A Mount Ararat Road, RICHMOND, Surrey TW10 6PQ
Administration:	Alison Chatterton 0992-552489	9, Oak Grove, HERTFORD, SG13 8AT

Journal Working Group

Editor:	Jonathan Barman	048839-575
Secretary:	Anthony Camacho	0727-860130
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (04393-385)
Support Team:	John Searle (081-948 6737), Ray Cannon (0252-874697), Sylvia Camacho, Bridget Barman, Gill Smith	

Typeset by APL-385 with MS Word 5.0 and GoScript

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for 'A Programming Language' - an interactive computer language noted for its elegance, conciseness, and fast development speed. It is supported on many timesharing bureaux and on most mainframe, mini and micro computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

APL People

The Old Malthouse
Clarence St, BATH, BA1 5NS
Tel:0225-462602

Compass R&D Ltd
15 Frederick Sanger Rd
Surrey Research Park
GUILDFORD, Surrey GU2 5YD
Tel:0483 302249
Fax:0483 302279

HMW Computing Ltd
Hamilton House,
1 Temple Avenue,
LONDON EC4Y 0HA
Tel:071-353 4212
Fax:071-353 3325

Cocking & Drury Ltd
180 Tottenham Court Rd
LONDON, W1P 9LE
Tel:071-436 9481

REUTER FILE
1-4 Singer St
LONDON EC2A 4BQ
Tel:071-867 1166
Fax:071-867 9792

Intelligent Programs Ltd
Unit 7, Hermitage Court
6-10 Sampson St
LONDON, E1 9NA
Tel:071-481 4813

Dyadic Systems Ltd
Riverside View, Basing Road,
Old Basing, BASINGSTOKE,
Hants, RG24 0AL
Tel:0256 811125
Fax:0256 811130

APL Impetus Ltd
Fusper, Sandy Lane
Ivy Hatch, SEVENOAKS
Kent, TN15 0PD
Tel:0732-885126

MicroAPL Ltd
South Bank Technopark
90 London Road
LONDON SE1 6LN
Tel:071-922 8866

Peter Cyriax Systems
213 Goldhurst Terrace
LONDON NW6 3EH
Tel:071-624-7013
Mobile:0860-377963



The British Computer Society, 13 Mansfield Street, London W1M 0BD.